Concurrency & Parallelism

لمهندسي الذكاء الاصطناعي (Machine Learning Engineers)

مذكرة احترافية شاملة — من الأساسيات إلى مشاريع Production حقيقية

Python · Threading · Multiprocessing · Asyncio · FastAPI · Production Systems

الفهرس

الفصل الأول: مقدمة في Concurrency و Parallelism 11
لماذا نحتاج إلى Concurrency و Parallelism؟ 11
المشكلة التي يحلانها 11
كيف كانت الأنظمة تعمل قبل ظهور هذه المفاهيم 11
لماذا لم تكن الطرق القديمة كافية 11
المفهوم الأول: Concurrency 11
الفكرة الأساسية 12
كيف يعمل داخليًا 12
المفهوم الثاني: Parallelism 12
الفكرة الأساسية 12
كيف يعمل داخليًا 12
الفرق الكامل بين Concurrency و Parallelism 12
Synchronous مقابل Asynchronous، والتنفيذ التسلسلي Sequential Execution 13
أمثلة من الواقع 13
CPU-bound مقابل I/O-bound 14
CPU-bound Tasks 14
I/O-bound Tasks 14
كيفية اختيار النموذج المناسب 14
كيف تُستخدم هذه المفاهيم داخل الشركات الكبرى 14
أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي 15
متى تستخدمها ومتى لا تستخدمها 15
أفضل الممارسات، الأخطاء الشائعة، والاعتبارات الأساسية 15
اعتبارات الأداء (Performance) 16
الاعتبارات الأمنية (Security) 16
اعتبارات قابلية التوسع (Scalability) 16
مثال عملي صغير 16
مثال من مشروع Production حقيقي 16
مشروع صغير لتطبيق ما تم شرحه 17
تحديات شائعة 17
ملخص الفصل 17
الفصل الثاني: Process و Thread 18
المشكلة التي يحلها مفهوم Process 18
كيف كانت الأنظمة تعمل قبل مفهوم الـ Process المعزول 18
ما هو Process؟ 18
مكونات الـ Process 18
ما هو Thread؟ 19
Process Memory مقابل Thread Memory 19

Process Memory 19
Thread Memory 19
Process — Process Creation كيفية إنشاء 19
Thread — Thread Creation كيفية إنشاء 20
Thread — Thread Lifecycle دورة حياة الـ 20
Context Switching — التبديل بين وحدات التنفيذ 20
الفكرة الأساسية 21
كيف يعمل Context Switching داخليًا 21
اعتبارات الأداء لـ Context Switching 21
مقارنة كاملة بين Process و Thread 21
متى نستخدم Process ومتى نستخدم Thread 21
كيف تُستخدم داخل الشركات الكبرى 22
اعتبارات الأداء، الأمان، وقابلية التوسع 22
الأداء 22
الأمان 22
قابلية التوسع 22
أفضل الممارسات والأخطاء الشائعة 22
مثال عملي صغير 23
مثال من مشروع Production حقيقي 23
مشروع صغير لتطبيق ما تم شرحه 23
تحديات شائعة 24
ملخص الفصل 24
الفصل الثالث: Python GIL 25
ما هي المشكلة التي أدت لظهور الـ GIL؟ 25
كيف كانت المشكلة تُحل قبل الـ GIL 25
ما هو الـ GIL؟ 25
الفكرة الأساسية 25
كيف يعمل الـ GIL داخليًا 26
تأثير الـ GIL على الأداء 26
تأثيره على المهام CPU-bound 26
تأثيره على المهام I/O-bound 26
لماذا لم يتم تصميم الـ GIL بشكل مختلف من الأول؟ 27
كيف تتجاوز قيد الـ GIL عمليًا 27
مستقبل Python بدون GIL 28
كيف تُستخدم هذه المعرفة داخل الشركات الكبرى 28
أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي 28
أفضل الممارسات والأخطاء الشائعة 28
اعتبارات الأداء والتوسع 29
مثال عملي صغير 29

مثال من مشروع Production حقيقي 29
مشروع صغير لتطبيق ما تم شرحه 29
تحديات شائعة 30
ملخص الفصل 30
الفصل الرابع: Threading بالتفصيل 31
المشكلة التي يحلها التنسيق بين Threads 31
Race Condition 31
الفكرة الأساسية 31
الحل: Thread Synchronization وLocks 31
Lock — القفل الأساسي 31
RLock — Reentrant Lock 32
Semaphore 32
Condition 33
Event 33
Barrier 33
مقارنة أدوات التزامن 34
Deadlock 34
الفكرة الأساسية 34
Starvation 35
Livelock 35
كيف تُستخدم هذه الأدوات داخل الشركات الكبرى 35
أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي 35
متى تستخدم Threading ومتى لا تستخدمه 36
أفضل الممارسات والأخطاء الشائعة 36
اعتبارات الأداء، الأمان، وقابلية التوسع 36
الأداء 36
الأمان 36
قابلية التوسع 36
مثال عملي صغير 37
مثال من مشروع Production حقيقي 37
مشروع صغير لتطبيق ما تم شرحه 37
تحديات شائعة 38
ملخص الفصل 38
الفصل الخامس: Multiprocessing 39
لماذا نحتاج إلى Multiprocessing؟ 39
المشكلة التي يحلها 39
كيف كانت المشكلة تُحل قبل ظهور مكتبة multiprocessing المريحة 39
الفكرة الأساسية 39
Process Pool وPool Executors 40

المشكلة التي يحلها Process Pool 40
ProcessPoolExecutor 40
مشاركة البيانات بين Processes 40
Pipes 41
Queues 41
Shared Memory 41
Managers 42
مقارنة طرق مشاركة البيانات بين Processes 42
Synchronization بين Processes 43
الأداء — Performance وCPU Parallelism 43
كيف تُستخدم داخل الشركات الكبرى 44
أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي 44
متى تستخدم Multiprocessing ومتى لا تستخدمه 44
أفضل الممارسات والأخطاء الشائعة 44
اعتبارات الأمان وقابلية التوسع 45
مثال عملي صغير 45
مثال من مشروع Production حقيقي 45
مشروع صغير لتطبيق ما تم شرحه 46
تحديات شائعة 46
ملخص الفصل 46
الفصل السادس: Async Programming 47
المشكلة التي يحلها Async Programming 47
حدود Threading مع الحمل الضخم 47
كيف كانت المشكلة تُحل قبل انتشار Async 47
الفكرة الأساسية: Single Thread, Many Tasks 47
Event Loop 48
الفكرة الأساسية 48
كيف يعمل داخليًا 48
async، Coroutines، وawait 48
Coroutine 48
await 48
Tasks وFutures 49
Task 49
Future 49
Cancellation وTimeout 49
إلغاء مهمة — Cancellation 49
مهلة زمنية — Timeout 50
Async Context Managers 50
Async Iterators وAsync Generators 50

Structured Concurrency 51
كيف تُستخدم داخل الشركات الكبرى 51
أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي 51
متى تستخدم Async ومتى لا تستخدمه 52
أفضل الممارسات والأخطاء الشائعة 52
اعتبارات الأداء، الأمان، وقابلية التوسع 52
الأداء 52
الأمان 52
قابلية التوسع 52
مثال عملي صغير 53
مثال من مشروع Production حقيقي 53
مشروع صغير لتطبيق ما تم شرحه 53
تحديات شائعة 53
ملخص الفصل 54
الفصل السابع: concurrent.futures 55
المشكلة التي تحلها concurrent.futures 55
الفكرة الأساسية: Executor وFuture 55
Executor 55
Future 55
ThreadPoolExecutor وProcessPoolExecutor 55
()submit 56
()map 56
()wait 57
()as_completed 57
معالجة الأخطاء مع Futures 57
كيف تُستخدم داخل الشركات الكبرى 58
أين تقع داخل مشروع Machine Learning حقيقي 58
متى تستخدم أيهما 58
أفضل الممارسات والأخطاء الشائعة 58
اعتبارات الأداء وقابلية التوسع 59
مثال عملي صغير 59
مثال من مشروع Production حقيقي 59
مشروع صغير لتطبيق ما تم شرحه 60
تحديات شائعة 60
ملخص الفصل 60
الفصل الثامن: التواصل بين وحدات العمل — Communication Between Workers 61
المشكلة التي يحلها التواصل المنظم بين الوحدات 61
الأداة الأساسية للتواصل — Queue 61
Priority Queue 62

Message Passing 62
نمط Producer-Consumer 62
الفكرة الأساسية 62
Work Queue وWorker Pool 63
Task Scheduling 64
مقارنة الأنماط الأساسية 64
كيف تُستخدم داخل الشركات الكبرى 64
أين تقع داخل مشروع Machine Learning حقيقي 64
أفضل الممارسات والأخطاء الشائعة 65
اعتبارات الأداء، الأمان، وقابلية التوسع 65
الأداء 65
الأمان 65
قابلية التوسع 65
مثال عملي صغير 66
مثال من مشروع Production حقيقي 66
مشروع صغير لتطبيق ما تم شرحه 66
تحديات شائعة 67
ملخص الفصل 67
الفصل التاسع: تحسين الأداء — Performance Optimization 68
المشكلة التي يحلها القياس المنظم 68
Profiling — تحليل الأداء التفصيلي 68
الفكرة الأساسية 68
Profiling في الأنظمة متعددة الوحدات 68
Benchmarking — القياس المقارن 69
الفرق بين Profiling وBenchmarking 69
مبادئ Benchmarking الصحيح 69
CPU Utilization 69
Memory Usage 70
Context Switching Cost 70
Serialization Cost 70
Choosing Workers — اختيار العدد الأمثل 71
Performance Tuning — منهجية شاملة 71
كيف تُستخدم هذه المبادئ داخل الشركات الكبرى 71
أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي 72
أفضل الممارسات والأخطاء الشائعة 72
مثال عملي صغير 72
مثال من مشروع Production حقيقي 73
مشروع صغير لتطبيق ما تم شرحه 73
تحديات شائعة 73

ملخص الفصل 73
الفصل العاشر: Concurrency في FastAPI 75
لماذا FastAPI مبني على أساس Async 75
Async Routes مقابل Sync Routes 75
Async Routes 75
Sync Routes 75
Background Tasks 76
Worker Processes: Uvicorn وGunicorn 76
Uvicorn 76
Gunicorn مع Uvicorn Workers 77
Async Database وAsync Redis 77
File Uploads وStreaming 77
رفع الملفات 77
Streaming Responses 78
WebSockets 78
الهندسة المتكاملة: كيف تُستخدم داخل الشركات الكبرى 78
أين يقع هذا الفصل داخل مشروع Machine Learning حقيقي 79
أفضل الممارسات والأخطاء الشائعة 79
اعتبارات الأداء، الأمان، وقابلية التوسع 79
الأداء 79
الأمان 80
قابلية التوسع 80
مثال عملي صغير 80
مثال من مشروع Production حقيقي 80
مشروع صغير لتطبيق ما تم شرحه 81
تحديات شائعة 81
ملخص الفصل 81
الفصل الحادي عشر: البرمجة المتوازية في Machine Learning 82
Parallel Data Loading 82
المشكلة التي يحلها 82
الحل: Data Loading متوازي 82
Data Preprocessing وParallel Feature Engineering 83
Batch Inference وParallel Prediction 83
Distributed Training — المفاهيم الأساسية 83
Data Parallelism 84
Model Parallelism 84
Parallel Hyperparameter Search 84
GPU مقابل CPU Parallelism 85
الفرق الجوهري 85

كيف تُستخدم هذه المفاهيم داخل الشركات الكبرى 86
أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي — ملخص شامل 86
أفضل الممارسات والأخطاء الشائعة 86
مثال عملي صغير 87
مثال من مشروع Production حقيقي 87
مشروع صغير لتطبيق ما تم شرحه 87
تحديات شائعة 87
ملخص الفصل 88
الفصل الثاني عشر: أنماط الإنتاج — Production Patterns 89
Task Queue وJob Queue 89
Celery وRedis Queue 89
Scheduled Jobs — المهام المجدولة 89
Retry Pattern — إعادة المحاولة 90
Idempotency — ضمان عدم التكرار 90
Rate Limiting — تحديد معدل الطلبات 90
Distributed Locks — الأقفال الموزّعة 91
مقارنة الأنماط الأساسية 91
كيف تُستخدم داخل الشركات الكبرى 92
أين تقع داخل مشروع Machine Learning حقيقي 92
أفضل الممارسات والأخطاء الشائعة 92
مثال عملي صغير 93
مثال من مشروع Production حقيقي 93
مشروع صغير لتطبيق ما تم شرحه 93
تحديات شائعة 93
ملخص الفصل 94
الفصل الثالث عشر: تتبع الأخطاء في التطبيقات المتزامنة 95
اكتشاف الـ Deadlocks 95
تتبع أخطاء Threads 95
تتبع أخطاء الكود غير المتزامن (Async) 96
Logging في الأنظمة المتزامنة 96
Tracing — التتبع الموزّع 97
Profiling للكود المتزامن 97
استراتيجيات عامة لتتبع الأخطاء المتزامنة 98
كيف تُستخدم داخل الشركات الكبرى 98
أين يقع هذا الفصل داخل مشروع Machine Learning حقيقي 98
أفضل الممارسات والأخطاء الشائعة 98
مثال عملي صغير 99
مثال من مشروع Production حقيقي 99
مشروع صغير لتطبيق ما تم شرحه 99

تحديات شائعة 100
ملخص الفصل 100
الفصل الرابع عشر: دليل مرجعي شامل — Best Practices 101
خارطة القرار الأساسية: أي أداة أستخدم؟ 101
مبادئ التصميم الأساسية 101
قواعد الأمان في Threading 101
قواعد الأمان في Multiprocessing 102
قواعد الأمان في Async 102
قواعد الأداء العامة 102
أخطاء شائعة يجب تجنبها دائمًا 102
قائمة تحقق قبل نشر أي نظام Concurrent/Parallel للإنتاج 103
اعتبارات قابلية التوسع (Scalability Considerations) 103
خلاصة الرحلة 103
الفصل الخامس عشر: مشاريع Production متكاملة 105
المشروع الأول: خط معالجة صور عالي الأداء 105
البنية والأدوات المستخدمة 105
المشروع الثاني: نظام معالجة بيانات متوازٍ 105
المشروع الثالث: خدمة Batch Prediction موزّعة 106
المشروع الرابع: واجهة برمجية لمعالجة مهام في الخلفية 107
المشروع الخامس: خدمة Inference لحظية 107
النشر (Deployment) 108
Monitoring 108
خلاصة المشاريع الخمسة 108
تحديات شائعة عبر المشاريع الخمسة 109
ملخص الفصل والمذكرة كاملة 109

الفصل الأول: مقدمة في Concurrency و Parallelism

قبل ما نبدأ في أي تفاصيل تقنية، لازم نتفق على حاجة مهمة: الفرق بين Concurrency و Parallelism مش مجرد تعريف نظري بتحفظه وتنساه، ده أساس فكري هتبني عليه كل قرار هتاخده بعد كده في حياتك كـ Machine Learning Engineer — من طريقة تحميل الـ Data، لغاية طريقة تشغيل الـ Inference في بيئة Production. لو الأساس ده مش واضح، هتلاقي نفسك تختار الأداة الغلط في كل مرة، وتقع في مشاكل performance وbugs صعب تفهم سببها.

لماذا نحتاج إلى Concurrency و Parallelism؟

المشكلة التي يحلانها

تخيل عندك سيرفر بيستقبل ١٠٠٠ طلب (request) في الثانية، كل طلب محتاج يقرأ من قاعدة بيانات، يعمل بعض المعالجة، ويرجع رد. لو السيرفر بيعالج كل طلب لوحده بالكامل قبل ما يبدأ اللي بعده (Sequential Execution)، وكل طلب بياخد حتى ١٠٠ ميلي ثانية بس، يبقى السيرفر هيقدر يعالج ١٠ طلبات في الثانية بس. الـ ٩٩٠ طلب الباقيين هينتظروا في طابور، والمستخدمين هيحسوا إن التطبيق بطيء جدًا أو متوقف.

المشكلة الأساسية اللي Concurrency و Parallelism جايين يحلوها هي: كيف نخلي البرنامج يعمل أكتر من حاجة في نفس الوقت (أو يديك إحساس إنه بيعمل كده)، عشان نستغل الموارد المتاحة (CPU cores, I/O devices, Network) بأقصى كفاءة، ونقلل زمن الانتظار، ونزود throughput النظام.

كيف كانت الأنظمة تعمل قبل ظهور هذه المفاهيم

في الأنظمة القديمة والبسيطة، البرنامج كان بيتنفذ بشكل Sequential بحت: تعليمة بعد تعليمة، ومهمة بعد مهمة. لو البرنامج محتاج يقرأ ملف من الـ Disk، البروسيسور كان بيقف فعليًا وينتظر لحد ما القراءة تخلص، من غير ما يعمل أي حاجة تانية مفيدة في الوقت ده. نفس الكلام لو البرنامج بيستنى رد من الشبكة (Network) أو من قاعدة بيانات.

المشكلة إن العمليات دي (I/O operations) بطيئة جدًا مقارنة بسرعة المعالج. لو الـ CPU بيقدر ينفذ مليارات العمليات في الثانية، لكنه بيقف وينتظر Disk أو Network اللي بياخدوا ميلي ثانية أو حتى ثواني، يبقى إحنا بنضيع قدرة معالجة هائلة وهي واقفة من غير فايدة.

> 📌 **ملاحظة مهمة**
>
> الحل القديم في الأنظمة الأولى كان تشغيل نسخة كاملة من البرنامج (Process منفصل) لكل مستخدم أو كل مهمة، وده كان مكلف جدًا في الذاكرة (Memory) والموارد، وبطيء في الإنشاء (Creation) والتبديل بين المهام (Context Switching).

لماذا لم تكن الطرق القديمة كافية

- الـ Sequential Execution بيهدر وقت المعالج أثناء انتظار I/O، وده بيقلل الـ Throughput بشكل كبير.
- تشغيل Process منفصل لكل مهمة مكلف جدًا: كل Process بياخد مساحة Memory مستقلة، وإنشاؤه بياخد وقت (Overhead) ملحوظ.
- التبديل بين Processes كتير (Context Switching) بيستهلك وقت معالجة إضافي مش مفيد للمستخدم.
- الأنظمة القديمة ما كانتش مصممة أصلًا للاستفادة من أكتر من CPU core، لأن الأجهزة القديمة كانت بمعالج واحد (Single Core).

المفهوم الأول: Concurrency

الفكرة الأساسية

الـ Concurrency معناها إن البرنامج قادر يدير أكتر من مهمة (Task) في نفس الفترة الزمنية، من غير ما يشترط إنهم بيتنفذوا فعليًا في نفس اللحظة بالظبط. الفكرة إن البرنامج بيقسم وقته بين المهام المختلفة بذكاء: لما مهمة تستنى I/O (زي قراءة من الشبكة)، البرنامج ينتقل يشتغل على مهمة تانية بدل ما يقعد فاضي.

مثال بسيط: طباخ واحد بيحضر ٣ أطباق في نفس الوقت. هو مش بيعمل كل حاجة فعليًا في نفس اللحظة (إيده واحدة)، لكنه بيدير وقته: يحط حاجة على النار، وأثناء ما هي بتستوي (I/O wait)، يقطع خضار لطبق تاني. ده Concurrency — إدارة ذكية للوقت والمهام، مش تنفيذ متزامن حرفيًا.

كيف يعمل داخليًا

جوه نظام التشغيل (Operating System) أو جوه الـ Runtime بتاع اللغة، بيتم تقسيم الوقت بين المهام المختلفة عن طريق آلية اسمها Scheduling. الـ Scheduler بيقرر إمتى ياخد كل Task/Thread فرصته في المعالج، وإمتى يوقفه (Context Switch) عشان ياخد Task تاني فرصته. في حالة الـ I/O-bound tasks، بمجرد ما المهمة تستنى استجابة من مصدر خارجي (Disk, Network, Database)، الـ Scheduler بينقل المعالج لمهمة تانية جاهزة للتنفيذ بدل ما يسيبه فاضي.

المفهوم الثاني: Parallelism

الفكرة الأساسية

الـ Parallelism معناها إن أكتر من مهمة بتتنفذ فعليًا وحرفيًا في نفس اللحظة الزمنية، وده مش ممكن غير لو عندك أكتر من وحدة معالجة فعلية (Multiple CPU Cores أو Multiple Machines). لو رجعنا لمثال الطباخ، الـ Parallelism يبقى معناه إن عندنا ٣ طباخين، كل واحد بيحضر طبق لوحده في نفس اللحظة بالظبط.

كيف يعمل داخليًا

في نظام بيه أكتر من CPU core، نظام التشغيل بيقدر يوزع Processes أو Threads مختلفة على Cores مختلفة، وكل Core بينفذ تعليماته الخاصة بشكل مستقل تمامًا وفي نفس اللحظة. ده بيحتاج hardware support حقيقي (أكتر من Core)، على عكس الـ Concurrency اللي ممكن تحصل حتى على معالج واحد (Single Core) عن طريق التبديل السريع بين المهام.

> 📌 **ملاحظة مهمة — Concurrency vs Parallelism**
>
> Concurrency = التعامل مع أكتر من حاجة في نفس الفترة الزمنية (إدارة وتبديل، مش بالضرورة تنفيذ فعلي متزامن).
>
> Parallelism = تنفيذ أكتر من حاجة فعليًا في نفس اللحظة (بيحتاج أكتر من Core فعلي).
>
> ممكن يكون عندك Concurrency من غير Parallelism (على معالج واحد)، وممكن يكون عندك الاتنين مع بعض.

الفرق الكامل بين Concurrency و Parallelism

المعيار	Concurrency	Parallelism
التعريف	إدارة أكتر من مهمة في نفس الفترة	تنفيذ أكتر من مهمة في نفس اللحظة
الهدف الأساسي	تحسين الاستجابة وتقليل الانتظار	تسريع المعالجة الفعلية

Parallelism	Concurrency	المعيار
أيوه (ضروري)	لأ (ممكن على core واحد)	يحتاج أكتر من Core؟
CPU-bound tasks	I/O-bound tasks	أفضل استخدام
Multiprocessing	asyncio, Threading	مثال في Python
معالجة صور على عدة Cores في نفس الوقت	سيرفر بيخدم آلاف الاتصالات	مثال واقعي

Synchronous مقابل Asynchronous، والتنفيذ التسلسلي Sequential Execution

عشان نفهم Concurrency صح، لازم نفرّق بين ٣ مفاهيم بيتلخبطوا مع بعض كتير:

1. Sequential Execution: تنفيذ تعليمة بعد التانية، بالترتيب، من غير أي تداخل — سواء كانت العمليات Sync أو Async من حيث الطبيعة.
2. Synchronous (متزامن): كل عملية لازم تخلص وترجع نتيجتها قبل ما ننتقل للي بعدها. البرنامج 'بيقف' (Blocking) لحد ما العملية تخلص.
3. Asynchronous (غير متزامن): العملية بتتبدأ، والبرنامج بيكمل شغل تاني من غير ما ينتظرها، ولما تخلص بتاخد النتيجة عن طريق آلية إشعار (Callback, Future, await).

```
Synchronous – Blocking #
:()def get_data
البرنامج بيقف هنا لحد ما الرد يوصل #  response = requests.get(url)
()return response.json

Asynchronous – Non-blocking #
:()async def get_data
بيسيب الفرصة لمهام تانية أثناء الانتظار #  response = await aiohttp_get(url)
()return response.json
```

> ⚠️ **خطأ شائع**
>
> غلط شائع جدًا: الاعتقاد إن Async معناها Parallel. ده غلط. الـ Async هي طريقة لعمل Concurrency بكفاءة عالية على Thread واحد، مش تنفيذ فعلي متوازي.

أمثلة من الواقع

التفسير	النوع	الموقف
كل طلب بيتعامل معاه بالتبديل الذكي وقت انتظار I/O	Concurrency	سيرفر Web بيخدم آلاف المستخدمين
كل جزء من البيانات بيتعالج فعليًا في نفس اللحظة	Parallelism	تدريب نموذج Deep Learning على GPU متعدد

التفسير	النوع	الموقف
معظم الوقت انتظار شبكة، مش معالجة فعلية	Concurrency (I/O-bound)	تحميل عدة صور من الإنترنت في نفس الوقت
استخدام فعلي متزامن لأكتر من Core	Parallelism (CPU-bound)	ضرب مصفوفات كبيرة (Matrix Multiplication) موزّع على Cores

CPU-bound مقابل I/O-bound

التفرقة دي من أهم القرارات اللي هتاخدها كـ ML Engineer قبل ما تختار أداة الـ Concurrency المناسبة.

CPU-bound Tasks

المهام اللي الوقت المستهلك فيها أساسًا في المعالجة الفعلية جوه المعالج، زي: العمليات الحسابية الكبيرة، معالجة الصور، تدريب النماذج، ضرب المصفوفات. هنا الحل المناسب هو Parallelism الحقيقي (Multiprocessing أو GPU)، لأن الـ Concurrency وحدها (زي Threading في Python بسبب الـ GIL اللي هنشرحه بالتفصيل في الفصل الثالث) مش هتديك تحسن حقيقي.

I/O-bound Tasks

المهام اللي معظم وقتها بيروح في الانتظار: انتظار رد من قاعدة بيانات، انتظار رد من API خارجي، قراءة/كتابة ملفات. هنا الحل الأنسب هو Concurrency (Async Programming أو Threading)، لأن المعالج مش هو عنق الزجاجة (Bottleneck)، والانتظار هو المشكلة.

> ✅ **أفضل الممارسات**
>
> قاعدة عملية سريعة: لو البرنامج بيستنى (شبكة، قاعدة بيانات، ملفات) ← فكر في Concurrency (asyncio / threading).
>
> لو البرنامج بيحسب (رياضيات، معالجة صور، تدريب نماذج) ← فكر في Parallelism (multiprocessing / GPU).

كيفية اختيار النموذج المناسب

السبب	الأداة المناسبة في Python	نوع المهمة
بساطة وكفاية للحمل المنخفض	Threading	I/O-bound بسيط (عدد قليل من الاتصالات)
أخف وأكفأ في استهلاك الموارد من Threads	Async (asyncio)	I/O-bound بحجم كبير جدًا (آلاف الاتصالات)
يتجاوز قيد الـ GIL ويستخدم عدة Cores فعليًا	Multiprocessing	CPU-bound
استغلال الانتظار والمعالجة الفعلية معًا	دمج Async مع Process Pool	مزيج CPU + I/O

كيف تُستخدم هذه المفاهيم داخل الشركات الكبرى

في شركات زي Google وMeta وNetflix وUber، القرار بين Concurrency و Parallelism مش قرار عشوائي، ده جزء من تصميم النظام (System Design) من البداية. سيرفرات الـ API غالبًا مبنية على نماذج Async (زي Node.js أو Python asyncio عن طريق FastAPI) عشان تقدر تخدم عدد ضخم من الطلبات المتزامنة بموارد أقل. في المقابل، الأنظمة اللي بتعالج بيانات ضخمة (زي Data Pipelines أو تدريب النماذج) بتعتمد على Parallelism حقيقي موزّع على عشرات أو آلاف الأجهزة (Distributed Computing).

> 🏢 **من واقع الشركات**
>
> فرق ML Engineer محترف بيسأل قبل كتابة أي كود: هل المهمة دي بتستنى ولا بتحسب؟ الإجابة دي بتحدد الأداة، مش العكس.

أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي

- تحميل ومعالجة البيانات (Data Loading & Preprocessing): غالبًا Parallelism لأنها CPU-bound.
- استدعاء APIs خارجية لجمع بيانات أو Features: Concurrency لأنها I/O-bound.
- خدمة النموذج للمستخدمين (Model Serving) عبر FastAPI: مزيج من Async (لاستقبال الطلبات) وProcess/Thread Pools (للمعالجة الثقيلة).
- تدريب النموذج نفسه: Parallelism على مستوى GPU/Cores متعددة.

متى تستخدمها ومتى لا تستخدمها

لا تستخدم	استخدم	الحالة
أي شكل Concurrency/Parallelism (الـ Overhead أكبر من الفايدة)	تنفيذ Sequential عادي	مهمة بسيطة وسريعة جدًا
Multiprocessing (مضيعة موارد)	Async / Threading	عدد كبير من طلبات شبكة
Threading وحده (بسبب GIL)	Multiprocessing / GPU	حسابات ثقيلة على المعالج
تعقيد غير مبرر بـ async/threads	Sequential	كود بسيط لسكريبت مرة واحدة

أفضل الممارسات، الأخطاء الشائعة، والاعتبارات الأساسية

> ✅ **أفضل الممارسات**
>
> حدد نوع المهمة (CPU-bound أم I/O-bound) قبل ما تختار الأداة.
>
> ابدأ بالحل الأبسط (Sequential) وطوّر لـ Concurrency/Parallelism لما تثبت إن فيه حاجة فعلاً محتاجة تحسين (قِس أولاً، لا تخمّن).
>
> افصل منطق الـ I/O عن منطق الحسابات الثقيلة في الكود عشان تقدر تختار الأداة المناسبة لكل جزء.

> ⚠️ **خطأ شائع**
>
> استخدام Multiprocessing لمهام I/O-bound بسيطة — ده بيضيف Overhead كبير في الذاكرة والـ Process creation من غير أي فايدة حقيقية.

الاعتقاد إن Threading في Python هيسرّع الحسابات الرياضية — بسبب الـ GIL، ده مش هيحصل (تفاصيل كاملة في الفصل الثالث).

إضافة Concurrency لمشروع بسيط من غير داعي، وده بيزود التعقيد وعدد الـ Bugs المحتملة (Race Conditions, Deadlocks) من غير فايدة حقيقية.

اعتبارات الأداء (Performance)

أي شكل Concurrency أو Parallelism بيجيب معاه تكلفة إضافية (Overhead): إنشاء Threads أو Processes، التبديل بينهم (Context Switching)، وتنسيق البيانات بينهم (Synchronization). لو المهمة الأصلية خفيفة، ممكن التكلفة دي تكون أكبر من الفايدة. القاعدة الذهبية: قِس الأداء قبل وبعد (Benchmark)، ومتفترضش إن الـ Concurrency هتحسن الأداء تلقائيًا.

الاعتبارات الأمنية (Security)

لما أكتر من مهمة بتتعامل مع نفس البيانات المشتركة (Shared State) في نفس الوقت، بيظهر خطر الـ Race Conditions اللي ممكن تؤدي لتسريب بيانات بين مستخدمين مختلفين، أو حالات غير متوقعة في النظام (زي معالجة نفس الطلب المالي مرتين). التعامل الآمن مع البيانات المشتركة هيتشرح بالتفصيل في فصل الـ Threading.

اعتبارات قابلية التوسع (Scalability)

النماذج المبنية على Async (زي asyncio) بتقدر تتوسع لعدد ضخم جدًا من الاتصالات المتزامنة بموارد أقل من Threading التقليدي، لأنها مش بتحتاج Thread منفصل لكل اتصال. في المقابل، Multiprocessing بيتوسع أفقيًا بعدد الـ Cores المتاحة، ولما تحتاج توسع أكبر من جهاز واحد، بندخل في عالم الـ Distributed Systems اللي هيتغطى في فصول لاحقة.

مثال عملي صغير

عايزين نحمّل بيانات من ٥ روابط مختلفة. هنقارن بين التنفيذ الـ Sequential والتنفيذ الـ Concurrent البسيط:

```
import time
import requests

urls = ["https://api.example.com/1", "https://api.example.com/2", "..."]

كل رابط بينتظر اللي قبله – Sequential #
()start = time.time
:for url in urls
requests.get(url)
print(f"Sequential: {time.time() - start:.2f}s")

هنشوف في الفصل السادس (Async) والرابع (Threading) إزاي نخلي #
الطلبات الخمسة تتنفذ بشكل Concurrent فعلي، وتقلل الوقت الكلي بشكل كبير #
```

مثال من مشروع Production حقيقي

في خدمة Model Serving حقيقية مبنية بـ FastAPI، السيرفر بيستقبل طلبات Inference من آلاف المستخدمين في نفس الوقت. استقبال الطلبات ومعالجة الـ I/O (قراءة الـ Request، الاتصال بقاعدة بيانات الـ Features، إرجاع الرد) بيتم بشكل Async عشان يخدم أكبر عدد ممكن من الطلبات المتزامنة بأقل موارد. لكن عملية الـ Inference نفسها (لو كانت ثقيلة على

المعالج) غالبًا بتتحول لـ Process Pool منفصل عشان متعطلش الـ Event Loop الأساسي. الهندسة دي (Async I/O + Process Pool للحسابات الثقيلة) هي النمط الأساسي اللي هنبنيه بالتفصيل في الفصل العاشر (Concurrency in FastAPI).

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل الأول**
>
> اكتب سكريبت بيقيس الفرق الفعلي بين تحميل ١٠ صفحات ويب بشكل Sequential، وبنفس المهمة لكن بعد ما تتعرف على asyncio في الفصل السادس.
>
> احسب: الوقت الكلي، متوسط الوقت لكل طلب، ونسبة التحسن.
>
> صنّف كل جزء من مشروع ML سبق واشتغلت عليه (تحميل بيانات، معالجة، تدريب، تقييم) على أساس إنه CPU-bound ولا I/O-bound.

تحديات شائعة

- صعوبة تحديد نوع المهمة بدقة (بعض المهام مختلطة I/O + CPU في نفس الوقت).
- الخلط بين مفهوم "الكود بيشتغل أسرع" و"الكود بيدير وقته أحسن" — مش كل Concurrency بتعني سرعة أعلى في المعالجة الفعلية.
- صعوبة تقدير الـ Overhead الحقيقي بدون قياس فعلي (Benchmarking) على البيئة المستهدفة.

ملخص الفصل

- Concurrency = إدارة أكتر من مهمة في نفس الفترة الزمنية، وممكن تحصل حتى على core واحد.
- Parallelism = تنفيذ فعلي متزامن لأكتر من مهمة، وبيحتاج أكتر من core حقيقي.
- I/O-bound tasks تناسبها Concurrency (Async/Threading)، وCPU-bound tasks تناسبها Parallelism (Multiprocessing/GPU).
- القرار بين الاتنين بيتحدد بنوع المهمة، مش بالموضة أو التعقيد التقني.
- الشركات الكبرى بتدمج الاتنين في نفس النظام: Async لاستقبال الطلبات، Parallelism للمعالجة الثقيلة.

الفصل القادم هياخدنا خطوة لجوّه أكتر: هنفهم بالتفصيل الفرق بين Process و Thread من الداخل — إزاي كل واحد فيهم بياخد Memory، إزاي بيتم إنشاؤه، وإزاي نظام التشغيل بيدير الـ Context Switching بينهم. الفهم ده أساسي قبل ما نقدر نغوص في تفاصيل الـ Python GIL في الفصل الثالث، ليه اتصمم بالشكل ده، وتأثيره العملي على قرارات الـ Concurrency اللي هنبنيها طول باقي المذكرة.

الفصل الثاني: Process و Thread

في الفصل الأول اتفقنا إن Concurrency و Parallelism هما الفكرة العامة (المفهوم)، لكن السؤال دلوقتي: مين اللي فعليًا بينفذ الكود جوه نظام التشغيل؟ الإجابة هي وحدتين أساسيتين: الـ Process والـ Thread. فهم الاتنين دول من الداخل — إزاي بياخدوا Memory، إزاي بيتولدوا، وإزاي نظام التشغيل بيبدّل بينهم — هو الأساس اللي هيخليك تفهم بعد كده ليه Python فيها حاجة اسمها GIL، وليه أحيانًا بنستخدم Threading وأحيانًا Multiprocessing لنفس النوع من المشاكل.

المشكلة التي يحلها مفهوم Process

لو نظام التشغيل سمح لكل البرامج تشتغل في نفس مساحة الذاكرة من غير عزل، أي برنامج فيه خطأ أو Bug (أو برنامج خبيث) هيقدر يقرأ أو يعدّل بيانات برنامج تاني شغال في نفس اللحظة. تخيل متصفح الإنترنت بتاعك يقدر يقرأ كلمات سر بنك برنامج تاني شغال جنبه — كارثة أمنية وكارثة استقرار (Stability) في نفس الوقت.

الـ Process هو الحل: وحدة معزولة تمامًا، ليها مساحة Memory خاصة بيها محدش غيرها يقدر يوصلها مباشرة، وليها كل الموارد اللي محتاجاها عشان تشتغل (Memory, File Handles, Environment Variables) بشكل مستقل عن أي Process تاني.

كيف كانت الأنظمة تعمل قبل مفهوم الـ Process المعزول

في الأنظمة القديمة جدًا (زي بعض أنظمة الـ Batch Processing في الستينات)، كان البرنامج بياخد الجهاز كله لوحده من الأول للآخر، برنامج واحد يشتغل، يخلص، بعدين اللي بعده يبدأ. النظام ده كان بسيط لكنه بيضيع موارد المعالج بشكل رهيب وقت ما البرنامج بيستنى I/O، ومحدش تاني يقدر يستخدم الجهاز في الوقت ده.

> 📌 **ملاحظة مهمة**
>
> ظهور مفهوم الـ Process في أنظمة التشغيل الحديثة (زي Multics ثم Unix) كان قفزة أساسية: سمح بتشغيل أكتر من برنامج في نفس الوقت بشكل آمن ومعزول، مع إدارة ذكية للمعالج بينهم.

ما هو Process؟

الـ Process هو نسخة قيد التنفيذ (Instance) من برنامج. لما تفتح أي برنامج — متصفح، محرر أكواد، سكريبت Python — نظام التشغيل بيعمل له Process جديد، وبيدّي له كل الموارد اللي محتاجها عشان يشتغل بشكل مستقل.

مكونات الـ Process

- Process ID (PID): رقم فريد يميز كل Process عن غيره في النظام.
- Memory Space: مساحة ذاكرة معزولة خاصة بالـ Process، محدش غيره يقدر يوصلها مباشرة.
- Code Segment: نسخة من تعليمات البرنامج المُنفَّذة.
- Data Segment: المتغيرات العامة (Global Variables) والبيانات الثابتة.
- Heap: الذاكرة الديناميكية اللي البرنامج بيحجزها وقت التشغيل (زي كائنات Python).
- Stack: بيتبع فيها استدعاءات الدوال (Function Calls) والمتغيرات المحلية.
- File Descriptors: مراجع للملفات والـ Sockets المفتوحة.
- Process Control Block (PCB): بنية بيانات داخل نظام التشغيل بتخزن كل حالة الـ Process (الـ PID، حالة التنفيذ، قيم الـ Registers، أولوية الجدولة).

ما هو Thread؟

الـ Thread هو أصغر وحدة تنفيذ جوه الـ Process. الـ Process ممكن يحتوي على Thread واحد (Single-threaded) أو أكتر من Thread (Multi-threaded)، وكل الـ Threads جوه نفس الـ Process بتشارك نفس مساحة الـ Memory (نفس الـ Heap ونفس الـ Data Segment)، لكن كل Thread ليه Stack خاص بيه وRegisters خاصة بيه.

> 📌 **ملاحظة مهمة — تشبيه عملي**
>
> الـ Process زي مبنى (Building) كامل بموارده الخاصة (كهرباء، مياه، عنوان مستقل).
>
> الـ Threads جوه الـ Process زي موظفين بيشتغلوا جوه نفس المبنى، بيشاركوا نفس المكتب والملفات (Shared Memory)، لكن كل واحد فيهم له مكتبه الشخصي الصغير (Stack) اللي بيحتفظ فيه بشغله الخاص.

Process Memory مقابل Thread Memory

الفرق ده من أهم الفروق اللي هتبني عليه فهمك لكل حاجة جاية بعد كده، خصوصًا لما نتكلم عن Race Conditions في فصل الـ Threading.

Process Memory

كل Process ليه مساحة Memory معزولة بالكامل عن أي Process تاني. لو عايز تبعت بيانات من Process لـ Process تاني، محتاج آلية اتصال صريحة اسمها IPC (Inter-Process Communication) — زي Pipes أو Queues أو Shared Memory مُعرَّفة صراحة — وده هيتشرح بالتفصيل في فصل الـ Multiprocessing.

Thread Memory

كل الـ Threads جوه نفس الـ Process بتشارك نفس الـ Heap ونفس الـ Data Segment تلقائيًا، من غير أي آلية اتصال خاصة. الميزة إن مشاركة البيانات بين Threads سهلة وسريعة جدًا (نفس المتغير في الذاكرة). لكن المشكلة إن السهولة دي هي نفسها مصدر الخطر: لو أكتر من Thread عدّلوا نفس المتغير في نفس الوقت من غير تنسيق، بيحصل Race Condition (هنشرحها بالتفصيل في الفصل الرابع).

المعيار	Process	Thread
الذاكرة	معزولة تمامًا عن باقي الـ Processes	مشتركة مع باقي الـ Threads في نفس الـ Process
التواصل	يحتاج IPC (Pipes, Queues, Shared Memory صريحة)	مباشر عن طريق المتغيرات المشتركة
تكلفة الإنشاء	عالية (نسخ كامل بيئة التشغيل)	منخفضة نسبيًا (نفس بيئة الـ Process)
الأمان بين الوحدات	عالٍ (عزل كامل)	أقل (مشاركة الذاكرة تفتح باب الأخطاء)
تأثر باقي الوحدات عند التعطل (Crash)	لا يؤثر على باقي الـ Processes	قد يعطّل الـ Process كله
مناسب لـ	عزل قوي، أو استغلال أكتر من CPU core	مشاركة بيانات سريعة، مهام I/O-bound خفيفة

كيفية إنشاء Process — Process Creation

لما تطلب من نظام التشغيل ينشئ Process جديد، بيحصل تسلسل من الخطوات: حجز مساحة Memory جديدة بالكامل، نسخ (أو مشاركة عن طريق آلية Copy-on-Write) بيانات الـ Process الأب لو الإنشاء عن طريق fork، تحميل الكود المطلوب تنفيذه، إنشاء Process Control Block جديد، وتسجيله عند الـ Scheduler عشان ياخد نصيبه من وقت المعالج.

> 📌 **ملاحظة مهمة**
>
> على أنظمة Unix/Linux، بيتم إنشاء Process جديد غالبًا عن طريق نداء النظام fork() (بينسخ الـ Process الحالي) متبوعًا بـ exec() (بيستبدل الكود المنفَّذ ببرنامج جديد). في Python، مكتبة multiprocessing بتخفي التفاصيل دي وتوفرها كـ API بسيط.

```
import multiprocessing

:def worker(name)
("{name} :يشتغل باسم f"Process)print

:'__if __name__ == '__main
p = multiprocessing.Process(target=worker, args=("A",))
يطلب من نظام التشغيل إنشاء Process جديد فعليًا #   ()p.start
ينتظر الـ Process لحد ما يخلص #    ()p.join
```

كيفية إنشاء Thread — Thread Creation

إنشاء Thread أخف بكثير من إنشاء Process، لأنه مش محتاج ينسخ مساحة Memory جديدة كاملة — الـ Thread الجديد بيستخدم نفس مساحة الـ Process الموجودة، وكل اللي محتاجه هو Stack خاص بيه، وتسجيله عند الـ Scheduler كوحدة تنفيذ مستقلة جوه نفس الـ Process.

```
import threading

:def worker(name)
("{name} :يشتغل باسم f"Thread)print

t = threading.Thread(target=worker, args=("A",))
إنشاء Thread جديد جوه نفس الـ Process الحالي #   ()t.start
ينتظر الـ Thread لحد ما يخلص #    ()t.join
```

دورة حياة الـ Thread — Thread Lifecycle

الـ Thread بيمر بمجموعة حالات (States) طول فترة وجوده، ومعرفتها مهمة جدًا لفهم إزاي الـ Scheduler بيقرر مين يشتغل إمتى:

4. New: الـ Thread اتعمل له تعريف لكنه لسه ما بدأش التنفيذ (لسه ما اتناداش عليه start()).
5. Runnable / Ready: الـ Thread جاهز للتنفيذ ومستنى دوره عند الـ Scheduler.
6. Running: الـ Thread فعليًا بينفذ تعليماته على المعالج دلوقتي.
7. Blocked / Waiting: الـ Thread واقف مؤقتًا (بينتظر I/O، أو بينتظر Lock، أو بينتظر إشارة من Thread تاني).
8. Terminated: الـ Thread خلّص تنفيذه بالكامل (أو اتوقف بسبب خطأ).

Context Switching — التبديل بين وحدات التنفيذ

الفكرة الأساسية

بما إن عدد الـ Cores الفعلية محدود (غالبًا أقل بكتير من عدد الـ Processes/Threads اللي عايزة تشتغل)، نظام التشغيل مضطر يبدّل المعالج بين وحدات التنفيذ المختلفة بسرعة كبيرة جدًا، بحيث يديك إحساس إن كل حاجة شغالة في نفس الوقت (حتى لو المعالج فعليًا بيبدّل بينهم آلاف المرات في الثانية).

كيف يعمل Context Switching داخليًا

9. الـ Scheduler بيقرر إيقاف الـ Thread/Process الحالي (سواء لأنه استهلك وقته المخصص، أو دخل في حالة Blocked لانتظار I/O).
10. نظام التشغيل بيحفظ حالة التنفيذ الحالية (قيم الـ Registers، الـ Program Counter) في الـ PCB أو TCB (Thread Control Block) الخاص بيه.
11. الـ Scheduler بيختار وحدة التنفيذ التالية اللي هتشتغل بناءً على خوارزمية الجدولة (Scheduling Algorithm).
12. نظام التشغيل بيسترجع (Restore) حالة الوحدة الجديدة من الذاكرة، ويكمّل تنفيذها من حيث ما وقفت بالظبط.

> 🏢 **من واقع الشركات**
>
> التبديل بين Threads جوه نفس الـ Process أرخص (أسرع) من التبديل بين Processes مختلفة، لأن الـ Threads بتشارك نفس مساحة الـ Memory، فمفيش حاجة إسمها تبديل جداول الذاكرة (Memory Page Tables) زي ما بيحصل مع الـ Processes.

اعتبارات الأداء لـ Context Switching

كل عملية Context Switching بتاخد وقت معالج فعلي (مش صفر)، وكل ما زاد عدد الـ Threads/Processes المتنافسة على نفس عدد الـ Cores المحدود، زاد عدد مرات التبديل، وزادت النسبة المهدرة من وقت المعالج في التبديل نفسه بدل التنفيذ الفعلي. ده سبب رئيسي ليه إنشاء آلاف الـ Threads لمهمة واحدة مش دايمًا فكرة كويسة، حتى لو المهمة I/O-bound.

مقارنة كاملة بين Process و Thread

المعيار	Process	Thread
تكلفة الإنشاء	عالية	منخفضة
تكلفة الـ Context Switching	عالية (تبديل جداول الذاكرة)	منخفضة (نفس مساحة الذاكرة)
مشاركة البيانات	صعبة (محتاجة IPC)	سهلة ومباشرة (خطر Race Conditions)
العزل والأمان	قوي	أضعف
الاستفادة من أكتر من Core	نعم، فعليًا	في Python: محدودة بسبب الـ GIL (الفصل القادم)
تأثر النظام عند تعطل وحدة واحدة	الوحدات التانية آمنة	ممكن يأثر على الـ Process كله

متى نستخدم Process ومتى نستخدم Thread

السبب	الاختيار الأنسب	الحالة
استغلال أكتر من Core فعليًا، تجاوز الـ GIL	Process (Multiprocessing)	مهمة CPU-bound (حسابات ثقيلة)
تكلفة إنشاء وتبديل أقل، ومشاركة بيانات سهلة	Thread (Threading)	مهمة I/O-bound خفيفة العدد
عزل كامل للذاكرة يمنع تأثر وحدة بالتانية	Process	الحاجة لعزل أمني قوي بين وحدات العمل
نفس مساحة الذاكرة، بدون تكلفة نسخ أو IPC	Thread	مشاركة بيانات كبيرة بسرعة بين وحدات العمل
تعطل Process واحد لا يهدد الباقي	Process	الحاجة لموثوقية عالية (فشل وحدة لا يوقف الباقي)

كيف تُستخدم داخل الشركات الكبرى

في أنظمة الإنتاج الحقيقية، القرار بين Process وThread مبني على تحليل واضح لنوع الحمل (Workload). خدمات الـ Web غالبًا بتشغّل عدة Processes (Workers) على مستوى السيرفر (زي Gunicorn workers قدام تطبيق FastAPI) عشان تستفيد من كل الـ CPU cores المتاحة وتضمن إن تعطل Worker واحد ما يوقفش الخدمة كلها، وجوه كل Process ممكن يستخدموا Threads أو Async Event Loop للتعامل مع عدد كبير من الاتصالات المتزامنة بكفاءة.

> 🏢 **من واقع الشركات**
>
> نمط شائع جدًا في الإنتاج: عدة Processes (لاستغلال الـ Cores والعزل) + جوه كل Process، Async/Threads (لإدارة عدد كبير من الطلبات بكفاءة). هنشوف النمط ده عمليًا في الفصل العاشر (Concurrency in FastAPI).

اعتبارات الأداء، الأمان، وقابلية التوسع

الأداء

زيادة عدد الـ Processes أكتر من عدد الـ CPU cores المتاحة فعليًا مش بتحسن الأداء بعد نقطة معينة — بالعكس، بتزود تكلفة الـ Context Switching من غير فايدة إضافية حقيقية في المعالجة الفعلية.

الأمان

العزل القوي بين الـ Processes بيحمي من كتير من فئات الأخطاء الأمنية (زي وصول كود خبيث لذاكرة عملية تانية)، وده سبب رئيسي ليه المتصفحات الحديثة (زي Chrome) بتشغّل كل تبويب في Process منفصل.

قابلية التوسع

الـ Processes بتتوسع أفقيًا بسهولة أكبر (تقدر تشغّل نسخ منها على أجهزة مختلفة، زي في أنظمة الـ Microservices)، بينما الـ Threads محصورة جوه حدود الجهاز الواحد لأنها بتشارك ذاكرة نفس الـ Process.

أفضل الممارسات والأخطاء الشائعة

> ✅ **أفضل الممارسات**

استخدم Process لما تحتاج عزل حقيقي أو استغلال فعلي لأكتر من Core.

استخدم Thread لما المهمة I/O-bound وعدد الوحدات معقول (عشرات مش آلاف).

دايمًا قِس عدد الـ CPU cores المتاحة فعليًا قبل ما تقرر عدد الـ Processes المناسب.

⚠️ خطأ شائع

إنشاء آلاف الـ Threads لمهمة واحدة على أمل إن ده هيسرّع الأداء — غالبًا بيبقى العكس بسبب تكلفة الـ Context Switching.

الاعتقاد إن Threads بتديك نفس عزل الـ Processes — مشاركة الذاكرة معناها إن خطأ في Thread ممكن يأثر على الـ Process كله.

نسيان إن إنشاء Process في Python (multiprocessing) بياخد وقت Overhead ملحوظ، فاستخدامه لمهام صغيرة وسريعة جدًا ممكن يكون أبطأ من التنفيذ الـ Sequential العادي.

مثال عملي صغير

مقارنة بسيطة توضح تكلفة الإنشاء: قياس الوقت اللازم لإنشاء ٥٠ Thread مقابل إنشاء ٥٠ Process لنفس المهمة البسيطة.

```
import time, threading, multiprocessing

:()def task
pass

Threads قياس تكلفة إنشاء الـ #
()start = time.time
threads = [threading.Thread(target=task) for _ in range(50)]
()for t in threads: t.start
()for t in threads: t.join
print(f"Threads: {time.time() - start:.4f}s")

(النتيجة هتكون أبطأ بشكل ملحوظ) Processes قياس تكلفة إنشاء الـ #
:'__if __name__ == '__main
()start = time.time
procs = [multiprocessing.Process(target=task) for _ in range(50)]
()for pr in procs: pr.start
()for pr in procs: pr.join
print(f"Processes: {time.time() - start:.4f}s")
```

مثال من مشروع Production حقيقي

في نظام Model Serving مبني على FastAPI خلف Gunicorn، غالبًا بيتم تشغيل عدد Processes (Workers) يساوي تقريبًا عدد الـ CPU cores المتاحة على السيرفر، عشان يستفيد من كل Core بشكل مستقل ومعزول. جوه كل Worker Process، السيرفر بيستخدم Async Event Loop عشان يخدم عدد كبير من طلبات الشبكة المتزامنة بكفاءة عالية بدون الحاجة لآلاف الـ Threads. الهندسة دي بتجمع أفضل ما في العالمين: عزل واستغلال الـ Cores من الـ Processes، وكفاءة إدارة الاتصالات من الـ Async.

مشروع صغير لتطبيق ما تم شرحه

📌 **ملاحظة مهمة — مشروع الفصل الثاني**

اكتب سكريبت بيطبع الـ Process ID الخاص بيه (os.getpid())، وبعدين ينشئ ٣ Threads، وكل Thread يطبع اسمه ويوضح إنه بيشارك نفس الـ Process ID.

بعد كده، عدّل السكريبت عشان ينشئ ٣ Processes بدل الـ Threads، ولاحظ إن كل Process ليه PID مختلف تمامًا.

استخدم أداة مراقبة النظام (زي htop أو Task Manager) وأنت السكريبت شغال، وشوف الفرق في عدد الـ Processes الظاهرة على الشاشة.

تحديات شائعة

- صعوبة تقدير العدد الأمثل من Processes/Threads بدون قياس فعلي على البيئة المستهدفة.
- الخلط بين 'مشاركة الذاكرة السهلة' في الـ Threads و'الأمان' — السهولة دي هي نفسها مصدر الخطر.
- نسيان إن كل Process في Python بياخد وقت بدء تشغيل (Startup) خاص بيه، وده بيأثر على المهام قصيرة العمر.

ملخص الفصل

- Process = وحدة تنفيذ معزولة بالكامل، ليها مساحة Memory خاصة، وتكلفة إنشائها عالية.
- Thread = وحدة تنفيذ أصغر جوه الـ Process، بتشارك نفس مساحة الـ Memory، وتكلفة إنشائها أقل.
- Context Switching هو الآلية اللي نظام التشغيل بيبدّل بيها بين وحدات التنفيذ، وليها تكلفة أداء حقيقية.
- استخدم Process للعزل واستغلال الـ Cores الفعلي، واستخدم Thread للمهام الخفيفة اللي محتاجة مشاركة بيانات سريعة.
- أنظمة الإنتاج الحقيقية غالبًا بتجمع الاتنين مع بعض: Processes للعزل والاستغلال، وThreads/Async جوه كل Process لإدارة الحمل.

دلوقتي وإحنا فاهمين الفرق بين Process و Thread من جوه، جاء الوقت لسؤال أساسي في عالم Python بالتحديد: ليه بنقول إن Threading في Python 'محدود' لما يجي الأمر لمهام CPU-bound؟ الإجابة اسمها Global Interpreter Lock (GIL)، وهي أشهر وأهم قيد تقني هيأثر على كل قرار Concurrency هتاخده في Python. الفصل الثالث هيشرحلك بالتفصيل الكامل إيه هو الـ GIL، ليه اتصمم بالشكل ده، وإزاي تتعامل معاه باحترافية.

الفصل الثالث: Python GIL

في الفصل التاني فهمنا إن الـ Threads بتشارك نفس مساحة الذاكرة، وده بيخليها أخف وأسرع من الـ Processes. المنطقي إن ده يبقى معناه إن استخدام أكتر من Thread في مهمة حسابية ثقيلة هيسرّعها بشكل ملحوظ لو الجهاز عنده أكتر من Core. لكن في Python بالتحديد، ده مش بيحصل — وده مش Bug، ده قرار تصميمي مقصود اسمه Global Interpreter Lock، أو اختصارًا GIL. فهم الـ GIL صح هو اللي هيخليك تاخد قرارات صح بعد كده: امتى تستخدم Threading، وامتى لازم تروح لـ Multiprocessing أو Async.

ما هي المشكلة التي أدت لظهور الـ GIL؟

الـ CPython (النسخة الرسمية والأكتر استخدامًا من مفسّر Python) بتدير الذاكرة عن طريق آلية اسمها Reference Counting: كل كائن (Object) في Python ليه عداد بيحسب عدد المراجع (References) اللي بتشاور عليه، ولما العداد يوصل صفر، الذاكرة بتتحرر تلقائيًا (Garbage Collection).

المشكلة: لو عندنا أكتر من Thread بيشتغلوا فعليًا في نفس اللحظة (Parallel) وبيعدّلوا نفس العداد ده لنفس الكائن، ممكن يحصل Race Condition على العداد نفسه — مثلًا Threadين يزودوا العداد في نفس اللحظة، لكن بسبب تداخل العمليات، النتيجة النهائية تبقى غلط (زيادة واحدة بس بدل اتنين). ده ممكن يؤدي لتسريب ذاكرة (Memory Leak) أو أسوأ من كده: تحرير ذاكرة لسه مستخدمة فعليًا، وده بيسبب Crash فوري للبرنامج.

كيف كانت المشكلة تُحل قبل الـ GIL

الحل البديهي كان: نحط Lock (قفل) حوالين كل عملية تعديل على أي عداد Reference Count بشكل منفرد (Fine-grained Locking). لكن التجربة العملية أثبتت إن الحل ده كارثي على الأداء: الأقفال الكتير جدًا دي بتستهلك وقت معالج كبير في الإدارة نفسها (Lock Overhead)، وحتى برامج الـ Single-threaded كانت بتبقى أبطأ بشكل ملحوظ عشان أي عملية بسيطة (زي y = x + 1) بقت محتاجة تدخل وتخرج من Lock.

> 📌 **ملاحظة مهمة**
>
> التجارب المبكرة على Fine-grained Locking في Python أظهرت إبطاء ملحوظ في الأداء حتى للبرامج اللي بتستخدم Thread واحد بس، بسبب تكلفة إدارة الأقفال المتعددة نفسها.

ما هو الـ GIL؟

الـ Global Interpreter Lock هو قفل واحد بس (مش قفل لكل متغير) بيضمن إن Thread واحد بس في اللحظة الواحدة هو اللي يقدر ينفذ Python Bytecode جوه المفسّر (Interpreter)، حتى لو عندك عدة Threads وعدة CPU cores متاحة.

الفكرة الأساسية

بدل ما نحط قفل صغير حوالين كل عملية، Python حطت قفل واحد كبير (Coarse-grained Lock) حوالين تنفيذ الـ Bytecode كله. أي Thread عايز ينفذ أي كود Python، لازم ياخد الـ GIL الأول. لما ياخده، باقي الـ Threads بيقفوا وينتظروا لحد ما الـ Thread الحالي يسيب الـ GIL (إما لأنه استهلك 'وقته' المحدد، أو لأنه دخل في عملية I/O ووقف).

النتيجة	الوصف	النهج
أمان لكن بطء شديد حتى للـ Single-threaded	قفل منفصل لكل عملية على العدادات	Fine-grained Locking

النهج	الوصف	النتيجة
GIL (Coarse-grained)	قفل واحد عام لتنفيذ الـ Bytecode بالكامل	أداء أسرع للـ Single-threaded، لكن قيد على الـ CPU-bound Parallelism

كيف يعمل الـ GIL داخليًا

13. أي Thread عايز ينفذ Python Bytecode لازم ياخد الـ GIL أولًا.
14. الـ Thread بيفضل ماسك الـ GIL وينفذ كوده لحد ما يحصل واحد من احتمالين: يستهلك عدد معين من التعليمات (أو مدة زمنية محددة، افتراضيًا حوالي ٥ ميلي ثانية في النسخ الحديثة)، أو يدخل في عملية I/O (زي قراءة ملف أو انتظار شبكة) فيسيب الـ GIL تلقائيًا.
15. لما الـ Thread يسيب الـ GIL (سواء طوعًا وقت I/O أو إجباريًا بعد انتهاء وقته)، الـ Scheduler بتاع Python بيدّي الفرصة لـ Thread تاني ينتظر ياخد الـ GIL.
16. العملية دي بتتكرر باستمرار — تبديل سريع جدًا للـ GIL بين الـ Threads، وده اللي بيدّي إحساس إن أكتر من Thread شغالين 'في نفس الوقت'، مع إن Thread واحد بس فعليًا بينفذ Bytecode في أي لحظة.

> 🏢 **من واقع الشركات**
>
> الـ GIL بيتحرر تلقائيًا وقت عمليات الـ I/O (قراءة ملفات، شبكة، قاعدة بيانات)، وده سبب رئيسي ليه Threading في Python فعّال جدًا للمهام I/O-bound رغم وجود الـ GIL — لأن الـ Thread بيسيب القفل أثناء الانتظار، فيقدر Thread تاني يشتغل.

تأثير الـ GIL على الأداء

تأثيره على المهام CPU-bound

لو عندك مهمة حسابية ثقيلة (زي ضرب مصفوفات كبيرة بكود Python عادي، مش عن طريق مكتبة زي NumPy اللي بتنفذ الكود الفعلي في C خارج الـ GIL)، وشغّلتها على ٤ Threads على جهاز بـ ٤ Cores، مش هتلاقي تحسن حقيقي في السرعة — ممكن كمان تلاقي النتيجة أبطأ شوية من التنفيذ الـ Sequential، بسبب تكلفة التبديل المستمر للـ GIL بين الـ Threads من غير أي فايدة فعلية في المعالجة المتوازية.

تأثيره على المهام I/O-bound

في المقابل، لو عندك مهمة I/O-bound (زي تحميل عدة ملفات من الإنترنت)، استخدام Threading بيدّي تحسن حقيقي وملحوظ، لأن كل Thread بيسيب الـ GIL أثناء انتظار الشبكة، فيقدر Thread تاني ياخد الـ GIL ويشتغل. هنا الـ GIL عمليًا مش عائق.

```
import threading, time

:()def cpu_task
x = 0
:for _ in range(50_000_000)
x += 1

تشغيل المهمة على thread واحد #
()start = time.time
```

```
()cpu_task
print(f"Single: {time.time()-start:.2f}s")

تشغيل نفس المهمة موزعة على 4 threads – بسبب الـ GIL، النتيجة #
مش هتبقى أسرع بشكل ملحوظ، وممكن تبقى أبطأ شوية #
()start = time.time
threads = [threading.Thread(target=cpu_task) for _ in range(4)]
()for t in threads: t.start
()for t in threads: t.join
print(f"4 Threads: {time.time()-start:.2f}s")
```

⚠️ خطأ شائع

الاعتقاد إن 'زيادة عدد الـ Threads دايمًا بتسرّع الكود' — في Python، ده صحيح بس للمهام I/O-bound. للمهام CPU-bound، الحل الصح هو Multiprocessing (هيتشرح بالتفصيل في الفصل الخامس) عشان كل Process ليه GIL خاص بيه ومفسّر Python منفصل تمامًا.

لماذا لم يتم تصميم الـ GIL بشكل مختلف من الأول؟

القرار مكانش عشوائي؛ كان مبني على موازنة بين عدة اعتبارات في وقت تصميم CPython:

- البساطة: الـ GIL بيبسّط كتير من تفاصيل تنفيذ CPython الداخلية، وبيقلل احتمالية وجود Bugs معقدة متعلقة بالـ Memory Management.
- الأداء للـ Single-threaded: أغلب برامج Python وقت تصميم اللغة كانت Single-threaded، فالأولوية كانت لجعل الحالة الشائعة دي أسرع ما يمكن.
- التوافق مع المكتبات الخارجية (C Extensions): كتير من مكتبات Python بتُكتب أجزاء منها بلغة C، والـ GIL بيسهّل كتابة الأكواد دي بأمان من غير الحاجة لإدارة Locking معقدة يدويًا في كل مكتبة.

كيف تتجاوز قيد الـ GIL عمليًا

متى تستخدمه	الفكرة	الحل
مهام CPU-bound تحتاج استغلال حقيقي لعدة Cores	كل Process ليه مفسّر Python خاص وGIL خاص به	Multiprocessing
عمليات رياضية ومصفوفات بأجزاء كبيرة منها (NumPy, Pandas)	الحسابات الثقيلة بتتنفذ في كود C خارج الـ GIL	مكتبات C/NumPy
مهام I/O-bound بحجم كبير من الاتصالات المتزامنة	Concurrency على مستوى الـ I/O بدون الحاجة لعدة Threads أصلًا	Async I/O
حالات خاصة، أقل شيوعًا في بيئات الإنتاج	مفسّرات مختلفة عن CPython (زي Jython، أو GraalPy) لا تحتوي على GIL بنفس الشكل	Alternative Interpreters

🏢 من واقع الشركات

لو مكتبة زي NumPy أو Pandas بتعمل عملية حسابية ثقيلة، جزء كبير من الوقت الفعلي بيتنفذ في كود C محسّن خارج الـ GIL تمامًا. ده سبب ليه ممكن تلاقي بعض العمليات في NumPy بتستفيد من Threading حتى لو الكود عمومًا محكوم بالـ GIL.

مستقبل Python بدون GIL

في السنوات الأخيرة، ظهر مشروع رسمي داخل CPython (معروف باسم PEP 703) بهدف توفير نسخة اختيارية من Python من غير GIL (Free-threaded CPython)، عشان تسمح باستغلال حقيقي لعدة Cores عن طريق Threading التقليدي. الفكرة بديلة عن الـ Reference Counting البسيط بآليات أكتر تعقيدًا لكنها آمنة بدون قفل عام واحد.

> 📌 **ملاحظة مهمة**
>
> التوجه ده لسه في مراحل تجريبية ومتاح كخيار اختياري في بعض إصدارات Python الحديثة، مش الوضع الافتراضي بعد. أي كود Production حاليًا لازم يفترض وجود الـ GIL إلا لو اتأكدت صراحة إنك شغال على نسخة Free-threaded ومكتباتك كلها متوافقة معاها.

كيف تُستخدم هذه المعرفة داخل الشركات الكبرى

مهندسو الـ ML والـ Backend في الشركات الكبرى بياخدوا قرار الـ GIL في الاعتبار من مرحلة التصميم: خدمات الـ Model Serving اللي فيها معالجة ثقيلة (زي Image Processing أو Feature Engineering معقد) بتُبنى غالبًا على أساس دمج Async I/O (لاستقبال الطلبات بكفاءة) مع Process Pool منفصل (لتنفيذ الحسابات الثقيلة فعليًا على عدة Cores)، بدل الاعتماد على Threading وحده للحسابات.

أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي

- معالجة الصور أو البيانات الثقيلة بكود Python نقي: تحتاج Multiprocessing وليس Threading.
- استدعاء عدة APIs خارجية لجمع بيانات Features: Threading أو Async مناسبين تمامًا لأنها I/O-bound.
- عمليات NumPy/Pandas الكبيرة: قد تستفيد جزئيًا من Threading لأن جزء من العمل بيتنفذ خارج الـ GIL في كود C.
- خدمة نموذج مدرَّب (Inference) عبر API: تصميم يجمع Async للطلبات وProcess Pool للحسابات الثقيلة.

أفضل الممارسات والأخطاء الشائعة

> ✅ **أفضل الممارسات**
>
> اسأل دايمًا: الكود ده بيستخدم مكتبة بتنفذ خارج الـ GIL (زي NumPy) ولا كود Python نقي؟ الإجابة بتغيّر القرار كليًا.
>
> لو مش متأكد من نوع المهمة، قِس فعليًا (Benchmark) الفرق بين Threading وMultiprocessing على البيانات الحقيقية بتاعتك قبل ما تقرر.
>
> استخدم Multiprocessing بحذر مع البيانات الكبيرة، لأن نقل البيانات بين Processes له تكلفة Serialization (هتتشرح بالتفصيل في فصل الأداء).

> ⚠️ **خطأ شائع**
>
> استخدام Threading على أمل تسريع كود Python نقي حسابي ثقيل — الـ GIL هيمنع أي تحسن حقيقي.
>
> الخلط بين "الكود شغال على أكتر من Thread" و"الكود بيتنفذ فعليًا بالتوازي" — في Python، ده مش نفس الشيء بسبب الـ GIL.

تجاهل تكلفة نقل البيانات بين Processes في Multiprocessing، والمقارنة الظالمة بينه وبين Threading من غير احتساب التكلفة دي.

اعتبارات الأداء والتوسع

الـ GIL بيحدد سقف واضح لأداء الـ CPU-bound tasks باستخدام Threading وحده، بغض النظر عن عدد الـ Cores المتاحة على السيرفر. أي تصميم نظام لازم ياخد السقف ده في الاعتبار من البداية، ويخطط لاستخدام Multiprocessing أو Distributed Computing للتوسع الأفقي في المهام الحسابية الثقيلة، بدل الاعتماد على زيادة عدد الـ Threads كحل وحيد.

مثال عملي صغير

مقارنة مباشرة توضح تأثير الـ GIL: نفس مهمة حسابية ثقيلة، مرة على Threads ومرة على Processes.

```
import time, threading, multiprocessing

:def cpu_bound(n)
x = 0
:for _ in range(n)
x += 1
return x

N = 25_000_000

محكوم بالـ GIL، أداء لا يتحسن فعليًا — Threading #
()start = time.time
threads = [threading.Thread(target=cpu_bound, args=(N,)) for _ in range(4)]
()for t in threads: t.start
()for t in threads: t.join
print(f"Threading: {time.time()-start:.2f}s")

Cores خاص به، استغلال حقيقي للـ GIL له Process كل — Multiprocessing #
:'__if __name__ == '__main
()start = time.time
procs = [multiprocessing.Process(target=cpu_bound, args=(N,)) for _ in
[range(4)
()for pr in procs: pr.start
()for pr in procs: pr.join
print(f"Multiprocessing: {time.time()-start:.2f}s")
```

مثال من مشروع Production حقيقي

في خط معالجة بيانات (Data Pipeline) لتحضير Features لنموذج Machine Learning، فريق هندسي بيلاحظ إن مرحلة تحويل البيانات (Data Transformation) بكود Python نقي بطيئة رغم استخدام Threading. بعد التحليل، بيتضح إن السبب هو الـ GIL — المرحلة دي CPU-bound بالكامل. الحل العملي: تحويل المرحلة لاستخدام Multiprocessing (أو الاستعانة بمكتبة زي Pandas/NumPy اللي بتنفذ جزء كبير من العمليات خارج الـ GIL)، وده بيدّي تسريع حقيقي وملحوظ متناسب مع عدد الـ Cores المتاحة.

مشروع صغير لتطبيق ما تم شرحه

📌 **ملاحظة مهمة — مشروع الفصل الثالث**

اكتب دالة حسابية ثقيلة بكود Python نقي (بدون NumPy)، وقِس وقت تنفيذها Sequential، ثم على ٤ Threads، ثم على ٤ Processes.

قارن النتائج التلاتة في جدول، واكتب استنتاجك الخاص عن تأثير الـ GIL بناءً على الأرقام الفعلية اللي حصلت عليها.

كرر نفس التجربة لكن بمهمة I/O-bound (زي تحميل عدة صفحات ويب)، ولاحظ الفرق في النتيجة بين Threading هنا وفي المهمة الحسابية.

تحديات شائعة

- صعوبة التفرقة بين الكود اللي بيتنفذ فعليًا خارج الـ GIL (زي أجزاء من NumPy) والكود اللي محكوم بالكامل بالـ GIL.
- الخلط بين نتائج قياس الأداء على مهام صغيرة جدًا (اللي بيطغى عليها الـ Overhead) واستنتاج عام غلط عن سلوك الـ GIL.
- عدم احتساب تكلفة نقل البيانات (Serialization/Pickling) عند الانتقال من Threading لـ Multiprocessing.

ملخص الفصل

- الـ GIL هو قفل واحد بيضمن إن Thread واحد بس ينفذ Python Bytecode في أي لحظة، حتى لو فيه عدة Cores متاحة.
- اتصمم أصلًا عشان يحل مشكلة أمان Reference Counting بدون التكلفة العالية لأقفال منفصلة على كل عملية.
- بيسيب تلقائيًا وقت عمليات I/O، فـ Threading فعّال جدًا للمهام I/O-bound رغم وجوده.
- بيمنع أي تسريع حقيقي للمهام CPU-bound باستخدام Threading وحده — الحل هو Multiprocessing أو مكتبات بتنفذ خارج الـ GIL.
- فيه توجه مستقبلي (PEP 703) لإتاحة نسخة اختيارية من Python بدون GIL، لكنه لسه مش الوضع الافتراضي في بيئات الإنتاج.

دلوقتي وإحنا فاهمين قيود الـ GIL، جاء وقت الغوص العملي في أول أداة Concurrency هنستخدمها بالتفصيل: Threading. الفصل الرابع هيشرحلك إزاي تنشئ Threads، تديرها، تنسّق بينها بأمان باستخدام أدوات زي Locks وSemaphores وConditions، وإزاي تتجنب المشاكل الشهيرة زي Race Conditions وDeadlocks اللي بتظهر بالتحديد لما تشتغل بأكتر من Thread بيشاركوا نفس البيانات.

الفصل الرابع: Threading بالتفصيل

في الفصل التاني اتعرفنا على مفهوم الـ Thread، وفي الفصل التالت فهمنا إزاي الـ GIL بيحدد امتى Threading بيبقى مفيد فعليًا (المهام I/O-bound) وامتى مش بيقدم فايدة حقيقية (المهام CPU-bound). دلوقتي جاء وقت الغوص العملي: إزاي تستخدم مكتبة threading في Python صح، إزاي تنسّق بين Threads بتشارك بيانات مشتركة بأمان، وإزاي تتجنب المشاكل الكلاسيكية اللي بتظهر بالتحديد في الأنظمة متعددة الـ Threads: Race Conditions، Deadlocks، Starvation، وLivelock.

المشكلة التي يحلها التنسيق بين Threads

لما يكون عندك أكتر من Thread بيشتغلوا على نفس البيانات المشتركة (Shared State) في نفس الوقت، ظهور نتائج غير متوقعة أو غلط بيبقى احتمال حقيقي جدًا، مش نظري. المشكلة مش في وجود الـ Threads نفسها، المشكلة في التعامل غير المنسّق مع البيانات المشتركة بينهم.

```
counter = 0

:()def increment
global counter
:for _ in range(100_000)
:عملية تبدو بسيطة، لكنها فعليًا 3 خطوات داخلية #    counter += 1
إعادة تخزينها (3  زيادتها (2  قراءة القيمة الحالية (1 #

لو شغّلنا الدالة دي على Threadين في نفس الوقت، #
النتيجة النهائية المتوقعة هي 200,000، لكن عمليًا #
ممكن تطلع أقل من كده بسبب تداخل الخطوات التلاتة بين الـ Threads #
```

Race Condition

الفكرة الأساسية

الـ Race Condition بتحصل لما نتيجة البرنامج بتعتمد على 'مين هيوصل الأول' بين Threadين أو أكتر بيحاولوا يعدّلوا نفس البيانات في نفس الوقت. حتى عملية زي counter += 1 اللي بتبان سطر واحد، فعليًا بتتحول لعدة خطوات جوه المفسّر، وممكن الـ Scheduler يبدّل بين الـ Threads في أي نقطة من الخطوات دي، فتحصل نتيجة غير متوقعة.

> ⚠️ **خطأ شائع**
>
> Race Condition من أخطر أنواع الـ Bugs لأنها مش بتظهر دايمًا — ممكن الكود يشتغل صح ٩٩ مرة، وفي المرة المية يطلع نتيجة غلط، وده بيخليها صعبة جدًا في الاكتشاف والتكرار (Reproduce) أثناء الاختبار.

الحل: Thread Synchronization وLocks

Lock — القفل الأساسي

الـ Lock هو أبسط أداة تزامن (Synchronization) في Python. بيضمن إن Thread واحد بس في اللحظة الواحدة يقدر يدخل جزء معين من الكود (اسمه Critical Section) — الجزء اللي بيتعامل مع البيانات المشتركة.

```
import threading

counter = 0
()lock = threading.Lock

:()def increment
global counter
:for _ in range(100_000)
الأول Lock لازم ياخد الـ Thread الـ #           :with lock
واحد بس هنا في كل لحظة counter += 1      # Critical Section – Thread
with block بيتحرر تلقائيًا لما نخرج من الـ Lock الـ #
```

✅ أفضل الممارسات

استخدم دايمًا with lock: بدل lock.acquire() وlock.release() اليدوية. لو حصل استثناء (Exception) جوه الكود، الـ with block بيضمن تحرير الـ Lock تلقائيًا، بينما النداء اليدوي ممكن يسيب الـ Lock ماسك للأبد لو الكود اتعطل قبل ما يوصل لـ release().

RLock — Reentrant Lock

مشكلة الـ Lock العادي: لو نفس الـ Thread حاول ياخد نفس الـ Lock مرتين (زي لما دالة بتاخد Lock وتنادي دالة تانية بتحاول تاخد نفس الـ Lock)، هيحصل Deadlock — الـ Thread هيستنى نفسه للأبد. الـ RLock (Reentrant Lock) بيحل المشكلة دي: بيسمح لنفس الـ Thread ياخد نفس الـ Lock أكتر من مرة، وبيتطلب نفس عدد مرات التحرير (release) بقد عدد مرات الأخذ (acquire).

```
import threading

()lock = threading.RLock

:()def outer
:with lock
RLock مع مسموح – تاني Lock الـ نفس بياخد Thread الـ نفس #    ()inner

:()def inner
:with lock
pass
```

Semaphore

الـ Semaphore شبيه بالـ Lock، لكن بدل ما يسمح بـ Thread واحد بس، بيسمح بعدد محدد (N) من الـ Threads يدخلوا الـ Critical Section في نفس الوقت. مفيد جدًا لما تحتاج تحدّد عدد الاتصالات المتزامنة مع مورد محدود (زي عدد اتصالات قاعدة بيانات، أو عدد طلبات API متزامنة مسموح بيها).

```
import threading

الخارجي API الـ مع بس متزامنة اتصالات 3 بـ السماح #
semaphore = threading.Semaphore(3)

:()def call_external_api
:with semaphore
```

```
# جوه هنا، 3 Threads بالكتير يقدروا يشتغلوا في نفس الوقت
()make_request
```

Condition

الـ Condition بتُستخدم لما Thread محتاج ينتظر لحد ما شرط معين يتحقق، وThread تاني هو اللي هيخبره (Notify) لما الشرط يتحقق. مفيدة جدًا في نمط Producer-Consumer (هنشرحه بالتفصيل في الفصل الثامن) — مثلًا Consumer بينتظر لحد ما Producer يضيف عنصر جديد للـ Queue.

```
import threading

()condition = threading.Condition
[] = queue

:()def consumer
:with condition
:while not queue
مؤقتًا لحد ما يتم إشعاره Lock ينتظر ويسيب الـ #    ()condition.wait
item = queue.pop(0)

:def producer(item)
:with condition
queue.append(item)
واحد مستني إن الشرط اتحقق Thread يخبر #     ()condition.notify
```

Event

الـ Event هي أبسط أداة إشارة (Signaling) بين Threads: علم (Flag) بسيط إما True أو False. Thread واحد أو أكتر يقدروا ينتظروا لحد ما Thread تاني يرفع العلم (set()).

```
import threading

()ready = threading.Event

:()def worker
("...مستني الإشارة")print
()ready.set يقف هنا لحد ما حد يعمل #           ()ready.wait
("استلمت الإشارة، بدأت الشغل!")print

:()def controller
# بعد ما الإعدادات تخلص
مستني يتحرك Thread يرفع العلم، وأي #           ()ready.set
```

Barrier

الـ Barrier بتخلي مجموعة من الـ Threads تستنى بعضها البعض لحد ما كل واحد فيهم يوصل لنقطة معينة، وبعدين يكملوا كلهم مع بعض في نفس اللحظة تقريبًا. مفيد في المهام اللي محتاجة كل الـ Threads تخلص مرحلة معينة قبل ما أي حد يبدأ المرحلة اللي بعدها.

```
import threading

threads 3 هينتظر #   barrier = threading.Barrier(3)

:def worker(id)
("خلّص المرحلة الأولى" f"Thread {id}")print
threads 3 يوصلوا هنا ينتظر لحد ما كل الـ #   ()barrier.wait
("بدأ المرحلة التانية" f"Thread {id}")print
```

مقارنة أدوات التزامن

مثال عملي	الاستخدام الأساسي	الأداة
تعديل عداد مشترك	منع أكتر من Thread من دخول Critical Section في نفس الوقت	Lock
دوال متداخلة بتاخد نفس القفل	نفس الـ Lock لكن يسمح لنفس الـ Thread يأخذه أكتر من مرة	RLock
تحديد عدد اتصالات متزامنة مع مورد	تحديد عدد Threads مسموح لهم بالدخول في نفس الوقت	Semaphore
نمط Producer-Consumer	انتظار شرط معين مع إشعار من Thread تاني	Condition
انتظار بدء التشغيل بعد اكتمال الإعداد	إشارة بسيطة True/False بين Threads	Event
تقسيم العمل لمراحل متزامنة	تزامن مجموعة Threads عند نقطة معينة	Barrier

Deadlock

الفكرة الأساسية

الـ Deadlock بيحصل لما Threadين (أو أكتر) بيستنوا بعض للأبد: Thread A ماسك Lock 1 وبينتظر Lock 2، في نفس الوقت Thread B ماسك Lock 2 وبينتظر Lock 1. مفيش حد فيهم هيسيب اللي في إيده، فالاتنين هيقفوا للأبد.

```
import threading

()lock_a = threading.Lock
()lock_b = threading.Lock

:()def thread_1
:with lock_a
lock_a → Deadlock وبينتظر lock_b ماسك thread_2 لو #   :with lock_b
pass

:()def thread_2
:with lock_b
ترتيب عكسي – ده اللي بيسبب المشكلة #   :with lock_a
pass
```

✅ **أفضل الممارسات**

الحل الأبسط والأكثر موثوقية لتجنب الـ Deadlock: التزم دايمًا بترتيب ثابت لأخذ الأقفال في كل مكان في الكود (مثلًا: دايمًا lock_a قبل lock_b، في كل الدوال بدون استثناء).

Starvation

الـ Starvation بتحصل لما Thread معين بيفضل مايجيلوش فرصة يشتغل، لأن Threads تانية (بأولوية أعلى أو بحظ أوفر في الجدولة) بتاخد الموارد أو الـ CPU باستمرار. الـ Thread المتضرر مش واقف Deadlock، لكنه عمليًا بيتأخر لأجل غير مسمى.

Livelock

الـ Livelock حالة مشابهة للـ Deadlock لكن أعقد: الـ Threads مش واقفين تمامًا، هما فعليًا بيتحركوا وبيغيروا حالتهم باستمرار، لكن من غير أي تقدم فعلي — زي شخصين ماشيين في ممر ضيق، كل واحد بيتنحى عشان يسيب التاني يعدي، وبالصدفة بيتنحوا لنفس الاتجاه في نفس اللحظة، فيفضلوا يتصادموا من غير ما حد يعدي.

السبب النموذجي	الحالة	المشكلة
تعديل بيانات مشتركة بدون تزامن	نتيجة غير متوقعة	Race Condition
انتظار دائري متبادل على أقفال متعددة	توقف كامل للأبد	Deadlock
توزيع غير عادل للموارد أو الأولوية	تأخير غير محدود لـThread معين	Starvation
استجابة متبادلة خاطئة لمحاولة تجنب تعارض	حركة مستمرة بدون تقدم فعلي	Livelock

كيف تُستخدم هذه الأدوات داخل الشركات الكبرى

في أنظمة الإنتاج الحقيقية، Locks وSemaphores بتُستخدم كتير في إدارة الوصول لموارد محدودة زي Connection Pools لقواعد البيانات، أو تحديد عدد الطلبات المتزامنة المسموح بيها لخدمة خارجية عشان متعملش Overload. لكن الاتجاه العام في الأنظمة الحديثة هو تقليل الاعتماد على Locking اليدوي المباشر قدر الإمكان، واستبداله بهياكل بيانات آمنة جاهزة (Thread-safe Data Structures) زي queue.Queue، أو الانتقال لنماذج Async اللي بتقلل الحاجة للـ Locking من الأساس.

🏢 **من واقع الشركات**

قاعدة معروفة بين المهندسين المحترفين: كل ما قللت عدد الأقفال والتعقيد في التزامن اليدوي، قل احتمال حدوث Bugs صعبة الاكتشاف زي Deadlocks. استخدم هياكل بيانات جاهزة وآمنة (Thread-safe) قبل ما تلجأ لـ Locking يدوي.

أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي

- تحديث Cache مشترك بين عدة Threads بتخدم طلبات Inference: يحتاج Lock لضمان الاتساق.
- تحديد عدد الاتصالات المتزامنة مع قاعدة بيانات الـ Features: مناسب لـ Semaphore.
- خط أنابيب معالجة بيانات فيه مرحلة تجميع (Aggregation) بتنتظر كل الـ Workers يخلصوا: مناسب لـ Barrier.
- نظام تسجيل (Logging) مشترك بين عدة Threads: يحتاج تزامن لضمان عدم تداخل الرسائل أو فقدها.

متى تستخدم Threading ومتى لا تستخدمه

الحالة	استخدم Threading	لا تستخدمه
مهام I/O-bound بعدد معقول (عشرات)	نعم	—
مهام CPU-bound حسابية	—	استخدم Multiprocessing بدلًا منه
مهام I/O-bound بعدد ضخم (آلاف الاتصالات)	—	استخدم Async بدلًا منه لتكلفة أقل
الحاجة لمشاركة بيانات بسيطة بسرعة	نعم	—

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

استخدم with lock: دايمًا بدل acquire()/release() اليدوية.

التزم بترتيب ثابت لأخذ الأقفال المتعددة في كل الكود لتجنب Deadlock.

قلّل حجم الـ Critical Section لأقصى درجة ممكنة — امسك القفل لأقل وقت ممكن.

فكّر في هياكل بيانات Thread-safe جاهزة (زي queue.Queue) قبل ما تبني Locking يدوي بنفسك.

⚠️ خطأ شائع

نسيان تحرير Lock بعد استثناء (Exception) — استخدم with بدل الاستدعاء اليدوي دايمًا.

أخذ أكتر من Lock بترتيب مختلف في أماكن مختلفة من الكود — سبب شائع جدًا للـ Deadlock.

استخدام Lock عادي بدل RLock في دوال متداخلة بتاخد نفس القفل — بيسبب Deadlock فوري مع نفس الـ Thread.

الإفراط في استخدام Locking لدرجة إن الكود يفقد فايدة الـ Concurrency أصلًا (Lock Contention عالي جدًا).

اعتبارات الأداء، الأمان، وقابلية التوسع

الأداء

كل Lock بيضيف تكلفة (Overhead) في الأخذ والتحرير، وكل ما زاد عدد الـ Threads المتنافسة على نفس القفل (Lock Contention)، زاد الوقت الضائع في الانتظار بدل التنفيذ الفعلي. الهدف دايمًا: أصغر Critical Section ممكن، وأقل عدد أقفال ممكن.

الأمان

البيانات المشتركة غير المحمية بشكل صحيح ممكن تؤدي لتسريب معلومات بين طلبات مستخدمين مختلفين في نظام خدمة، خصوصًا لو كان فيه حالة مشتركة (Shared State) بين طلبات API مختلفة بيتم التعامل معاها في Threads متوازية.

قابلية التوسع

النظام اللي بيعتمد بشكل كبير على Locking اليدوي المعقد بيصعب توسيعه وصيانته مع نمو الفريق وحجم الكود. النظم اللي بتقلل الحالة المشتركة (Shared State) من الأساس — عن طريق تمرير البيانات بدل مشاركتها، أو استخدام أنماط Message Passing (هنشرحها في الفصل الثامن) — بتكون أسهل في التوسع والصيانة على المدى الطويل.

مثال عملي صغير

مقارنة مباشرة بين نسخة فيها Race Condition ونسخة محمية بـ Lock:

```
import threading

:()def run_unsafe
counter = 0
:()def inc
nonlocal counter
:for _ in range(100_000)
counter += 1
threads = [threading.Thread(target=inc) for _ in range(4)]
()for t in threads: t.start
()for t in threads: t.join
("(400000 :المتوقع) Lock: {counter} بدون"f)print

:()def run_safe
counter = 0
()lock = threading.Lock
:()def inc
nonlocal counter
:for _ in range(100_000)
:with lock
counter += 1
threads = [threading.Thread(target=inc) for _ in range(4)]
()for t in threads: t.start
()for t in threads: t.join
("(400000 :المتوقع) Lock: {counter} مع"f)print
```

مثال من مشروع Production حقيقي

في خدمة Model Serving بتحتفظ بـ Cache داخل الذاكرة (In-memory Cache) للنتائج المتكررة عشان تقلل عدد مرات تنفيذ الـ Inference الفعلي، عدة Threads بتخدم طلبات مختلفة وبتحاول تقرأ وتكتب في نفس الـ Cache في نفس الوقت. بدون Lock مناسب، ممكن يحصل تلف في بيانات الـ Cache (Corrupted State) أو نتائج غلط ترجع للمستخدم. الحل العملي: استخدام Lock حوالين عمليات الكتابة في الـ Cache، مع تصميم يقلل وقت مسك الـ Lock لأقل قدر ممكن عشان ميأثرش على زمن استجابة الطلبات.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل الرابع**
>
> طبّق نمط Producer-Consumer بسيط باستخدام threading.Condition: Producer بيضيف أرقام عشوائية لقائمة كل ثانية، وConsumer بياخدها ويطبعها فور توفرها.

جرّب تعمل Deadlock مقصود بأخذ Lockين بترتيب مختلف في Threadين، وشوف البرنامج بيتوقف فعليًا، وبعدين صلّحه بتوحيد ترتيب الأخذ.

استخدم Semaphore بقيمة ٢ لمحاكاة نظام بيسمح باتصالين متزامنين بس مع 'مورد' وهمي، وجرّب تشغّل ٥ Threads وشوف السلوك.

تحديات شائعة

- صعوبة اكتشاف Race Conditions لأنها مش بتظهر في كل تشغيل — محتاجة اختبار متكرر أو أدوات تحليل متخصصة (هنتعرف عليها في الفصل الثالث عشر).
- صعوبة تحديد الحجم الأمثل للـ Critical Section: كبير جدًا يقلل الفايدة من الـ Concurrency، وصغير جدًا زيادة عن اللزوم ممكن تفتح باب لأخطاء تزامن دقيقة.
- نسيان إن أدوات التزامن نفسها (Lock, Semaphore..) بتضيف تعقيد للكود، فلازم تتستخدم بس لما تكون فعلاً ضرورية.

ملخص الفصل

- Race Condition بتحصل لما أكتر من Thread يعدّلوا بيانات مشتركة بدون تزامن، والنتيجة بتعتمد على توقيت غير مضمون.
- Lock وRLock وSemaphore وCondition وEvent وBarrier أدوات أساسية للتنسيق الآمن بين Threads، وكل واحدة مناسبة لسيناريو مختلف.
- Deadlock (انتظار دائري)، Starvation (حرمان مستمر)، وLivelock (حركة بدون تقدم) هي المشاكل الكلاسيكية اللي لازم تصممي الكود عشان تتجنبها.
- الترتيب الثابت لأخذ الأقفال، وتقليل حجم الـ Critical Section، واستخدام with دايمًا، هي أهم قواعد الأمان العملية.
- الأنظمة الحديثة بتميل لتقليل الحاجة لـ Locking اليدوي عن طريق هياكل بيانات جاهزة أو أنماط Message Passing.

لحد دلوقتي كل الأدوات اللي اتكلمنا عنها بتشتغل جوه نفس الـ Process (Threading)، ومحكومة بقيود الـ GIL للمهام الحسابية. الفصل الخامس هيوسّع الأفق: هنتعرف على Multiprocessing، الأداة اللي بتخليك تتجاوز الـ GIL فعليًا وتستغل كل الـ CPU cores المتاحة على الجهاز، هنشرح إزاي تدير Process Pools، تتشارك بيانات بين Processes معزولة تمامًا عن بعض، وإزاي تختار العدد الأمثل من الـ Workers لمهمتك.

الفصل الخامس: Multiprocessing

في الفصل التالت فهمنا إن الـ GIL بيمنع Threading من إعطاء تسريع حقيقي للمهام CPU-bound، وفي الفصل الرابع اتعلمنا إزاي ننسّق بين Threads بأمان. دلوقتي جاء وقت الأداة اللي فعليًا بتتجاوز قيد الـ GIL: Multiprocessing. الفصل ده هيوريك إزاي تستغل كل الـ CPU cores المتاحة على الجهاز فعليًا، إزاي تدير مجموعة Processes بكفاءة (Process Pool)، وإزاي تتشارك بيانات بين Processes معزولة تمامًا عن بعضها البعض.

لماذا نحتاج إلى Multiprocessing؟

المشكلة التي يحلها

عندك سيرفر بـ ٨ CPU cores، وعندك مهمة حسابية ثقيلة (زي معالجة صور، أو تحويل بيانات ضخمة) مكتوبة بكود Python نقي. لو استخدمت Threading، الـ GIL هيضمن إن core واحد بس هو الشغال فعليًا في أي لحظة، بغض النظر عن عدد الـ Threads — يعني ٧ cores من الـ ٨ قاعدين من غير فايدة. المشكلة اللي Multiprocessing جاي يحلها: إزاي نستغل كل الـ ٨ cores دول فعليًا في نفس اللحظة.

كيف كانت المشكلة تُحل قبل ظهور مكتبة multiprocessing المريحة

قبل ما مكتبة multiprocessing المريحة تظهر في Python، المبرمجين كانوا مضطرين يستخدموا نداءات نظام التشغيل مباشرة (زي os.fork() على Unix) بشكل يدوي، وده كان معقد جدًا، خاص بنظام تشغيل معين (مش هيشتغل بنفس الشكل على Windows)، وسهل الوقوع في أخطاء زي تسريب الموارد أو عدم انتظار الـ Processes الفرعية بشكل صحيح (Zombie Processes).

> 📌 **ملاحظة مهمة**
>
> مكتبة multiprocessing في Python بتوفر واجهة برمجية (API) موحدة وسهلة، بتشتغل بنفس الشكل تقريبًا على Windows وLinux وmacOS، وبتخفي تفاصيل نداءات نظام التشغيل المعقدة عن المطور.

الفكرة الأساسية

كل Process يتم إنشاؤه عن طريق multiprocessing بيحصل على نسخة مستقلة تمامًا من مفسّر Python، بذاكرتها الخاصة وGIL الخاص بيها. بما إن كل Process معزول تمامًا، مفيش تنافس على GIL واحد مشترك، وكل Process يقدر يشتغل فعليًا على Core منفصل في نفس اللحظة بالظبط — ده Parallelism حقيقي، مش مجرد Concurrency.

```
import multiprocessing, time

:def cpu_heavy(n)
x = 0
:for _ in range(n)
x += 1
return x

:'__if __name__ == '__main
N = 25_000_000
()start = time.time
processes = [multiprocessing.Process(target=cpu_heavy, args=(N,)) for _ in
[range(4)
```

```
()for p in processes: p.start
()for p in processes: p.join
print(f"4 Processes: {time.time()-start:.2f}s")
على جهاز بـ 4 cores أو أكتر، الوقت هيكون قريب من ربع #
وقت تنفيذ نفس المهمة على Thread واحد – تسريع حقيقي #
```

Process Pool وPool Executors

المشكلة التي يحلها Process Pool

إنشاء Process جديد لكل مهمة صغيرة مكلف (زي ما اتكلمنا في الفصل التاني). لو عندك ١٠,٠٠٠ مهمة صغيرة، إنشاء ١٠,٠٠٠ Process منفصل هيبقى أبطأ بكتير من الفايدة المرجوة. الحل: نعمل مجموعة ثابتة من الـ Processes (Pool) من البداية، ونوزع المهام عليهم، وكل Process بيشتغل على أكتر من مهمة طول عمره، بدل إعادة إنشاء Process لكل مهمة.

```
from multiprocessing import Pool

:def square(x)
return x * x

:'__if __name__ == '__main
:with Pool(processes=4) as pool
results = pool.map(square, range(10))
print(results)   # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
بتوزع الـ 10 مهام على processes 4 ثابتين، Pool الـ #
وكل Process بياخد أكتر من مهمة طول عمره #
```

ProcessPoolExecutor

واجهة أحدث وأكتر مرونة موجودة في concurrent.futures (هنشرحها بالتفصيل في الفصل السابع)، بتوفر نفس فكرة الـ Pool لكن بأسلوب برمجي أقرب لأسلوب الـ Futures، وبتسهّل التعامل مع نتائج المهام والأخطاء بشكل أوضح.

```
from concurrent.futures import ProcessPoolExecutor

:def square(x)
return x * x

:'__if __name__ == '__main
:with ProcessPoolExecutor(max_workers=4) as executor
results = list(executor.map(square, range(10)))
print(results)
```

✅ أفضل الممارسات

استخدم Pool أو ProcessPoolExecutor دايمًا بدل إنشاء Process منفصل لكل مهمة يدويًا، إلا لو كان عندك سبب واضح ومحدد (زي مهام قليلة جدًا طويلة العمر).

مشاركة البيانات بين Processes

بما إن كل Process معزول تمامًا في مساحة ذاكرة مستقلة، مفيش طريقة مباشرة لمشاركة متغير عادي بين Processes زي ما بيحصل مع Threads. لازم تستخدم آليات صريحة اسمها IPC (Inter-Process Communication).

Pipes

الـ Pipe قناة اتصال ثنائية الاتجاه بين Processين بالتحديد، بتسمح بإرسال واستقبال البيانات بينهم مباشرة.

```
from multiprocessing import Process, Pipe

:def sender(conn)
("مرحبًا من الـ Process الفرعي")conn.send
()conn.close

:'__if __name__ == '__main
()parent_conn, child_conn = Pipe
p = Process(target=sender, args=(child_conn,))
()p.start
استقبال الرسالة من الـ Process الفرعي #   print(parent_conn.recv())
()p.join
```

Queues

الـ Queue أكتر مرونة من الـ Pipe: بتسمح بالتواصل بين عدد أكبر من الـ Processes (مش بس اتنين)، وبتدعم نمط Producer-Consumer بشكل طبيعي وآمن (Thread-safe و Process-safe في نفس الوقت).

```
from multiprocessing import Process, Queue

:def producer(q)
:for i in range(5)
q.put(i)
إشارة إن الإنتاج خلص #   q.put(None)

:def consumer(q)
:while True
()item = q.get
:if item is None
break
("{item} :استلمت"f)print

:'__if __name__ == '__main
()q = Queue
p1 = Process(target=producer, args=(q,))
p2 = Process(target=consumer, args=(q,))
()p1.start(); p2.start
()p1.join(); p2.join
```

Shared Memory

لما تحتاج أداء أعلى من الـ Pipes/Queues (اللي بتعتمد على نسخ البيانات فعليًا بينهم — Serialization)، multiprocessing بيوفر Value و Array: مساحة ذاكرة فعلية مشتركة بين Processes بدون نسخ، مناسبة للبيانات البسيطة زي أرقام أو مصفوفات ثابتة الحجم.

```
from multiprocessing import Process, Value, Array

:def worker(shared_num, shared_arr)
shared_num.value += 10
:for i in range(len(shared_arr))
shared_arr[i] *= 2

:'__if __name__ == '__main
5 عدد صحيح مشترك، القيمة الابتدائية #          num = Value('i', 5)
مصفوفة أعداد صحيحة مشتركة #  arr = Array('i', [1, 2, 3])
p = Process(target=worker, args=(num, arr))
()p.start(); p.join
print(num.value, list(arr))  # 15 [2, 4, 6]
```

⚠️ خطأ شائع

الوصول لـ Shared Memory (زي Value وArray) لسه محتاج تزامن (Lock) لو أكتر من Process بيعدّلوا نفس القيمة في نفس الوقت — نفس مبدأ الـ Race Condition اللي شرحناه في الفصل الرابع، لكن هنا على مستوى Processes بدل Threads.

Managers

لو محتاج تشارك هياكل بيانات أعقد من رقم أو مصفوفة بسيطة (زي Dictionary أو List ديناميكية)، multiprocessing.Manager بيوفر نسخ 'مُدارة' (Managed) من هياكل البيانات دي، بتشتغل عن طريق Process وسيط بيدير الوصول لها من كل الـ Processes التانية بشكل آمن.

```
from multiprocessing import Process, Manager

:def worker(shared_dict, key, value)
shared_dict[key] = value

:'__if __name__ == '__main
:with Manager() as manager
()shared_dict = manager.dict
] = processes
Process(target=worker, args=(shared_dict, f"key{i}", i*10))
for i in range(5)
[
()for p in processes: p.start
()for p in processes: p.join
print(dict(shared_dict))
```

مقارنة طرق مشاركة البيانات بين Processes

الاستخدام المناسب	الأداء	الطريقة
اتصال مباشر بين Processين بالضبط	سريع	Pipe
Producer-Consumer بين عدة Processes	متوسط (Serialization لكل عنصر)	Queue

الاستخدام المناسب	الأداء	الطريقة
بيانات بسيطة وثابتة الحجم (أرقام، مصفوفات)	سريع جدًا (بدون نسخ)	Value / Array
هياكل بيانات معقدة (Dict, List) بمرونة أعلى	أبطأ نسبيًا (Process وسيط)	Manager

Synchronization بين Processes

نفس أدوات التزامن اللي اتعلمناها في الفصل الرابع (Lock, Semaphore, Event, Condition) متوفرة كمان جوه multiprocessing، لكن بتشتغل على مستوى Processes بدل Threads. المنطق نفسه: لو أكتر من Process بيعدّلوا نفس الـ Shared Memory، لازم تزامن يمنع الـ Race Conditions.

```
from multiprocessing import Process, Value, Lock

:def increment(shared_num, lock)
:for _ in range(100_000)
:with lock
shared_num.value += 1

:'__if __name__ == '__main
num = Value('i', 0)
()lock = Lock
processes = [Process(target=increment, args=(num, lock)) for _ in range(4)]
()for p in processes: p.start
()for p in processes: p.join
Lock بفضل الـ print(num.value)   # 400000
```

الأداء — Performance وCPU Parallelism

الفايدة الحقيقية من Multiprocessing بتظهر بوضوح في المهام CPU-bound الثقيلة، وبتقل كل ما زاد عدد الـ Processes عن عدد الـ CPU cores الفعلية المتاحة (زيادة العدد فوق كده بتزود الـ Overhead بس من غير فايدة إضافية). كمان لازم تاخد في الاعتبار تكلفة نقل البيانات بين Processes (Serialization/Pickling) — لو البيانات المُتبادلة كبيرة جدًا، التكلفة دي ممكن تلغي الفايدة اللي حصلت عليها من الـ Parallelism.

العامل	تأثيره على الأداء
عدد الـ Processes مقابل عدد الـ Cores الفعلية	الأمثل غالبًا يساوي عدد الـ Cores؛ الزيادة بعد كده بتضر الأداء
حجم البيانات المتبادلة بين Processes	بيانات كبيرة = تكلفة Serialization عالية تقلل الفايدة
حجم المهمة الواحدة (Task Granularity)	مهام صغيرة جدًا = تكلفة الإنشاء والتنسيق أكبر من الفايدة
استخدام Pool بدل Processes منفردة	يقلل تكلفة الإنشاء المتكرر بشكل كبير

أفضل الممارسات

قاعدة عملية شائعة: ابدأ بعدد Processes يساوي multiprocessing.cpu_count()، وقِس الأداء الفعلي، وعدّل بناءً على النتيجة — العدد المثالي بيعتمد كمان على طبيعة المهمة نفسها (هل فيها انتظار I/O جزئي أم لأ).

كيف تُستخدم داخل الشركات الكبرى

في أنظمة معالجة البيانات الضخمة (زي خطوط تجهيز الـ Features لتدريب النماذج)، Multiprocessing بيُستخدم كطبقة أساسية لتوزيع العمل على كل الـ Cores المتاحة على جهاز واحد، قبل حتى التفكير في التوزيع على عدة أجهزة (Distributed Computing). الأنظمة اللي بتحتاج توسع أكبر من جهاز واحد بتبني فوق نفس المبادئ دي لكن بأدوات متخصصة (زي Ray أو Spark)، اللي بدورها بتستخدم Multiprocessing وIPC كأساس تحتي.

أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي

- معالجة صور بالجملة (Batch Image Preprocessing) قبل التدريب: توزيع الصور على Process Pool.
- حساب Features معقدة على مجموعة بيانات كبيرة: تقسيم البيانات وتوزيعها على Processes متوازية.
- Hyperparameter Search: تشغيل عدة تجارب تدريب مستقلة بالتوازي على Processes مختلفة (هيتشرح تفصيليًا في الفصل الحادي عشر).
- Batch Inference على عدد ضخم من العينات: توزيع العينات على Process Pool لتسريع المعالجة الكلية.

متى تستخدم Multiprocessing ومتى لا تستخدمه

الحالة	استخدم Multiprocessing	لا تستخدمه
مهام CPU-bound حسابية ثقيلة	نعم	—
مهام I/O-bound بحتة	—	استخدم Threading أو Async بدلًا منه
مهام صغيرة جدًا وسريعة	—	الـ Overhead أكبر من الفايدة، استخدم Sequential
الحاجة لعزل قوي بين وحدات العمل	نعم	—
بيانات ضخمة محتاجة تُنقل باستمرار بين الوحدات	بحذر (فكر في Shared Memory)	قد تكون التكلفة أعلى من الفايدة

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

استخدم Pool أو ProcessPoolExecutor بدل إنشاء Processes منفردة يدويًا لمهام كتيرة.

احرص على وضع كود إنشاء الـ Processes داخل if __name__ == '__main__': (ضروري خصوصًا على Windows لتجنب حلقة إنشاء لا نهائية).

قِس عدد الـ Cores المتاحة فعليًا (multiprocessing.cpu_count()) قبل تحديد عدد الـ Workers.

قلّل حجم البيانات المنقولة بين Processes قدر الإمكان — مرر مسارات ملفات أو معرّفات بدل بيانات ضخمة كاملة لما أمكن.

⚠️ خطأ شائع

نسيان if __name__ == '__main__': على Windows — بيؤدي لإعادة تنفيذ السكريبت كله بشكل لا نهائي وإنشاء Processes بلا توقف.

استخدام Multiprocessing لمهام I/O-bound بسيطة — تكلفة الإنشاء والـ IPC أكبر بكتير من فايدة العزل هنا.

نقل بيانات ضخمة جدًا بين Processes عن طريق Queue أو Pipe عادي بدل استخدام Shared Memory، مما يضيف تكلفة Serialization كبيرة غير ضرورية.

افتراض إن زيادة عدد الـ Processes فوق عدد الـ Cores المتاحة هيحسن الأداء أكتر — ده مش صحيح غالبًا.

اعتبارات الأمان وقابلية التوسع

العزل الكامل للذاكرة بين Processes بيوفر أمان أعلى من Threading — تعطل Process واحد (Crash) ما بيأثرش على باقي الـ Processes التانية، وده مهم جدًا في أنظمة الإنتاج اللي محتاجة موثوقية عالية. من ناحية التوسع، الـ Multiprocessing بيستغل موارد الجهاز الواحد بالكامل، لكن للتوسع لأكتر من جهاز، لازم الانتقال لأنظمة Distributed Computing متخصصة تبني فوق نفس مبادئ الـ IPC والعزل.

مثال عملي صغير

مقارنة مباشرة بين Sequential وThreading وMultiprocessing لنفس المهمة الحسابية الثقيلة، عشان تشوف بعينك الفرق اللي شرحناه في الفصل التالت:

```
import time, threading, multiprocessing

:def cpu_task(n)
x = 0
:for _ in range(n)
x += 1

N = 20_000_000

()start = time.time
for _ in range(4): cpu_task(N)
print(f"Sequential: {time.time()-start:.2f}s")

()start = time.time
threads = [threading.Thread(target=cpu_task, args=(N,)) for _ in range(4)]
()for t in threads: t.start
()for t in threads: t.join
GIL لا يتحسن بسبب الـ #   print(f"Threading: {time.time()-start:.2f}s")

:'__if __name__ == '__main
()start = time.time
procs = [multiprocessing.Process(target=cpu_task, args=(N,)) for _ in
[range(4)
()for pr in procs: pr.start
()for pr in procs: pr.join
تسريع حقيقي وملحوظ #   print(f"Multiprocessing: {time.time()-start:.2f}s")
```

مثال من مشروع Production حقيقي

في خط تجهيز بيانات (Data Pipeline) لتدريب نموذج Computer Vision، عندك مليون صورة محتاجة تتعمل لها معالجة أولية (تغيير الحجم، تطبيع الألوان، استخراج بعض الـ Features البسيطة) قبل التدريب. المعالجة دي CPU-bound بالكامل.

الحل العملي: تقسيم قائمة الصور على ProcessPoolExecutor بعدد Workers يساوي عدد الـ CPU cores المتاحة، وكل Worker بيعالج جزء من الصور بشكل مستقل تمامًا، وفي الآخر بيتم تجميع النتائج. النهج ده بيقلل وقت المعالجة الكلي بنسبة تقارب عدد الـ Cores المستخدمة.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل الخامس**
>
> اكتب دالة بتحاكي معالجة صورة (زي حساب مجموع أرقام كبير كبديل بسيط للحساب الفعلي)، وطبّقها على قائمة من ٢٠ عنصر باستخدام ProcessPoolExecutor.
>
> قِس الوقت الكلي بعدد Workers مختلف (١, ٢, ٤, ٨) وارسم العلاقة بين عدد الـ Workers والوقت الكلي، ولاحظ عند أي نقطة التحسن بيبدأ يقل.
>
> جرّب تشارك عداد مشترك (Value) بين ٤ Processes بدون Lock، وشوف النتيجة الغلط، وبعدين صلّحها بإضافة Lock.

تحديات شائعة

- نسيان إن كل بيانات بتتبعت لـ Process تانية لازم تكون قابلة للـ Pickling (تحويلها لصيغة ممكن نقلها) — مش كل الكائنات في Python ممكن تتنقل بسهولة بين Processes.
- صعوبة تتبع الأخطاء (Debugging) لما بتحصل جوه Process فرعي — الأخطاء مش دايمًا بتظهر بوضوح في الـ Process الرئيسي بدون معالجة صحيحة.
- تقدير خاطئ للفايدة المتوقعة من الـ Parallelism بدون احتساب تكلفة الإنشاء والتنسيق الفعلية.

ملخص الفصل

- Multiprocessing بيتجاوز قيد الـ GIL عن طريق تشغيل عدة مفسّرات Python مستقلة، كل واحد بذاكرته والـGIL الخاص بيه.
- Process Pool (أو ProcessPoolExecutor) هو الأسلوب الأمثل لإدارة عدد كبير من المهام بدون تكلفة إنشاء Process لكل مهمة على حدة.
- مشاركة البيانات بين Processes تحتاج آليات صريحة: Pipes وQueues للتواصل، وValue/Array/Manager لمشاركة الحالة.
- أدوات التزامن (Lock, Semaphore..) لسه لازمة حتى مع Multiprocessing لو فيه Shared Memory بيتعدّل من أكتر من Process.
- الفايدة الحقيقية تظهر في المهام CPU-bound الثقيلة، وتقل أو تنعدم في المهام I/O-bound أو الصغيرة جدًا.

لحد دلوقتي غطينا Threading (للمهام I/O-bound) وMultiprocessing (للمهام CPU-bound). الفصل السادس هيقدّم أداة تالتة مختلفة تمامًا في الفلسفة: Async Programming. هنشرح إزاي الـ Event Loop بيدير عدد ضخم جدًا من المهام I/O-bound على Thread واحد بس بكفاءة عالية جدًا، من غير الحاجة لـ Threads أصلًا، وإزاي async وawait بيشتغلوا من الداخل، وامتى الأسلوب ده بيتفوق حتى على Threading نفسه.

الفصل السادس: Async Programming

في الفصل الرابع اتعلمنا Threading كحل للمهام I/O-bound، وفي الفصل الخامس اتعلمنا Multiprocessing كحل للمهام CPU-bound. دلوقتي هنتعرف على أداة تالتة، مختلفة تمامًا في الفلسفة: Async Programming. الفكرة إنها بتحل نفس مشكلة الـ I/O-bound اللي بيحلها Threading، لكن بأسلوب مختلف جذريًا: من غير ما تحتاج تنشئ ولا Thread واحد إضافي. بدل كده، Thread واحد بس بيدير آلاف المهام في نفس الوقت بذكاء شديد.

المشكلة التي يحلها Async Programming

حدود Threading مع الحمل الضخم

تخيل سيرفر محتاج يخدم ١٠,٠٠٠ اتصال متزامن (Concurrent Connections) — سيناريو عادي جدًا في تطبيقات الويب الحديثة. لو استخدمنا Thread منفصل لكل اتصال، هنحتاج ١٠,٠٠٠ Thread. كل Thread بياخد مساحة Memory معينة (الـ Stack بتاعه، عادة عدة ميجابايت)، وكل تبديل بين Threads (Context Switching) بياخد وقت معالجة حقيقي.
النتيجة: استهلاك ذاكرة هائل، وتكلفة تبديل عالية جدًا، حتى لو كل Thread فعليًا مش بيعمل حاجة أغلب وقته (بيستنى I/O بس).

> 📌 **ملاحظة مهمة**
>
> المشكلة مش في Threading نفسه، المشكلة في التوسع (Scale): Threading بيتعامل كويس جدًا مع عشرات أو مئات الاتصالات، لكن بيبدأ يترنح مع الآلاف، بسبب تكلفة الذاكرة والـ Context Switching لكل Thread إضافي.

كيف كانت المشكلة تُحل قبل انتشار Async

قبل انتشار الأسلوب الـ Async الحديث، السيرفرات كانت بتحل المشكلة دي بطرق زي Thread Pools محدودة العدد (بيتشارك فيها آلاف الاتصالات على عدد قليل نسبيًا من الـ Threads)، أو باستخدام آليات نظام تشغيل منخفضة المستوى (زي select أو epoll على Linux) بشكل يدوي ومعقد جدًا لإدارة عدد ضخم من الـ Sockets على Thread واحد. الأسلوب ده كان فعّال لكنه صعب جدًا في الكتابة والصيانة.

الفكرة الأساسية: Single Thread, Many Tasks

الـ Async Programming بيعتمد على مبدأ بسيط لكنه قوي جدًا: بدل ما ننشئ Thread لكل مهمة، بنكتب الكود بطريقة تسمح لمهمة توقف نفسها طوعًا (Voluntary Yield) لما تستنى I/O، وتسيب الفرصة لمهمة تانية تشتغل على نفس الـ Thread. مفيش تبديل قسري من نظام التشغيل زي الـ Threading، الكود نفسه هو اللي بيحدد نقاط التوقف بشكل صريح (عن طريق await).

Async	Threading	المعيار
Thread واحد يدير آلاف المهام	Thread لكل مهمة متزامنة	عدد وحدات التنفيذ
الكود نفسه عند await (طوعي)	نظام التشغيل (قسري)	من يقرر التبديل
منخفض جدًا (Coroutine خفيفة)	مرتفع نسبيًا (Stack خاص)	استهلاك الذاكرة لكل مهمة
ممتاز (عشرات الآلاف بسهولة)	محدود (آلاف قليلة بصعوبة)	مناسب لعدد اتصالات ضخم
يحتاج فهم async/await وEvent Loop	أقل تعقيدًا نسبيًا	التعقيد في الكتابة

Event Loop

الفكرة الأساسية

الـ Event Loop هو قلب نظام الـ Async بالكامل. هو حلقة تكرار مستمرة، بتراقب كل المهام (Coroutines) المسجلة عندها، وبتقرر مين شغال دلوقتي، ومين واقف بينتظر I/O، ومين جاهز يكمّل شغله بعد ما الـ I/O بتاعه خلص.

كيف يعمل داخليًا

17. الـ Event Loop بيمسك مهمة (Coroutine) جاهزة للتنفيذ من قائمة المهام.
18. بينفذها لحد ما توصل لنقطة await على عملية I/O (زي طلب شبكة).
19. بدل ما يقف وينتظر رد الـ I/O، الـ Event Loop بيسجّل الانتظار ده عند نظام التشغيل (عن طريق آليات زي epoll)، وينتقل فورًا لمهمة تانية جاهزة.
20. لما الـ I/O بتاع المهمة الأولى يخلص فعليًا (رد وصل من الشبكة مثلًا)، نظام التشغيل بيخبر الـ Event Loop، وهو بيرجع للمهمة دي ويكمّلها من حيث وقفت بالظبط.
21. الدورة دي بتتكرر باستمرار لحد ما كل المهام تخلص.

> 📌 **ملاحظة مهمة**
>
> الـ Event Loop بيدير كل ده على Thread واحد بس. مفيش Parallelism فعلي هنا (زي ما شرحنا في الفصل الأول) — ده Concurrency بحت، إدارة ذكية للانتظار، مش تنفيذ فعلي متزامن على أكتر من Core.

async، Coroutines، وawait

Coroutine

الـ Coroutine هي دالة خاصة بتقدر توقف تنفيذها مؤقتًا في نقطة معينة، وتسيب الفرصة لكود تاني يشتغل، وبعدين تكمّل من حيث وقفت بالظبط. في Python، بنعرّفها بكلمة async def.

```
import asyncio

:()async def say_hello
("البداية")print
نقطة توقف: يسيب الفرصة لمهام تانية #   await asyncio.sleep(1)
("بعد ثانية")print

نداء الدالة العادي say_hello()async def لا ينفذ الكود فعليًا، #
بيرجع كائن Coroutine لازم يتنفذ عن طريق الـ Event Loop: #
asyncio.run(say_hello())
```

await

كلمة await هي نقطة التوقف الصريحة: بتقول للـ Event Loop 'أنا مستني نتيجة العملية دي، ممكن تشتغل على حاجة تانية لحد ما أنا أبقى جاهز'. مينفعش تستخدم await إلا جوه دالة async def، ومينفعش تستخدمها مع دوال عادية (Synchronous) مش مصممة أصلًا للعمل مع asyncio.

> ⚠️ **خطأ شائع**
>
> غلط شائع جدًا: استخدام دالة Synchronous بطيئة (زي requests.get()) العادية جوه async def من غير await مخصص لها. الدالة دي هتوقف الـ Event Loop بالكامل (Blocking) لحد ما تخلص، وهتلغي كل فايدة الـ Async. لازم تستخدم نسخة Async مخصصة (زي aiohttp) بدلها.

Tasks وFutures

Task

الـ Task هي طريقة لتشغيل Coroutine 'في الخلفية' جوه الـ Event Loop، بدون ما تنتظرها فورًا بـ await. ده بيسمح لك تبدأ كذا مهمة في نفس الوقت، وتستناهم كلهم مع بعض بعد كده.

```
import asyncio

:async def fetch(name, delay)
await asyncio.sleep(delay)
"خلصت return f"{name}

:()async def main
(Sequential) اللي قبلها await لو استخدمنا مباشرة، كل واحدة هتستنى #
'في نفس الوقت' تشتغل كلها Tasks بدل كده، ننشئ #
task1 = asyncio.create_task(fetch("A", 2))
task2 = asyncio.create_task(fetch("B", 1))
result1 = await task1
result2 = await task2
3 مش ،ثانية 2~ الوقت الكلي #   print(result1, result2)

asyncio.run(main())
```

Future

الـ Future كائن بيمثل نتيجة عملية لسه ما خلصتش، هيكون متاح في المستقبل. الـ Task في asyncio هي فعليًا نوع خاص من الـ Future، بتلف حوالين Coroutine وبتدير تنفيذها. غالبًا كمطور، هتتعامل مع Tasks مباشرة أكتر من Futures الخام، لكن فهم إن Task = Future + Coroutine مفيد لفهم الـ API الداخلي لـ asyncio.

Timeout وCancellation

إلغاء مهمة — Cancellation

أي Task ممكن يتم إلغاؤها في أي وقت قبل ما تخلص، عن طريق task.cancel(). ده مفيد جدًا لو مثلًا المستخدم قفل الاتصال قبل ما السيرفر يخلص معالجة طلبه.

```
import asyncio

:()async def long_task
:try
await asyncio.sleep(10)
:except asyncio.CancelledError
```

```
("المهمة اتلغت قبل ما تخلص")print
مهم جدًا: إعادة رفع الاستثناء بعد التنظيف #   raise

:()async def main
task = asyncio.create_task(long_task())
await asyncio.sleep(1)
()task.cancel
await asyncio.gather(task, return_exceptions=True)

asyncio.run(main())
```

مهلة زمنية — Timeout

بدل ما تستنى مهمة لأجل غير مسمى، asyncio.wait_for بتسمح لك تحدد مهلة زمنية قصوى، ولو المهمة ما خلصتش في الوقت ده، بيتم إلغاؤها تلقائيًا ورفع استثناء TimeoutError.

```
import asyncio

:()async def slow_operation
await asyncio.sleep(5)
"تمام" return

:()async def main
:try
result = await asyncio.wait_for(slow_operation(), timeout=2)
:except asyncio.TimeoutError
("العملية أخذت وقت أطول من المسموح")print

asyncio.run(main())
```

أفضل الممارسات

في أي نظام Production حقيقي بيتواصل مع خدمات خارجية، وضع Timeout واضح على كل عملية I/O أمر أساسي وليس اختياريًا — بدونه، خدمة خارجية بطيئة أو متعطلة ممكن توقف نظامك بالكامل.

Async Context Managers

زي الـ Context Managers العادية (with)، لكن مصممة للعمل مع عمليات async — مفيدة جدًا لإدارة موارد محتاجة setup/teardown غير متزامن، زي فتح وإغلاق اتصال شبكة.

```
import aiohttp

:async def fetch_data(url)
:async with aiohttp.ClientSession() as session
:async with session.get(url) as response
()return await response.json
بيضمنوا إغلاق الاتصال تلقائيًا بشكل غير متزامن async with الاتنين #
```

Async Iterators وAsync Generators

زي الـ Iterators وGenerators العادية، لكن بتسمح بعملية await جوه كل خطوة من التكرار — مفيدة لما تكون بتقرأ بيانات بشكل تدريجي من مصدر غير متزامن (زي قراءة نتائج قاعدة بيانات على دفعات، أو Streaming بيانات من الشبكة).

```
:async def async_range(n)
:for i in range(n)
محاكاة عملية I/O بين كل عنصر والتاني #   await asyncio.sleep(0.1)
yield i

:()async def main
:async for number in async_range(5)
print(number)
```

Structured Concurrency

مبدأ برمجي مهم بيقول إن أي مهام فرعية (Tasks) تم إنشاؤها جوه دالة معينة، لازم تخلص (أو تتلغي) قبل ما الدالة الأم ترجع نتيجتها — بحيث ما يبقاش فيه Tasks 'شاردة' شغالة في الخلفية من غير حد بيتابعها أو بيدير دورة حياتها. asyncio.gather وasyncio.TaskGroup (في النسخ الحديثة من Python) بيوفروا الأسلوب ده بشكل مباشر.

```
import asyncio

:()async def main
:async with asyncio.TaskGroup() as tg
tg.create_task(fetch("A", 1))
tg.create_task(fetch("B", 2))
الخروج من الـ TaskGroup لا يحصل إلا بعد ما كل الـ Tasks تخلص #
أو لو واحدة فشلت، الباقي بيتلغي تلقائيًا – إدارة نظيفة ومضمونة #
```

> 🏢 **من واقع الشركات**
>
> قبل asyncio.TaskGroup، كان شائع إنشاء Tasks بـ asyncio.create_task() من غير متابعة واضحة لدورة حياتها، وده كان بيسبب مشاكل صعبة التتبع (Tasks بتفشل بصمت، أو بتفضل شغالة بعد ما الدالة الأم خلصت). الـ Structured Concurrency بيحل المشكلة دي بشكل منهجي.

كيف تُستخدم داخل الشركات الكبرى

Async Programming هو الأساس اللي بنيت عليه أغلب أطر عمل الـ Web الحديثة عالية الأداء (زي FastAPI في Python، أو Node.js بشكل عام). في خدمات بتستقبل آلاف الطلبات في الثانية، Async بيسمح للسيرفر يخدم عدد ضخم من الاتصالات المتزامنة بموارد Memory وCPU أقل بكتير من الاعتماد على Threading التقليدي، وده بيترجم مباشرة لتكلفة تشغيل أقل (Infrastructure Cost) لنفس الحمل.

أين يقع هذا الفهم داخل مشروع Machine Learning حقيقي

- استقبال طلبات Inference من عدد ضخم من المستخدمين في خدمة Model Serving مبنية بـ FastAPI.
- جمع بيانات أو Features من عدة APIs خارجية في نفس الوقت بأقل زمن استجابة ممكن.
- Streaming نتائج نموذج توليدي (زي نماذج اللغة) للمستخدم أول بأول بدل انتظار النتيجة كاملة.
- التواصل غير المتزامن مع قواعد بيانات وCaches (زي Redis) بكفاءة عالية داخل السيرفر.

متى تستخدم Async ومتى لا تستخدمه

الحالة	استخدم Async	لا تستخدمه
عدد ضخم من الاتصالات المتزامنة I/O-bound	نعم، الخيار الأمثل	—
مهام CPU-bound حسابية	—	استخدم Multiprocessing بدلًا منه
مكتبات خارجية Synchronous بحتة بلا بديل Async	بحذر (قد تحتاج تشغيلها في Thread/Process منفصل)	—
كود بسيط بعدد اتصالات قليل جدًا	غير ضروري، تعقيد زيادة	Threading أو Sequential أبسط

أفضل الممارسات والأخطاء الشائعة

✅ **أفضل الممارسات**

استخدم دايمًا مكتبات مصممة خصيصًا لـ Async (زي aiohttp بدل requests) داخل كود async def.

حط Timeout واضح على كل عملية I/O خارجية بدون استثناء.

استخدم asyncio.TaskGroup أو asyncio.gather لإدارة مجموعات المهام بدل ترك Tasks شاردة.

افهم إن Async بيحسّن الـ Concurrency للـ I/O، مش السرعة الحسابية — للمهام الحسابية استخدم Multiprocessing.

⚠️ **خطأ شائع**

استدعاء دالة Synchronous بطيئة (Blocking) جوه كود async — بتوقف الـ Event Loop بالكامل وتلغي كل فايدة الـ Async.

نسيان await قبل استدعاء Coroutine — بيرجع كائن Coroutine بدل النتيجة الفعلية، وده بيسبب أخطاء صامتة ومحيرة.

الاعتقاد إن Async بيسرّع الحسابات — هو بيحسّن إدارة الانتظار بس، مش المعالجة الفعلية.

إنشاء Tasks كتير بدون متابعة دورة حياتها (بدون TaskGroup أو gather)، مما يؤدي لأخطاء صامتة أو تسريب موارد.

اعتبارات الأداء، الأمان، وقابلية التوسع

الأداء

Async بيوفر أفضل استخدام ممكن للموارد في المهام I/O-bound الكثيفة، لأن مفيش تكلفة إنشاء Threads أو Context Switching بمعناه التقليدي. لكن لازم الحذر: أي عملية Blocking حتى لو صغيرة جوه كود Async بتوقف الـ Event Loop بالكامل، وده بيأثر على كل المهام التانية المنتظرة، مش بس المهمة الحالية.

الأمان

بما إن كل حاجة بتشتغل على Thread واحد، مخاطر Race Conditions التقليدية (زي في Threading) أقل بكتير، لأن مفيش تنفيذ فعلي متزامن حرفيًا. لكن لسه فيه مخاطر تزامن منطقية (Logical Race Conditions) لو مهمتين بيعدّلوا نفس البيانات بين نقطتين await متتاليتين.

قابلية التوسع

Async ممتاز جدًا للتوسع الرأسي (خدمة عدد أكبر من الاتصالات على نفس الجهاز)، لكن لسه محتاج Multiprocessing أو عدة Processes (زي عدة Workers خلف Gunicorn) عشان يستغل أكتر من CPU core، لأن الـ Event Loop نفسه بيشتغل على Thread واحد بس.

مثال عملي صغير

نرجع لمثال الفصل الأول (تحميل ٥ روابط)، ونشوف الفرق الفعلي بين Sequential وAsync:

```
import asyncio, aiohttp, time

urls = ["https://api.example.com/1", "https://api.example.com/2", "..."]

:async def fetch(session, url)
:async with session.get(url) as response
()return await response.text

:()async def main
:async with aiohttp.ClientSession() as session
tasks = [fetch(session, url) for url in urls]
results = await asyncio.gather(*tasks)
return results

()start = time.time
asyncio.run(main())
print(f"Async: {time.time() - start:.2f}s")
# مقارنة بالنسخة الـ Sequential من الفصل الأول، الفرق هيكون كبير جدًا
# مع زيادة عدد الروابط
```

مثال من مشروع Production حقيقي

في خدمة Model Serving مبنية على FastAPI، كل طلب Inference محتاج يجيب بيانات إضافية (Features) من قاعدة بيانات، ويستدعي خدمة Caching (زي Redis) للتحقق من وجود نتيجة محفوظة مسبقًا، وفي الآخر يرجع الرد للمستخدم. كل العمليات دي I/O-bound. باستخدام async def وawait في مسارات FastAPI، السيرفر بيقدر يخدم آلاف الطلبات المتزامنة بكفاءة عالية جدًا، لأن كل طلب بيسيب الفرصة للطلبات التانية أثناء انتظار قاعدة البيانات أو الـ Cache، بدل ما يحجز Thread كامل لنفسه طول فترة الانتظار.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل السادس**
>
> اكتب دالة async بتحاكي ٣ عمليات I/O مختلفة (باستخدام asyncio.sleep بأوقات مختلفة)، وشغّلها مرة Sequential (بـ await متتالي) ومرة باستخدام asyncio.gather، وقارن الوقت الكلي.
>
> جرّب تعمل Timeout على واحدة من العمليات باستخدام asyncio.wait_for، وشوف السلوك لما العملية تاخد وقت أطول من المسموح.
>
> اكتب Async Generator بسيط بيولّد أرقام بفاصل زمني، واستخدمه بـ async for.

تحديات شائعة

- صعوبة اكتشاف عملية Blocking مخفية جوه مكتبة خارجية بتستخدمها داخل كود async من غير ما تلاحظ.
- تعلم منحنى مختلف تمامًا عن البرمجة التقليدية: التفكير بشكل غير متزامن (Asynchronous Mindset) محتاج وقت وممارسة.
- صعوبة الـ Debugging لأن الـ Stack Trace في الأخطاء غير المتزامنة أحيانًا بيكون أعقد من الكود العادي التتابعي.

ملخص الفصل

- Async Programming بيدير عدد ضخم من المهام I/O-bound على Thread واحد بس، عن طريق Event Loop بيبدّل بينهم عند نقاط await الصريحة.
- Coroutines (async def) هي الوحدات القابلة للتوقف والاستكمال، وawait هي نقطة التوقف الصريحة.
- Tasks بتسمح بتشغيل عدة Coroutines 'في نفس الوقت' بدل انتظارهم واحدة واحدة.
- Cancellation وTimeout أساسيان في أي نظام Production حقيقي للتعامل مع العمليات البطيئة أو المعلّقة.
- Structured Concurrency (زي TaskGroup) بيضمن إن كل المهام الفرعية تتابَع وتتنظف بشكل صحيح.
- Async ممتاز للـ I/O-bound، لكنه مش بديل عن Multiprocessing للمهام CPU-bound.

لحد دلوقتي اتعلمنا ٣ أدوات منفصلة: Multiprocessing، وThreading، وAsync، وكل واحدة ليها API خاص بيها. الفصل السابع هيقدّملك concurrent.futures — مكتبة موحدة بتوفر واجهة برمجية واحدة ومتسقة للتعامل مع Threading وMultiprocessing مع بعض (عن طريق ThreadPoolExecutor وProcessPoolExecutor)، بأسلوب Futures نظيف وسهل القراءة، وهنشرح إزاي تستخدمها بفعالية في مشاريعك بدل التعامل المباشر مع threading أو multiprocessing في كل مرة.

الفصل السابع: concurrent.futures

لحد دلوقتي اتعلمنا ٣ أدوات منفصلة تمامًا: threading بواجهته الخاصة (Thread, Lock, Semaphore..)، multiprocessing بواجهته الخاصة (Process, Pool, Queue..)، وasyncio بفلسفته المختلفة تمامًا (Event Loop, Coroutines, await). المشكلة إن التنقل بين الـ APIs التلاتة دي محتاج تتعلم تفاصيل مختلفة لكل واحدة. مكتبة concurrent.futures جايه تحل المشكلة دي: واجهة برمجية موحدة وبسيطة، بتخليك تستخدم Threading أو Multiprocessing تقريبًا بنفس الكود، وبتفكرك بأسلوب Futures نظيف وسهل القراءة.

المشكلة التي تحلها concurrent.futures

لو عندك كود بيستخدم threading.Thread مباشرة، والنهاردة عايز تجرب تحول نفس الكود لـ multiprocessing.Process بدل الـ Threads (لأنك اكتشفت إن المهمة CPU-bound مش I/O-bound)، هتحتاج تعيد كتابة جزء كبير من الكود، لأن الـ APIs مختلفة شكلًا (إنشاء، بدء تشغيل، انتظار نتيجة، معالجة أخطاء).
concurrent.futures بتحل المشكلة دي عن طريق توحيد الواجهة: نفس الكود تقريبًا، بس تغيّر ThreadPoolExecutor لـ ProcessPoolExecutor، وخلاص.

> 📌 **ملاحظة مهمة**
>
> concurrent.futures مبنية فوق threading وmultiprocessing، مش بديل عنهم — هي طبقة تجريد (Abstraction Layer) بتوحد الواجهة وتبسّط الاستخدام اليومي، خصوصًا للحالة الشائعة جدًا: 'عندي مجموعة مهام، عايز أشغّلها بالتوازي وأجمع النتائج'.

الفكرة الأساسية: Executor وFuture

Executor

الـ Executor هو المدير المسؤول عن تشغيل المهام — إما على مجموعة Threads (ThreadPoolExecutor) أو مجموعة Processes (ProcessPoolExecutor). أنت بتقدّم له المهام، وهو بيدير توزيعها وتنفيذها من غير ما تحتاج تدير الـ Threads أو Processes بنفسك يدويًا.

Future

لما تقدّم مهمة للـ Executor، مش بيرجعلك النتيجة فورًا (لأن المهمة لسه مش خلصت). بدل كده، بيرجعلك كائن Future — 'إيصال' بيمثل نتيجة هتكون متاحة في المستقبل. تقدر تسأل الـ Future 'خلصت؟'، أو تستناه لحد ما يخلص وتاخد النتيجة الفعلية منه.

ProcessPoolExecutor وThreadPoolExecutor

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

:def task(n)
return n * n

I/O-bound لمهمة #
:with ThreadPoolExecutor(max_workers=4) as executor
future = executor.submit(task, 5)
```

```
print(future.result())   # 25

Executor نفس الكود بالظبط، بس غيّرنا اسم الـ — CPU-bound لمهمة #
:'__if __name__ == '__main
:with ProcessPoolExecutor(max_workers=4) as executor
future = executor.submit(task, 5)
print(future.result())   # 25
```

> 🏢 **من واقع الشركات**
>
> لاحظ إن الكود نفسه تقريبًا في الحالتين — ده جوهر فايدة concurrent.futures: توحيد الواجهة بغض النظر عن الأداة التحتية (Thread أم Process).

()submit

submit() بتقدّم مهمة واحدة للتنفيذ، وترجع Future فورًا من غير ما تستنى المهمة تخلص. ده بيسمح لك تقدّم كذا مهمة على التوالي بسرعة، وتستناهم كلهم مع بعض بعد كده.

```
from concurrent.futures import ThreadPoolExecutor

:def download(url)
محاكاة عملية تحميل #
"{url} محتوى"return f

urls = ["url1", "url2", "url3"]

:with ThreadPoolExecutor(max_workers=3) as executor
futures = [executor.submit(download, url) for url in urls]
:for future in futures
ويطبع نتيجته Future ينتظر كل #   print(future.result())
```

()map

لو عايز تطبّق نفس الدالة على قائمة من المدخلات (نمط شائع جدًا)، map() بتبسّط الكود أكتر من submit() في حلقة، وبترجع النتائج بنفس ترتيب المدخلات تلقائيًا.

```
from concurrent.futures import ThreadPoolExecutor

:def download(url)
"{url} محتوى"return f

urls = ["url1", "url2", "url3"]

:with ThreadPoolExecutor(max_workers=3) as executor
results = executor.map(download, urls)
:for result in results
بالظبط urls نفس ترتيب #   print(result)
```

المعيار	()submit	()map

نفس الدالة على قائمة مدخلات متجانسة	مهام مختلفة، أو محتاج تتحكم في كل Future على حدة	الاستخدام الأمثل
يحافظ على ترتيب المدخلات تلقائيًا	حسب ما أنت بتطلبه يدويًا	ترتيب النتائج
أقل مرونة، نتيجة موحدة	متاح لكل Future على حدة	التحكم الفردي (إلغاء، فحص حالة)
أبسط وأقصر للحالة المتجانسة	أطول شوية في الكتابة	البساطة للحالة الشائعة

()wait

wait() بتاخد مجموعة Futures وتستناها حسب شرط معين: كلهم يخلصوا (ALL_COMPLETED)، أو أول واحد يخلص (FIRST_COMPLETED)، أو أول واحد يفشل (FIRST_EXCEPTION). بترجعلك مجموعتين: اللي خلصوا (done) واللي لسه شغالين (not_done).

```
from concurrent.futures import ThreadPoolExecutor, wait, FIRST_COMPLETED

:with ThreadPoolExecutor(max_workers=3) as executor
futures = [executor.submit(download, url) for url in urls]
done, not_done = wait(futures, return_when=FIRST_COMPLETED)
("{len(not_done)} :لسه شغال ,{len(done)} :خلص لحد دلوقتي"f)print
```

()as_completed

as_completed() بترجعلك الـ Futures أول ما كل واحدة فيهم تخلص، بغض النظر عن ترتيب تقديمها — مفيدة جدًا لما عايز تعالج النتائج أول ما توصل، بدل ما تستنى كل المهام تخلص مع بعض.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

:with ThreadPoolExecutor(max_workers=3) as executor
futures = {executor.submit(download, url): url for url in urls}
:for future in as_completed(futures)
url = futures[future]
:try
()result = future.result
print(f"{url}: {result}")
:except Exception as e
("{e} :فشلت f"{url})print
```

✅ أفضل الممارسات

استخدم as_completed() لما عايز تعالج كل نتيجة أول ما توصل (زي عرض تقدم فوري للمستخدم)، واستخدم map() أو قائمة Futures عادية لما ترتيب النتائج مهم بالنسبالك.

معالجة الأخطاء مع Futures

لو مهمة معينة فشلت (رفعت Exception)، الخطأ ده بيتخزن جوه الـ Future، ومش بيظهر إلا لما تنادي future.result() — في اللحظة دي، الاستثناء بيتم رفعه من جديد في الكود اللي بيستنى النتيجة. ده مهم جدًا: لو نسيت تنادي result() على Future فشلت، ممكن الخطأ يمر بصمت من غير ما تلاحظه.

```
:def risky_task(n)
:if n == 0
("القيمة لا يمكن أن تكون صفر")raise ValueError
return 10 / n

:with ThreadPoolExecutor(max_workers=2) as executor
future = executor.submit(risky_task, 0)
:try
()result = future.result
:except ValueError as e
("{e} :حصل خطأ"f)print
```

⚠️ خطأ شائع

خطأ شائع: تقديم مهام كتيرة بـ submit() من غير ما تنادي result() على أي واحدة فيهم للتأكد من نجاحها. لو فشلت مهمة، الخطأ ممكن يختفي بصمت لو محدش نادى result() عليها أبدًا.

كيف تُستخدم داخل الشركات الكبرى

concurrent.futures هي الاختيار الافتراضي الشائع في كود الإنتاج لما تحتاج تشغّل مجموعة مهام بالتوازي بسرعة، من غير الحاجة لبناء إدارة يدوية معقدة لـ Threads أو Processes. كتير من الفرق الهندسية بتفضّلها على الاستخدام المباشر لـ threading أو multiprocessing للحالات الشائعة، وبتلجأ للاستخدام المباشر بس لما تحتاج تحكم دقيق جدًا (زي أدوات تزامن متقدمة مش متوفرة في الواجهة الموحدة).

أين تقع داخل مشروع Machine Learning حقيقي

- معالجة دفعة (Batch) من الطلبات في خدمة Inference بالتوازي باستخدام ThreadPoolExecutor أو ProcessPoolExecutor حسب طبيعة العملية.
- تحميل عدة ملفات بيانات أو نماذج مدربة مسبقًا في نفس الوقت أثناء بدء تشغيل الخدمة (Startup).
- تشغيل عدة تجارب Hyperparameter Search بالتوازي باستخدام ProcessPoolExecutor، مع جمع النتائج أول ما كل تجربة تخلص عن طريق as_completed().
- استدعاء عدة APIs خارجية لجمع Features بالتوازي باستخدام ThreadPoolExecutor.

متى تستخدم أيهما

الاختيار المناسب	الحالة
ThreadPoolExecutor	مهام I/O-bound (شبكة، قاعدة بيانات، ملفات)
ProcessPoolExecutor	مهام CPU-bound (حسابات، معالجة صور)
asyncio أفضل من ThreadPoolExecutor (الفصل السادس)	عدد ضخم جدًا من اتصالات I/O المتزامنة (آلاف)
concurrent.futures بشكل عام — الخيار الأبسط	مجموعة مهام متوازية بسيطة، عدد معقول

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

استخدم with عند إنشاء الـ Executor دايمًا، عشان يضمن إغلاق كل الموارد بشكل صحيح تلقائيًا.

استخدم map() للحالات المتجانسة البسيطة، و as_completed() لما عايز تعالج النتائج أول ما توصل.

نادِ دايمًا result() على كل Future (أو استخدم as_completed) عشان تتأكد من التقاط أي أخطاء حصلت.

حدد max_workers بشكل مدروس: عدد الـ CPU cores لـ ProcessPoolExecutor، وعدد أكبر (لكن معقول) لـ ThreadPoolExecutor حسب طبيعة الانتظار.

⚠️ خطأ شائع

نسيان استدعاء result() على Future، مما يخفي أخطاء حقيقية حصلت جوه المهمة.

استخدام ProcessPoolExecutor لمهام I/O-bound — تكلفة إنشاء Processes أكبر من الفايدة هنا.

الخلط بين map() اللي بيحافظ على الترتيب و as_completed() اللي بيرجع النتائج بترتيب الانتهاء الفعلي، مما يؤدي لمعالجة غلط للنتائج.

اعتبارات الأداء وقابلية التوسع

نفس القواعد اللي اتكلمنا عنها في فصلي Threading و Multiprocessing بتنطبق هنا بالكامل، لأن concurrent.futures مبنية فوقهم مباشرة: عدد Workers مناسب لنوع المهمة، تكلفة نقل البيانات بين Processes، وتكلفة Context Switching بين Threads. الفايدة الإضافية هنا هي سهولة التبديل بين الاثنين لتجربة أيهما أنسب فعليًا لمهمتك بأقل تعديل ممكن في الكود.

مثال عملي صغير

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

:def fetch(name, delay)
time.sleep(delay)
"{delay}s بعد خلصت return f"{name}

tasks = {"A": 2, "B": 1, "C": 3}

:with ThreadPoolExecutor(max_workers=3) as executor
futures = {executor.submit(fetch, name, delay): name for name, delay in
{()tasks.items
:for future in as_completed(futures)
بترتيب الانتهاء الفعلي – C بعدين ،A بعدين ،أولًا B هتطبع #    print(future.result())
```

مثال من مشروع Production حقيقي

في خدمة بترجّع توصية (Recommendation) لمستخدم، السيرفر محتاج يجمع بيانات من ٤ مصادر مختلفة بالتوازي: تفضيلات المستخدم، سجل المشتريات، نتيجة نموذج التوصية نفسه، ومخزون المنتجات المتاح. باستخدام ThreadPoolExecutor مع as_completed()، السيرفر بيقدر يبدأ معالجة كل نتيجة أول ما توصل، ولو مصدر واحد بطيء أو فشل، الباقي بيكمل شغله بشكل مستقل من غير ما يعطّل الطلب كله.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل السابع**
>
> اكتب دالة بتحاكي استدعاء API بتوقيت عشوائي (٠٫٥ إلى ٣ ثواني)، وشغّلها على ٥ مدخلات مختلفة باستخدام ThreadPoolExecutor وmap()، وقِس الوقت الكلي.
>
> أعد نفس التمرين باستخدام as_completed() بدل map()، واطبع كل نتيجة فور وصولها مع الوقت اللي استغرقته.
>
> أضف مهمة واحدة من الخمسة بتفشل عمدًا (raise Exception)، وتأكد إنك بتلتقط الخطأ صح من غير ما يعطّل باقي المهام.

تحديات شائعة

- نسيان الفرق الجوهري بين ThreadPoolExecutor وProcessPoolExecutor رغم تشابه الواجهة — الاختيار الغلط بيلغي أي فايدة من الـ Concurrency.
- صعوبة تتبع أخطاء حصلت جوه Futures لم يتم استدعاء result() عليها أبدًا.
- الخلط بين أسلوب concurrent.futures (Blocking، مناسب لكود Synchronous) وأسلوب asyncio (Non-blocking) — الاتنين مختلفين في الفلسفة، ومحاولة دمجهم بشكل خاطئ بيسبب تعقيد غير ضروري.

ملخص الفصل

- concurrent.futures بتوفر واجهة موحدة للتعامل مع Threading وMultiprocessing عن طريق ThreadPoolExecutor وProcessPoolExecutor.
- الـ Future هو كائن بيمثل نتيجة مهمة هتكون متاحة في المستقبل، وresult() هي الطريقة للحصول على النتيجة الفعلية (أو رفع الخطأ لو حصل).
- submit() لتقديم مهام فردية بتحكم كامل، وmap() للحالة المتجانسة البسيطة.
- wait() وas_completed() أدوات للتحكم في كيفية انتظار مجموعة Futures ومعالجة نتائجها.
- الاختيار بين ThreadPoolExecutor وProcessPoolExecutor بيعتمد بالكامل على نوع المهمة (I/O-bound أم CPU-bound)، بالضبط زي القرار في الفصلين الرابع والخامس.

لحد دلوقتي ركزنا على "إزاي ننفذ مهام بالتوازي". الفصل الثامن هياخدنا لسؤال مكمّل ومهم بنفس القدر: إزاي وحدات العمل دي (Threads, Processes, Tasks) تتواصل مع بعض بشكل منظم؟ هنشرح أنماط أساسية زي Queue وPriority Queue، ونمط Producer-Consumer اللي لمحنا له في الفصل الرابع، وWorker Pool، وTask Scheduling — الأنماط اللي بتحوّل مجموعة وحدات عمل منفصلة لنظام متكامل يشتغل مع بعض بتناغم.

الفصل الثامن: التواصل بين وحدات العمل — Communication Between Workers

في الفصول السابقة اتعلمنا إزاي ننفذ مهام بالتوازي (Threading, Multiprocessing, Async)، وإزاي نستخدم واجهة موحدة لإدارتها (concurrent.futures). لكن في أي نظام حقيقي، وحدات العمل دي نادرًا ما بتشتغل لوحدها بمعزل عن بعض — غالبًا محتاجة تتواصل: وحدة بتنتج بيانات ووحدة تانية بتستهلكها، أو مهام محتاجة تتوزع على مجموعة Workers بشكل منظم. الفصل ده هيغطي الأنماط الأساسية اللي بتحوّل مجموعة وحدات عمل منفصلة لنظام متكامل ومنظم.

المشكلة التي يحلها التواصل المنظم بين الوحدات

تخيل نظام فيه وحدة بتقرأ صور من مصدر خارجي، ووحدة تانية بتعالج الصور دي، ووحدة تالتة بتخزن النتائج. لو كل وحدة اشتغلت بمعزل تام عن التانية، هتحتاج آلية واضحة تنسّق بينهم: إمتى الوحدة التانية تعرف إن فيه صورة جديدة جاهزة للمعالجة؟ إزاي نضمن إن مفيش صورة اتعالجت مرتين أو ضاعت من غير معالجة؟ إزاي نوزع الشغل بالتساوي بين عدة Workers بدل ما واحد يشتغل بجنون والباقي قاعدين؟

> 📌 **ملاحظة مهمة**
>
> التواصل غير المنظم بين وحدات العمل (زي الاعتماد على متغيرات مشتركة عشوائية أو Polling يدوي متكرر) بيؤدي غالبًا لكود هش، صعب الصيانة، وعرضة لـ Race Conditions. الأنماط اللي هنشرحها في الفصل ده هي حلول مجرّبة ومُثبتة لهذه المشكلة بالتحديد.

Queue — الأداة الأساسية للتواصل

الـ Queue (طابور) هي هيكل بيانات بيتبع مبدأ FIFO (أول داخل، أول خارج)، ومصممة لتكون آمنة تمامًا للاستخدام من عدة Threads أو Processes في نفس الوقت من غير الحاجة لـ Lock يدوي إضافي — التزامن مبني جواها بالفعل.

```
import queue, threading

()q = queue.Queue

:()def producer
:for i in range(5)
q.put(i)
("{i} :أنتجت"f)print

:()def consumer
:while True
ينتظر تلقائيًا لو الطابور فاضي #          ()item = q.get
:if item is None
break
("{item} :استهلكت"f)print
يخبر الطابور إن المهمة دي خلصت فعليًا #           ()q.task_done

t1 = threading.Thread(target=producer)
t2 = threading.Thread(target=consumer)
()t1.start(); t2.start
```

⚠️ **خطأ شائع**

queue.Queue (من مكتبة queue) مخصصة للتواصل بين Threads. multiprocessing.Queue (اللي شرحناها في الفصل الخامس) مخصصة للتواصل بين Processes. الاتنين بنفس الفلسفة، لكن مش قابلين للتبديل المباشر — استخدم الأداة المناسبة لنوع وحدة العمل اللي بتستخدمها.

Priority Queue

في بعض الأنظمة، مش كل المهام بنفس الأهمية. الـ Priority Queue بتضمن إن العناصر ذات الأولوية الأعلى (رقم أقل عادة) تخرج من الطابور قبل غيرها، بغض النظر عن ترتيب دخولها.

```
import queue

()pq = queue.PriorityQueue

(("مهمة عادية" ,3))pq.put
(("مهمة عاجلة جدًا" ,1))pq.put
(("مهمة متوسطة" ,2))pq.put

:()while not pq.empty
()priority, task = pq.get
print(f"[{priority}] {task}")
النتيجة بترتيب الأولوية: عاجلة جدًا، متوسطة، عادية #
بغض النظر عن ترتيب الإضافة الفعلي #
```

Message Passing

نمط عام أوسع من مجرد Queue: بدل ما وحدات العمل تشارك ذاكرة مباشرة (وتحتاج Locks معقدة لحمايتها)، كل وحدة بتتواصل عن طريق إرسال 'رسائل' واضحة ومحددة لوحدات تانية. الفلسفة دي بتقلل الحاجة للـ Shared State بشكل كبير، وبتخلي كل وحدة عمل معزولة ومستقلة أكتر، وده بيقلل احتمالية Race Conditions بشكل جوهري.

الميزة الرئيسية	الفكرة	النهج
أداء عالي لمشاركة بيانات كبيرة	وحدات العمل بتشارك بيانات مباشرة، محمية بأقفال	Shared Memory + Locks
عزل أفضل، تعقيد أقل، أسهل في التتبع	وحدات العمل بتتواصل برسائل واضحة عبر Queue	Message Passing

🏢 **من واقع الشركات**

Message Passing هو الأساس الفلسفي اللي بُنيت عليه أنظمة كبيرة زي Actor Model (المستخدم في أنظمة زي Erlang/Elixir)، وكمان أساس أنظمة الـ Message Queues في الإنتاج (زي RabbitMQ وKafka)، اللي هتشوف نمطها المبسّط هنا بيتكرر على مستوى أنظمة موزّعة كاملة.

نمط Producer-Consumer

الفكرة الأساسية

لمحنا لنمط ده في الفصل الرابع، ودلوقتي هنشرحه بشكل متكامل. النمط بيقسّم العمل لدورين واضحين: Producer بينتج بيانات أو مهام ويحطها في Queue مشترك، وConsumer بياخد العناصر دي من الـ Queue ويعالجها. الفصل بين الدورين بيسمح لكل واحد فيهم يشتغل بالسرعة المناسبة له، من غير ما يعطّل التاني — الـ Queue بيعمل كـ 'ممتص صدمات' (Buffer) بين السرعتين المختلفتين.

```
import threading, queue, time

task_queue = queue.Queue(maxsize=10)   # حد أقصى لحجم الطابور

def producer():
    for i in range(20):
        task_queue.put(i)              # ينتظر تلقائيًا لو الطابور ممتلئ
        print(f"أنتج: {i}")
    task_queue.put(None)               # إشارة النهاية

def consumer():
    while True:
        item = task_queue.get()
        if item is None:
            task_queue.put(None)       # تمرير الإشارة لـ Consumers تانية لو موجودة
            break
        time.sleep(0.1)                # محاكاة وقت معالجة
        print(f"استهلك: {item}")

threading.Thread(target=producer).start()
threading.Thread(target=consumer).start()
```

✅ أفضل الممارسات

تحديد maxsize لطابور Producer-Consumer مهم جدًا في الإنتاج: بدون حد أقصى، لو الـ Producer أسرع بكتير من الـ Consumer، الطابور ممكن يكبر بلا حدود ويستهلك كل الذاكرة المتاحة (Memory Exhaustion). الحد الأقصى بيخلي put() تنتظر تلقائيًا لو الطابور ممتلئ، وده بيوازن السرعة بين الاتنين (يسمى Backpressure).

Work Queue وWorker Pool

توسيع طبيعي لنمط Producer-Consumer: بدل Consumer واحد، بنشغّل مجموعة Workers (Worker Pool) كلهم بياخدوا مهام من نفس الـ Queue المشترك. أي Worker يبقى فاضي، بياخد المهمة التالية تلقائيًا — ده بيوزع الحمل بالتساوي بدون أي تنسيق يدوي إضافي، لأن الـ Queue نفسه بيدير التوزيع.

```
import threading, queue

work_queue = queue.Queue()
for i in range(20):
    work_queue.put(i)

def worker(worker_id):
    while not work_queue.empty():
        try:
            item = work_queue.get_nowait()
        except queue.Empty:
```

```
break
("{item} بيعالج f"Worker {worker_id})print
()work_queue.task_done

workers = [threading.Thread(target=worker, args=(i,)) for i in range(4)]
()for w in workers: w.start
()for w in workers: w.join
Workers 4 # بياخدوا مهام من نفس الطابور، كل واحد حسب سرعته الخاصة
```

🏢 من واقع الشركات

ميزة Worker Pool المبني على Queue مشترك: التوزيع تلقائي وعادل بطبيعته. لو Worker معين اتأخر في مهمة، الباقي بيكمل ياخد مهام جديدة من غير ما ينتظره — عكس التقسيم اليدوي الثابت (زي تقسيم القائمة لـ ٤ أجزاء متساوية مقدمًا) اللي ممكن يسيب Worker واحد شغال لوحده لو باقي المهام خلصت بدري.

Task Scheduling

في أنظمة أكبر، مش كل المهام بتُنفذ فورًا. Task Scheduling هو تنظيم متى وبأي ترتيب تُنفذ المهام: بعضها لازم يتنفذ بشكل دوري (كل ساعة مثلًا)، بعضها لازم يتنفذ في وقت محدد مستقبلي، وبعضها لازم يعاد تنفيذه لو فشل. المفاهيم الأساسية هنا (Scheduling, Retry, Idempotency) هتتوسع بالتفصيل في الفصل الثاني عشر (Production Patterns)، لكن الأساس اللي هيبنى عليها كله هو نفس فكرة الـ Queue: مهام بتتحط في طابور، وWorkers بتسحب منه وتنفذ.

مقارنة الأنماط الأساسية

متى تستخدمه	الفكرة الأساسية	النمط
معدلات إنتاج واستهلاك مختلفة، محتاجة Buffer بينهم	دور إنتاج منفصل عن دور استهلاك، مربوطين بـ Queue	Producer-Consumer
توزيع حمل كبير على عدة وحدات عمل بالتساوي	عدة Consumers بياخدوا من نفس الـ Queue	Worker Pool
أنظمة فيها مهام عاجلة يجب تنفيذها أولًا	ترتيب المهام حسب الأهمية لا الترتيب الزمني	Priority Queue
تقليل التعقيد والحاجة لـ Locking المعقد	تواصل عبر رسائل بدل مشاركة ذاكرة مباشرة	Message Passing

كيف تُستخدم داخل الشركات الكبرى

الأنماط دي هي نفسها الأساس اللي بُنيت عليه أنظمة الرسائل الموزّعة الكبرى زي RabbitMQ وApache Kafka وAmazon SQS، اللي بتُستخدم على نطاق واسع في الشركات التقنية الكبرى للتواصل بين الخدمات المختلفة (Microservices). الفرق الأساسي إن الأنظمة دي بتوفر نفس مبدأ Producer-Consumer وWorker Pool، لكن على مستوى موزّع بين عدة أجهزة (مش بس Threads/Processes على جهاز واحد)، مع ضمانات إضافية زي عدم فقدان الرسائل حتى لو حصل عطل في أحد الأجهزة.

أين تقع داخل مشروع Machine Learning حقيقي

- خط معالجة بيانات (Data Pipeline): مرحلة قراءة البيانات (Producer) منفصلة عن مرحلة المعالجة (Consumer)، مربوطين بـ Queue.
- خدمة Inference بتستقبل طلبات وتحطها في Work Queue، وعدة Workers بيعالجوها بالتوازي حسب توفرهم.
- نظام تدريب نماذج بيه مهام بأولويات مختلفة (تجربة عاجلة لفريق البحث مقابل تدريب دوري روتيني)، مناسب لـ Priority Queue.
- جمع Logs أو Metrics من عدة مصادر عن طريق Message Passing بدل مشاركة ملف واحد من عدة كتابات متزامنة.

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

حدد دايمًا maxsize لأي Queue في بيئة الإنتاج لتجنب استهلاك ذاكرة غير محدود (Backpressure).

استخدم إشارة واضحة لنهاية العمل (زي None) بدل الاعتماد على تخمين توقيت انتهاء الـ Producer.

فضّل Message Passing (Queue) على المشاركة المباشرة للذاكرة كل ما كان ممكن — تعقيد أقل وأخطاء أقل.

استخدم task_done() وjoin() على الـ Queue عشان تضمن إن كل المهام اتعالجت فعليًا قبل ما تنهي البرنامج.

⚠️ خطأ شائع

Queue بدون maxsize في نظام فيه Producer أسرع من Consumer — ممكن يؤدي لاستهلاك ذاكرة غير محدود وCrash فعلي للنظام.

استخدام queue.Queue مع Processes بدل multiprocessing.Queue — الأداتين مش قابلتين للتبديل، وده هيسبب أخطاء أو سلوك غير متوقع.

نسيان معالجة حالة الطابور الفاضي في Worker Pool (get_nowait()) بدون try/except مما يؤدي لأخطاء غير متوقعة.

الاعتماد على متغيرات مشتركة عادية للتواصل بين Workers بدل Queue مخصصة وآمنة — إعادة اختراع عجلة أقل أمانًا.

اعتبارات الأداء، الأمان، وقابلية التوسع

الأداء

حجم الـ Queue وعدد الـ Workers لازم يتوازنوا مع معدل الإنتاج الفعلي. Queue كبير جدًا مع عدد Workers قليل بيسبب تأخير كبير في معالجة المهام القديمة، وعدد Workers أكبر من اللازم بيسبب تنافس غير مفيد على نفس الـ Queue (Lock Contention داخلي).

الأمان

استخدام Queue آمن (Thread-safe أو Process-safe حسب النوع) بيمنع فئة كاملة من مشاكل الـ Race Conditions اللي ممكن تحصل لو حاولت تبني نفس السلوك يدويًا بمتغيرات مشتركة عادية.

قابلية التوسع

نمط Worker Pool المبني على Queue بيتوسع بسهولة: عايز تعالج حمل أكبر؟ زوّد عدد الـ Workers. النمط ده بيتوسع بشكل طبيعي جدًا من Threads داخل عملية واحدة، إلى Processes على نفس الجهاز، إلى أنظمة رسائل موزّعة على عدة أجهزة (زي Celery اللي هيتشرح في الفصل الثاني عشر) — نفس الفلسفة، مستويات مختلفة من التوزيع.

مثال عملي صغير

نظام بسيط بـ Producer واحد و٣ Workers بيعالجوا المهام بالتوازي، مع قياس توزيع الحمل بينهم:

```
import threading, queue, time, random

()task_queue = queue.Queue
[] = results
()results_lock = threading.Lock

:()def producer
:for i in range(15)
task_queue.put(i)
:for _ in range(3)
workers 3 إشارة نهاية لكل الـ #   task_queue.put(None)

:def worker(worker_id)
count = 0
:while True
()item = task_queue.get
:if item is None
break
وقت معالجة متفاوت #   time.sleep(random.uniform(0.05, 0.2))
count += 1
:with results_lock
results.append((worker_id, count))

()threading.Thread(target=producer).start
workers = [threading.Thread(target=worker, args=(i,)) for i in range(3)]
()for w in workers: w.start
()for w in workers: w.join
هياخد عدد مهام مختلف حسب سرعته الفعلية worker كل #   print(results)
```

مثال من مشروع Production حقيقي

في نظام معالجة صور بالجملة قبل التدريب، مرحلة قراءة الصور من التخزين السحابي (I/O-bound) منفصلة تمامًا عن مرحلة تغيير حجمها ومعالجتها (CPU-bound). Producer (أو مجموعة Producers) بتقرأ الصور وتحطها في Queue مشترك، وWorker Pool (مبني على Process Pool لأن المعالجة CPU-bound) بياخد الصور من الطابور ويعالجها. الفصل ده بيسمح لكل مرحلة تتوسع بشكل مستقل: تقدر تزود عدد Producers لو القراءة بطيئة، أو تزود عدد Workers لو المعالجة هي عنق الزجاجة، من غير ما تأثر على المرحلة التانية.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل الثامن**
>
> طبّق نظام Work Queue بـ Producer واحد بيولّد ٣٠ مهمة، و٣ Workers بياخدوا منها، واطبع عدد المهام اللي عالجها كل Worker في النهاية.
>
> أضف PriorityQueue بدل Queue العادي، وخلي بعض المهام 'عاجلة' وبعضها 'عادية'، وتأكد إن العاجلة بتتعالج أولًا.

جرّب تشغّل نفس نظام Producer-Consumer بدون تحديد maxsize، وبعدين مع maxsize صغير، ولاحظ الفرق في سلوك الـ Producer لما الطابور يمتلئ.

تحديات شائعة

- تحديد الحجم الأمثل للـ Queue وعدد الـ Workers المناسب بدون قياس فعلي على الحمل الحقيقي.
- التعامل الصحيح مع حالة انتهاء العمل (Graceful Shutdown) لكل الـ Workers بدون فقدان مهام لم تُعالج بعد.
- التفرقة بين استخدام Queue بسيط داخل عملية واحدة، والحاجة الفعلية لنظام رسائل موزّع أكبر (زي Celery أو Kafka) لما الحمل يكبر عن حدود جهاز واحد.

ملخص الفصل

- Queue هي الأداة الأساسية والآمنة للتواصل بين وحدات العمل، سواء Threads أو Processes.
- PriorityQueue بتضمن معالجة المهام الأهم أولًا بغض النظر عن ترتيب وصولها.
- Message Passing نمط بيقلل الحاجة لمشاركة الذاكرة المباشرة، وبيقلل تعقيد وأخطاء التزامن.
- Producer-Consumer وWorker Pool أنماط أساسية بتفصل الإنتاج عن الاستهلاك، وبتوزع الحمل تلقائيًا بعدالة بين عدة Workers.
- نفس المبادئ دي هي أساس أنظمة الرسائل الموزّعة الكبرى (RabbitMQ, Kafka) اللي بتُستخدم في الشركات التقنية على نطاق واسع.

لحد دلوقتي ركزنا على "إزاي ننفذ الكود بالتوازي وننسّق بينه". لكن فيه سؤال أهم من كده لسه ما جاوبناش عليه بشكل منهجي: إزاي أعرف فعليًا إن اللي عملته حسّن الأداء؟ الفصل التاسع هيغطي Performance Optimization: إزاي تقيس (Profile) وتقارن (Benchmark) أداء الكود بشكل صحيح وعلمي، إزاي تحدد عنق الزجاجة الحقيقي في نظامك، وإزاي تختار عدد الـ Workers الأمثل بناءً على أرقام حقيقية مش تخمين.

الفصل التاسع: تحسين الأداء — Performance Optimization

طول الفصول التمنية اللي فاتت، كنا بنقول جمل زي 'قِس الأداء الفعلي قبل ما تقرر' و'استخدم Benchmark بدل التخمين'. الفصل ده هو المكان اللي هنشرح فيه إزاي تعمل الكلام ده فعليًا وبشكل منهجي. من غير قياس حقيقي ومنظم، أي قرار Concurrency أو Parallelism بتاخده هو تخمين، مهما كان مبني على فهم نظري قوي — والتخمين في هندسة الأنظمة مكلف.

المشكلة التي يحلها القياس المنظم

مهندس بيشوف كود بطيء، وبيحس بديهيًا إن المشكلة في مكان معين (مثلًا 'أكيد المشكلة في قاعدة البيانات')، فبيقضي أيام يحسّن الجزء ده، ويكتشف في الآخر إن الأداء ما اتحسنش تقريبًا، لأن عنق الزجاجة الحقيقي (Bottleneck) كان في مكان تاني تمامًا (مثلًا معالجة البيانات في الذاكرة، مش القاعدة). المشكلة إن الحدس البشري عن أداء الأنظمة المعقدة غالبًا غلط، والحل الوحيد الموثوق هو القياس الفعلي.

> ✅ **أفضل الممارسات**
>
> قاعدة ذهبية في هندسة الأداء: 'لا تحسّن ما لم تقِس' (Don't optimize what you haven't measured). أي تحسين أداء بدون قياس مسبق هو مقامرة، مش هندسة.

Profiling — تحليل الأداء التفصيلي

الفكرة الأساسية

الـ Profiling هو عملية تحليل تفصيلية لتنفيذ البرنامج، بتوريك بالظبط كل دالة استغرقت وقت قد إيه، وكام مرة اتنادت، وده بيساعدك تحدد عنق الزجاجة الحقيقي بدقة، مش بالتخمين.

```
import cProfile, pstats

:()def slow_function
total = 0
:for i in range(1_000_000)
total += i ** 2
return total

()profiler = cProfile.Profile
()profiler.enable
()slow_function
()profiler.disable

stats = pstats.Stats(profiler).sort_stats('cumulative')
أعلى 10 دوال استهلاكًا للوقت #   stats.print_stats(10)
```

Profiling في الأنظمة متعددة الوحدات

تحليل الأداء في نظام Concurrent أو Parallel أعقد من برنامج Sequential عادي، لأن الوقت مش بس 'كام ثانية استغرقت الدالة'، لازم كمان تعرف: كام وقت قضته وحدة العمل في الانتظار (Waiting) مقابل التنفيذ الفعلي (Executing)، وهل فيه Threads أو Processes قاعدة فاضية أغلب الوقت.

🏢 من واقع الشركات

أدوات Profiling عادية (زي cProfile) بتحلل Thread واحد بشكل جيد، لكن لتحليل تفاعل عدة Threads/Processes مع بعض، محتاج أدوات متخصصة أكتر (زي py-spy اللي بتوفر رؤية حية للـ Threads الشغالة فعليًا، أو أدوات تتبع Async زي المدمجة في asyncio نفسها). هنتعمق في أدوات تحليل الكود المتزامن تحديدًا في الفصل الثالث عشر.

Benchmarking — القياس المقارن

الفرق بين Profiling وBenchmarking

الـ Profiling بيوريك 'فين الوقت بيروح جوه تنفيذ واحد'، بينما الـ Benchmarking بيقارن بين أسلوبين أو أكتر (زي Threading مقابل Multiprocessing لنفس المهمة) عشان تعرف أيهما أسرع فعليًا في حالتك بالتحديد.

```
import timeit

:()def sequential_version
return [x**2 for x in range(100_000)]

:()def list_comprehension_alt
[] = result
:for x in range(100_000)
result.append(x**2)
return result

t1 = timeit.timeit(sequential_version, number=100)
t2 = timeit.timeit(list_comprehension_alt, number=100)
print(f"List comprehension: {t1:.4f}s")
print(f"Loop + append: {t2:.4f}s")
```

✅ أفضل الممارسات

استخدم timeit بدل قياس time.time() يدوي للمقارنات الدقيقة — timeit بيشغّل الكود عدة مرات ويتعامل مع تأثيرات خارجية (زي عمليات نظام التشغيل الخلفية) بشكل أفضل بكتير من قياس واحد بسيط.

مبادئ Benchmarking الصحيح

22. قِس على بيانات واقعية بحجم مشابه لحجم الإنتاج الفعلي، مش على بيانات تجريبية صغيرة جدًا.
23. كرر القياس عدة مرات وخد المتوسط (أو الـ Median)، لتجنب تأثير ضوضاء عشوائية (زي عملية خلفية من نظام التشغيل أثرت على قياس واحد بالصدفة).
24. قِس على نفس البيئة (نفس الجهاز، نفس الحمل) اللي هيشتغل عليها النظام فعليًا قدر الإمكان.
25. قارن دايمًا مع خط أساس واضح (Baseline) — الحل الحالي أو الأبسط، مش مجرد قياس مطلق بدون سياق.

CPU Utilization

مقياس بيوضح نسبة استغلال المعالج الفعلية أثناء تنفيذ البرنامج. لو عندك ٨ Cores ونظامك بيستخدم نسبة قليلة منهم (زي ١٥٪) رغم إنه شغال بأقصى طاقته، ده مؤشر قوي إن المشكلة مش في المعالجة الفعلية، لكن في الانتظار (I/O-bound)، وإن الحل المناسب هو تحسين الـ Concurrency (Threading/Async) بدل زيادة Parallelism.

الاتجاه المناسب للحل	التفسير المحتمل	نسبة استخدام المعالج
تحسين Concurrency (Threading/Async)	النظام بيقضي معظم وقته في انتظار I/O	منخفضة جدًا (< ٢٠٪)
قد يحتاج دمج Async مع Process Pool	مزيج من الحساب والانتظار	متوسطة (٤٠-٧٠٪)
تحسين الخوارزمية أو زيادة Parallelism الفعلي	المعالج هو عنق الزجاجة فعليًا	عالية جدًا (> ٩٠٪) على كل الـ Cores

Memory Usage

كل Thread أو Process إضافي بيستهلك ذاكرة (Stack للـ Threads، مساحة كاملة للـ Processes). في الأنظمة اللي بتشغّل عدد كبير من وحدات العمل، استهلاك الذاكرة ممكن يبقى عنق الزجاجة الحقيقي قبل حتى ما توصل لحدود المعالج. مراقبة استهلاك الذاكرة أثناء زيادة عدد الـ Workers مهمة جدًا لتحديد الحد الأقصى العملي قبل ما النظام يبدأ يعاني من Swapping (استخدام القرص الصلب كذاكرة احتياطية، وده بيبطئ الأداء بشكل كارثي).

> 📌 **ملاحظة مهمة**
>
> أدوات زي memory_profiler أو tracemalloc المدمجة في Python بتساعدك تحدد بالظبط أي جزء من الكود بيستهلك ذاكرة زيادة عن اللازم، خصوصًا في الأنظمة اللي بتشغّل عدد كبير من الـ Processes أو Threads بالتوازي.

Context Switching Cost

رجعنا لمفهوم اتكلمنا عنه في الفصل التاني، لكن دلوقتي من زاوية القياس: كل عملية Context Switching (سواء بين Threads أو Processes) بتاخد وقت معالج حقيقي، مش صفر. لو عدد وحدات العمل أكبر بكتير من عدد الـ Cores المتاحة، النظام بيقضي وقت متزايد في التبديل بدل التنفيذ الفعلي، وده بيظهر بوضوح في نتائج الـ Benchmarking كتراجع في الأداء رغم زيادة عدد الـ Workers.

Serialization Cost

لمحنا للمفهوم ده في فصل الـ Multiprocessing: أي بيانات بتتنقل بين Processes (عن طريق Queue, Pipe, أو نتيجة مهمة في Process Pool) لازم تتحول لصيغة قابلة للنقل (Pickling في Python) ثم تُعاد لصيغتها الأصلية في الطرف التاني. العملية دي مش مجانية — للبيانات الكبيرة (زي مصفوفات ضخمة أو صور)، تكلفة الـ Serialization ممكن تلغي جزء كبير من فايدة الـ Parallelism نفسها.

```
import time, pickle
import numpy as np

big_array = np.random.rand(10_000_000)

()start = time.time
serialized = pickle.dumps(big_array)
deserialized = pickle.loads(serialized)
("{time.time()-start:.4f}s :لمصفوفة كبيرة Serialization تكلفة"f)print
# لو المهمة نفسها بتاخد وقت أقل من ده، فتكلفة نقل البيانات
```

```
# بتلغي فايدة استخدام Multiprocessing من الأساس
```

⚠️ خطأ شائع

لو تكلفة نقل البيانات بين Processes أكبر من (أو قريبة من) الوقت اللي هتوفره من المعالجة المتوازية نفسها، Multiprocessing مش هيفيدك — أو ممكن كمان يبطئ الكود عن التنفيذ الـ Sequential العادي.

Choosing Workers — اختيار العدد الأمثل

مفيش رقم سحري ثابت لعدد الـ Workers المناسب لكل الحالات؛ العدد الأمثل بيعتمد على نوع المهمة:

نوع المهمة	نقطة بداية معقولة	ملاحظة
CPU-bound (Multiprocessing)	عدد الـ CPU cores الفعلية	الزيادة عن كده غالبًا بتضر الأداء
I/O-bound (Threading)	أكبر بكتير من عدد الـ Cores (عشرات)	العدد الأمثل بيعتمد على زمن الانتظار الفعلي لكل مهمة
I/O-bound (Async)	لا يوجد مفهوم 'Workers' تقليدي	محدود بموارد النظام (Sockets, Memory) مش بعدد Threads
مهمة مختلطة (CPU + I/O)	يحتاج قياس فعلي ودمج Async + Process Pool	لا توجد قاعدة ثابتة، لازم Benchmark

✅ أفضل الممارسات

الطريقة العملية الصح لتحديد العدد الأمثل: ابدأ برقم منطقي (زي عدد الـ Cores للـ CPU-bound)، وقِس الأداء الفعلي بزيادة العدد تدريجيًا (٢، ٤، ٨، ١٦..)، وارصد النقطة اللي التحسن فيها بيقف أو يبدأ يتراجع. الرقم ده هو العدد الأمثل لبيئتك بالتحديد، مش رقم عام تقرأه في مقال.

Performance Tuning — منهجية شاملة

26. حدد هدف الأداء بوضوح (زي 'وقت الاستجابة يجب أن يكون أقل من ٢٠٠ ميلي ثانية').
27. قِس الأداء الحالي فعليًا (Baseline) قبل أي تعديل.
28. استخدم Profiling لتحديد عنق الزجاجة الحقيقي، مش المُفترَض.
29. طبّق تحسين واحد في كل مرة، وقِس أثره بشكل منعزل — تعديل عدة حاجات مع بعض بيصعّب معرفة أيهم أثّر فعليًا.
30. كرر العملية لحد ما توصل للهدف، أو توصل لنقطة تناقص العائد (كل تحسين إضافي بياخد جهد أكبر بكتير من الفايدة المكتسبة).

كيف تُستخدم هذه المبادئ داخل الشركات الكبرى

فرق الهندسة في الشركات الكبرى بتعتمد بشكل كبير على المراقبة المستمرة (Continuous Monitoring) والقياس الآلي كجزء أساسي من دورة تطوير أي نظام، مش كخطوة استثنائية بتحصل مرة واحدة. أي تغيير كبير في نظام Production بيتقاس أثره على الأداء قبل وبعد النشر، وغالبًا بيتم نشره تدريجيًا (Canary Deployment) عشان يتقاس تأثيره الفعلي على جزء صغير من الحمل الحقيقي قبل التعميم الكامل.

أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي

- تحديد عدد الـ Workers الأمثل لخدمة Model Serving بناءً على قياس فعلي لزمن الاستجابة تحت أحمال مختلفة.
- قياس تكلفة نقل البيانات (Features) بين Processes في خط معالجة بيانات كبير، وتحديد هل Multiprocessing يستحق التكلفة فعلًا.
- تحليل استهلاك الذاكرة لنموذج مُحمَّل في الذاكرة عبر عدة Workers، لتحديد الحد الأقصى العملي لعدد الـ Workers المتاحة.
- قياس نسبة استخدام المعالج والذاكرة في خدمة Inference لتحديد هل عنق الزجاجة في الشبكة، قاعدة البيانات، أم الحساب نفسه.

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

قِس دايمًا قبل التحسين وبعده، ووثّق النتائج (خط أساس واضح، ونتيجة كل تعديل).

استخدم أدوات Profiling حقيقية بدل التخمين أو "الحدس الهندسي" غير المدعوم بأرقام.

اختبر على بيانات وأحمال قريبة من الواقع الفعلي، مش على أمثلة صغيرة جدًا مضللة.

احسب تكلفة الـ Overhead (إنشاء Workers، Serialization، Context Switching) كجزء من أي قرار Concurrency/Parallelism.

⚠️ خطأ شائع

تحسين جزء من الكود بناءً على حدس شخصي بدون Profiling فعلي، والاكتشاف لاحقًا إنه مش عنق الزجاجة الحقيقي.

قياس الأداء على بيانات صغيرة جدًا غير ممثلة للحمل الحقيقي في الإنتاج.

تجاهل تكلفة الـ Overhead (خصوصًا Serialization في Multiprocessing) عند مقارنة الأساليب المختلفة.

زيادة عدد الـ Workers باستمرار على أمل تحسن الأداء، من غير قياس النقطة اللي التحسن بيتوقف عندها فعليًا.

مثال عملي صغير

Benchmark منهجي بسيط لتحديد العدد الأمثل من الـ Workers لمهمة معينة:

```
import time
from concurrent.futures import ProcessPoolExecutor

:def cpu_task(n)
x = 0
:for _ in range(n)
x += 1
return x

:def benchmark(worker_counts, n_tasks, task_size)
:for workers in worker_counts
()start = time.time
:with ProcessPoolExecutor(max_workers=workers) as executor
list(executor.map(cpu_task, [task_size] * n_tasks))
```

```
elapsed = time.time() - start
print(f"Workers={workers}: {elapsed:.2f}s")

:'__if __name__ == '__main
benchmark(worker_counts=[1, 2, 4, 8, 16], n_tasks=16, task_size=5_000_000)
لاحظ عند أي عدد Workers التحسن بيبدأ يقف أو يتراجع #
```

مثال من مشروع Production حقيقي

فريق هندسي بيلاحظ إن خدمة Model Serving بتاعته بتاخد وقت استجابة أطول من المتوقع تحت حمل عالي. بدل تخمين السبب، الفريق بيعمل Profiling كامل، وبيكتشف إن السبب الحقيقي مش في النموذج نفسه (اللي سريع فعلًا)، لكن في عملية تحويل البيانات (Serialization) بين الـ Worker Process اللي بيستقبل الطلب والـ Process اللي بينفذ الـ Inference — البيانات كانت بتتبعت بصيغة غير محسّنة. بعد تحسين طريقة نقل البيانات (استخدام صيغة أخف وأسرع في التحويل)، وقت الاستجابة قل بشكل كبير من غير أي تغيير في النموذج نفسه — مثال حي على أهمية القياس قبل الحكم على مكان المشكلة.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل التاسع**
>
> استخدم cProfile على دالة بتقوم بعدة عمليات مختلفة (بعضها سريع وبعضها بطيء عمدًا)، وحدد من نتيجة الـ Profiling أي جزء هو الأبطأ فعليًا.
>
> اعمل Benchmark لنفس المهمة الحسابية بعدد Workers مختلف (١, ٢, ٤, ٨)، وارسم جدول بالنتائج، وحدد العدد الأمثل لجهازك.
>
> قِس تكلفة Serialization لنقل قائمة كبيرة (مليون عنصر) بين Processes، وقارنها بوقت معالجة بسيطة لنفس القائمة، واستنتج هل الـ Multiprocessing يستحق العناء هنا.

تحديات شائعة

- صعوبة عزل تأثير تعديل واحد بالضبط لما بتحصل عدة تغييرات في نفس الوقت.
- نتائج Benchmark متضاربة بسبب عوامل خارجية (حمل الجهاز، عمليات خلفية) لو محدش كرر القياس عدة مرات.
- الخلط بين تحسين أداء جزء صغير من الكود (Micro-optimization) وتحسين الأداء الكلي للنظام — أحيانًا الجزء المُحسَّن مش مؤثر أصلًا على التجربة الكلية.

ملخص الفصل

- لا تحسّن الأداء بدون قياس أولًا — الحدس البشري عن مكان المشكلة غالبًا غلط.
- Profiling بيوريك فين الوقت بيروح فعليًا، وBenchmarking بيقارن بين أساليب مختلفة بشكل موضوعي.
- CPU Utilization وMemory Usage مؤشرات أساسية لتحديد نوع عنق الزجاجة (حساب أم انتظار أم ذاكرة).
- تكلفة Context Switching وSerialization حقيقية ولازم تتحسب كجزء من أي قرار Concurrency/Parallelism.
- لا يوجد رقم سحري لعدد الـ Workers الأمثل — لازم يُحدَّد بالقياس الفعلي على بيئتك وحملك بالتحديد.

لحد دلوقتي غطينا كل أدوات الـ Concurrency والـ Parallelism الأساسية، وكيفية قياس أدائها بشكل صحيح. الفصل العاشر هياخد كل ده ويطبّقه عمليًا في سياق واحد محدد ومهم جدًا لمهندس الـ ML: بناء API حقيقي بـ FastAPI. هنشرح إزاي تدمج Async Routes مع Sync Routes، إزاي تدير Background Tasks، إزاي تختار عدد Worker Processes الصحيح (Uvicorn وGunicorn)، وإزاي تتعامل مع قواعد بيانات وRedis بشكل Async، وWebSockets، وStreaming — كل ده في إطار خدمة إنتاج حقيقية.

الفصل العاشر: Concurrency في FastAPI

لحد دلوقتي غطينا كل أداة على حدة بشكل منفصل ونظري. الفصل ده مختلف: هنطبّق كل حاجة اتعلمناها في سياق عملي واحد ومحدد جدًا — بناء API حقيقي بـ FastAPI، وهو من أكتر الأطر استخدامًا لخدمة نماذج الـ Machine Learning في الإنتاج. هنشوف إزاي القرارات اللي اتكلمنا عنها نظريًا (Async مقابل Threading مقابل Multiprocessing) بتترجم لقرارات تصميم فعلية في كود API حقيقي.

لماذا FastAPI مبني على أساس Async

FastAPI مبني فوق Starlette وuvicorn، واللي بدورهم مبنيين على asyncio (اللي شرحناه بالتفصيل في الفصل السادس). الاختيار ده مش عشوائي: خدمات الـ API الحديثة بتحتاج تخدم آلاف الاتصالات المتزامنة، وزي ما شرحنا، Async هو الأنسب لعدد ضخم من الاتصالات I/O-bound بموارد أقل من Threading التقليدي.

Async Routes مقابل Sync Routes

Async Routes

لما تعرّف مسار (Route) بـ async def، FastAPI بيشغّله مباشرة جوه الـ Event Loop الرئيسي. ده الخيار الأمثل لو كل العمليات جوه المسار async فعليًا (زي استعلامات قاعدة بيانات async، أو استدعاءات API خارجية عن طريق مكتبة async).

```
from fastapi import FastAPI
import asyncio

()app = FastAPI

app.get("/predict-async")@
:async def predict_async(features: list[float])
حقيقية I/O async عملية #   result = await run_model_async(features)
return {"prediction": result}
```

Sync Routes

لو عرّفت المسار بـ def عادية (بدون async)، FastAPI بيلاحظ ده تلقائيًا، وبيشغّل الدالة في Thread Pool منفصل (بعيدًا عن الـ Event Loop الرئيسي) بدل ما ينفذها مباشرة عليه. ده مهم جدًا: لو كان عندك كود Blocking (زي مكتبة Synchronous بحتة) وشغّلته جوه async def مباشرة، هيوقف الـ Event Loop بالكامل زي ما شرحنا في الفصل السادس. لكن لو عرّفته كـ def عادية، FastAPI بيحميك من المشكلة دي تلقائيًا.

```
app.get("/predict-sync")@
:def predict_sync(features: list[float])
بطيء Synchronous كود #   result = run_model_blocking(features)
return {"prediction": result}
منفصل تلقائيًا، Thread Pool بيشغّل الدالة دي في FastAPI #
Event Loop فمش هتوقف باقي الطلبات المتزامنة على الـ #
```

المعيار	async def Routes	def Routes (Sync)
مكان التنفيذ	الـ Event Loop الرئيسي مباشرة	Thread Pool منفصل تلقائيًا

كود Synchronous أو مكتبات لا تدعم async	عمليات I/O مكتوبة بمكتبات async	الاستخدام الأمثل
أقل خطورة، لكن أقل كفاءة لعدد ضخم من الطلبات	كود Blocking داخلها يوقف كل النظام	الخطر عند سوء الاستخدام

⚠️ خطأ شائع

الخطأ الأخطر في FastAPI: استخدام مكتبة Synchronous (زي requests العادية بدل aiohttp) جوه async def. هتشتغل الدالة، لكنها هتوقف الـ Event Loop بالكامل أثناء انتظار الرد، وهتبطئ كل الطلبات المتزامنة التانية على السيرفر، مش بس طلبك أنت.

Background Tasks

أحيانًا محتاج تنفذ عملية بعد ما ترجع الرد للمستخدم فورًا (زي إرسال إيميل تأكيد، أو تسجيل Log تفصيلي)، من غير ما تخلي المستخدم ينتظرها. BackgroundTasks في FastAPI بتوفر الحل ده مباشرة.

```
from fastapi import FastAPI, BackgroundTasks

()app = FastAPI

:def log_prediction(features, result)
(كتابة في قاعدة بيانات تحليلية مثلًا) عملية بتاخد وقت #
save_to_analytics_db(features, result)

app.post("/predict")@
:async def predict(features: list[float], background_tasks: BackgroundTasks)
result = await run_model_async(features)
background_tasks.add_task(log_prediction, features, result)
والتسجيل بيحصل بعد كده ،الرد بيرجع فورًا #   return {"prediction": result}
```

✅ أفضل الممارسات

BackgroundTasks في FastAPI مناسبة للمهام الخفيفة والقصيرة نسبيًا. للمهام الثقيلة أو الطويلة (زي معالجة فيديو أو تدريب نموذج)، الحل الأنسب هو نظام Task Queue متخصص زي Celery (هيتشرح في الفصل الثاني عشر)، لأن BackgroundTasks بتشتغل جوه نفس الـ Process، وممكن تأثر على أداء السيرفر لو كانت ثقيلة جدًا.

Gunicorn و Worker Processes: Uvicorn

زي ما شرحنا في الفصل التاني، الـ Event Loop بتاع asyncio بيشتغل على Thread واحد بس، وبالتالي على Core واحد بس. عشان تستغل باقي الـ Cores على السيرفر، لازم تشغّل عدة نسخ (Workers) من التطبيق، كل واحد Process منفصل بالكامل.

Uvicorn

هو سيرفر الـ ASGI (Asynchronous Server Gateway Interface) اللي FastAPI بتشتغل عليه فعليًا. Uvicorn بيقدر يشغّل عدة Workers بنفسه مباشرة:

```
uvicorn main:app --workers 4
خاص بيه Event Loop وبذاكرته الخاصة، منفصل تمامًا Process هنا worker كل #
```

Gunicorn مع Uvicorn Workers

في بيئات الإنتاج، شائع جدًا استخدام Gunicorn كمدير (Process Manager) Processes فوق Uvicorn:
Gunicorn بيدير عدد الـ Workers، بيعيد تشغيل أي Worker يتعطل تلقائيًا، ويوفر مرونة أعلى في الإدارة والمراقبة مقارنة بتشغيل Uvicorn وحده.

```
gunicorn main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker
Gunicorn بيدير 4 Uvicorn workers، كل واحد Process مستقل #
لو worker واحد اتعطل Gunicorn، بيعيد تشغيله تلقائيًا (Crash) #
```

من واقع الشركات

عدد الـ Workers المناسب غالبًا بيبدأ بقاعدة (٢ × عدد CPU cores) + 1 كنقطة انطلاق شائعة، لكن زي ما شرحنا في الفصل التاسع، الرقم الأمثل الحقيقي لازم يتحدد بالقياس الفعلي على حملك وبيئتك، مش بقاعدة عامة ثابتة.

Async Database وAsync Redis

لو السيرفر بيستخدم مكتبة قاعدة بيانات Synchronous عادية (زي psycopg2 التقليدية) جوه async def، هيحصل نفس مشكلة الـ Blocking اللي شرحناها. الحل: استخدام مكتبات مصممة أصلًا لـ async، زي asyncpg أو SQLAlchemy بوضع الـ Async، وredis.asyncio للتعامل مع Redis بشكل غير متزامن.

```
import asyncpg

app.get("/user/{user_id}")@
:async def get_user(user_id: int)
conn = await asyncpg.connect(DATABASE_URL)
:try
row = await conn.fetchrow("SELECT * FROM users WHERE id = $1", user_id)
return dict(row)
:finally
()await conn.close
Event Loop الاتصال والاستعلام بالكامل، async لا يوقف الـ #
```

أفضل الممارسات

في الإنتاج، استخدم Connection Pool (زي asyncpg.create_pool()) بدل فتح اتصال جديد بقاعدة البيانات لكل طلب — فتح اتصال جديد كل مرة مكلف، والـ Pool بيعيد استخدام الاتصالات الموجودة بكفاءة، بنفس فلسفة استخدام Semaphore لتحديد عدد الاتصالات المتزامنة اللي شرحناها في الفصل الرابع.

Streaming وFile Uploads

رفع الملفات

FastAPI بيدعم استقبال ملفات كبيرة بشكل Async بحيث ميحجزش كل الملف في الذاكرة دفعة واحدة، وده مهم جدًا لملفات كبيرة (زي صور أو بيانات ضخمة لمعالجة النموذج).

```
from fastapi import UploadFile
```

```
app.post("/upload-image")@
:async def upload_image(file: UploadFile)
Event Loop الـ توقف لا ،async قراءة #    ()contents = await file.read
result = await process_image_async(contents)
return {"result": result}
```

Streaming Responses

بدل ما تنتظر النتيجة كاملة قبل ما ترجعها للمستخدم، Streaming بيسمح لك ترجع البيانات تدريجيًا أول بأول — مفيد جدًا لنماذج توليدية (زي نماذج اللغة) اللي بتنتج النتيجة كلمة بكلمة، وده بيحسّن تجربة المستخدم بشكل كبير (الإحساس بالاستجابة السريعة) حتى لو الوقت الكلي نفسه.

```
from fastapi.responses import StreamingResponse

:()async def generate_tokens
:()async for token in model_stream_generator
yield token

app.get("/generate")@
:()async def generate
return StreamingResponse(generate_tokens(), media_type="text/plain")
Async Generators يستخدم # اللي شرحناها في الفصل السادس مباشرة
```

WebSockets

للتواصل ثنائي الاتجاه المستمر بين السيرفر والعميل (زي تحديثات حية لنتائج نموذج، أو محادثة تفاعلية)، FastAPI بيوفر دعم WebSockets مبني بالكامل على نفس أساس الـ Async.

```
from fastapi import WebSocket

app.websocket("/ws/predict")@
:async def websocket_predict(websocket: WebSocket)
()await websocket.accept
:try
:while True
()data = await websocket.receive_json
result = await run_model_async(data["features"])
await websocket.send_json({"prediction": result})
:except Exception
()await websocket.close
```

الهندسة المتكاملة: كيف تُستخدم داخل الشركات الكبرى

النمط الأكتر نضجًا وشيوعًا في خدمات Model Serving الحقيقية بيجمع كل الأدوات اللي اتعلمناها في المذكرة دي مع بعض:

31. طبقة Gunicorn بتدير عدة Uvicorn Workers (Processes منفصلة) عشان تستغل كل الـ CPU cores المتاحة على السيرفر (الفصل الثاني والخامس).
32. جوه كل Worker، async def Routes بتستقبل الطلبات وتتعامل مع I/O (قاعدة بيانات، APIs، Cache خارجية) بكفاءة عالية على Event Loop واحد (الفصل السادس).

33. أي حسابات ثقيلة CPU-bound (زي Inference لنموذج كبير) بتتحول لـ Process Pool منفصل (عن طريق ProcessPoolExecutor، الفصل السابع) عشان متعطلش الـ Event Loop.
34. طلبات متعددة لمصادر بيانات مختلفة بتتنفذ بالتوازي باستخدام asyncio.gather (الفصل السادس).
35. مهام خلفية خفيفة (Logging, Analytics) بتتنفذ عن طريق BackgroundTasks، والمهام الثقيلة بتتحول لنظام Task Queue متخصص (الفصل الثاني عشر).

> 🏢 **من واقع الشركات**
>
> الهندسة دي (Multiprocessing على مستوى Workers + Async I/O جوه كل Worker + Process Pool للحسابات الثقيلة) هي النمط الأشمل اللي بيجمع كل أدوات المذكرة دي مع بعض في نظام إنتاج حقيقي واحد.

أين يقع هذا الفصل داخل مشروع Machine Learning حقيقي

- خدمة Model Serving الأساسية: استقبال طلبات Inference وإرجاع النتائج بأقل زمن استجابة ممكن تحت حمل عالٍ.
- واجهة برمجية لتقديم بيانات Training أو Monitoring لأنظمة تانية داخل الشركة.
- خدمات مساعدة (Feature Store APIs) بتوفر بيانات جاهزة للنماذج بشكل سريع وموثوق.
- خدمات Streaming لنماذج توليدية بتحتاج ترجع النتائج تدريجيًا للمستخدم.

أفضل الممارسات والأخطاء الشائعة

> ✅ **أفضل الممارسات**
>
> استخدم async def بس لو كل العمليات جوه المسار فعليًا async — وإلا خليها def عادية وخلي FastAPI يديرها في Thread Pool.
>
> استخدم مكتبات async مخصصة (aiohttp, asyncpg, redis.asyncio) داخل مسارات async def، مش النسخ الـ Synchronous التقليدية.
>
> استخدم Connection Pools لقواعد البيانات والـ Cache بدل فتح اتصال جديد لكل طلب.
>
> حدد عدد Workers بناءً على قياس فعلي (الفصل التاسع)، وابدأ بقاعدة (٢ × Cores) + 1 كنقطة انطلاق فقط.

> ⚠️ **خطأ شائع**
>
> استدعاء مكتبة Synchronous بطيئة (زي requests) جوه async def — بيوقف الـ Event Loop بالكامل ويبطئ كل الطلبات المتزامنة.
>
> استخدام BackgroundTasks لمهام ثقيلة جدًا أو طويلة — بتأثر على أداء الـ Worker نفسه؛ استخدم Task Queue متخصص بدلها.
>
> فتح اتصال قاعدة بيانات جديد لكل طلب بدل استخدام Connection Pool — تكلفة إضافية غير ضرورية على كل طلب.
>
> الخلط بين عدد Uvicorn/Gunicorn Workers (Processes) وعدد الـ Threads داخل كل Worker — الاتنين مفاهيم مختلفة وقرارات منفصلة.

اعتبارات الأداء، الأمان، وقابلية التوسع

الأداء

الهندسة الصحيحة (Async I/O + Process Pool للحسابات) بتوفر أفضل استغلال ممكن للموارد: عدد ضخم من الاتصالات المتزامنة بأقل ذاكرة، ومعالجة فعلية متوازية للحسابات الثقيلة. أي خطأ في الفصل بين الاتنين (زي كود Blocking جوه async) بيدمر الفايدة دي بالكامل.

الأمان

العزل بين Worker Processes (زي ما شرحنا في الفصل التاني) بيضمن إن تعطل Worker واحد (بسبب خطأ غير متوقع) ما يوقفش الخدمة بالكامل — Gunicorn هيعيد تشغيله تلقائيًا بينما باقي الـ Workers مستمرين في خدمة الطلبات.

قابلية التوسع

النظام المصمم بالشكل ده بيتوسع بسهولة على مستويين: أفقيًا عن طريق زيادة عدد Workers على نفس الجهاز (لحد حدود الـ Cores المتاحة)، وأفقيًا أكبر عن طريق تشغيل عدة نسخ من الخدمة كاملة على أجهزة مختلفة خلف Load Balancer، وهو النمط القياسي في أنظمة الإنتاج الكبرى.

مثال عملي صغير

خدمة FastAPI بسيطة بتجمع بين async Route لجلب بيانات، وProcess Pool لحساب ثقيل، مع Background Task للتسجيل:

```
from fastapi import FastAPI, BackgroundTasks
from concurrent.futures import ProcessPoolExecutor
import asyncio

()app = FastAPI
process_pool = ProcessPoolExecutor(max_workers=4)

:def heavy_computation(data)
عملية CPU-bound ثقيلة #
return sum(x**2 for x in data)

:def log_request(data, result)
("{result} :النتيجة ،عنصر {len(data)} لـ حساب طُلب"f)print

app.post("/compute")@
:async def compute(data: list[int], background_tasks: BackgroundTasks)
()loop = asyncio.get_event_loop
Event Loop الـ توقف بدون ،منفصل Process Pool في الثقيل الحساب تشغيل #
result = await loop.run_in_executor(process_pool, heavy_computation, data)
background_tasks.add_task(log_request, data, result)
return {"result": result}
```

مثال من مشروع Production حقيقي

خدمة Model Serving لنموذج تصنيف صور بتستقبل آلاف الطلبات في الدقيقة. الهندسة النهائية: Gunicorn بيدير ٨ Uvicorn Workers (بعدد الـ CPU cores على السيرفر). كل Worker بيستقبل الطلب، بيجيب بيانات إضافية من Redis (async)، وبيبعت الصورة لـ Process Pool منفصل مخصص لتنفيذ الـ Inference الفعلي (لأنه CPU-bound وثقيل)، وأثناء انتظار النتيجة، الـ Worker بيقدر يخدم طلبات تانية على نفس الـ Event Loop. النتيجة: زمن استجابة منخفض حتى تحت حمل عالٍ، واستغلال كامل لكل الـ Cores المتاحة على السيرفر.

مشروع صغير لتطبيق ما تم شرحه

> 📌 **ملاحظة مهمة — مشروع الفصل العاشر**
>
> ابنِ خدمة FastAPI بسيطة فيها مسار async (بيستخدم asyncio.sleep لمحاكاة I/O) ومسار sync (بيستخدم time.sleep)، وراقب الفرق في سلوك الطلبات المتزامنة بين الاتنين.
>
> أضف Background Task بسيطة بتسجل كل طلب في ملف نصي، وتأكد إن الرد بيرجع للمستخدم قبل ما التسجيل يخلص.
>
> شغّل نفس الخدمة بعدد Workers مختلف (١ مقابل ٤) باستخدام uvicorn --workers، وقِس الفرق في عدد الطلبات اللي الخدمة تقدر تخدمها في الثانية.

تحديات شائعة

- اكتشاف متأخر لكود Blocking مخفي جوه مكتبة خارجية بتُستخدم داخل async def.
- صعوبة تحديد التوازن الصحيح بين عدد Workers (Processes) وعدد الاتصالات المتزامنة لكل Worker.
- التعامل الصحيح مع إغلاق الاتصالات والموارد (Connection Pools) بشكل نظيف عند إيقاف السيرفر (Graceful Shutdown).

ملخص الفصل

- FastAPI مبني على asyncio، وبيفرّق تلقائيًا بين async def Routes (تشتغل على الـ Event Loop) وdef Routes (تشتغل في Thread Pool منفصل).
- BackgroundTasks مناسبة للمهام الخفيفة بعد الرد، والمهام الثقيلة تحتاج Task Queue متخصص.
- Gunicorn وUvicorn بيديروا عدة Worker Processes لاستغلال كل الـ CPU cores المتاحة.
- استخدام مكتبات async مخصصة (لقواعد البيانات، Redis، الشبكة) ضروري لتجنب توقف الـ Event Loop.
- الهندسة الأنضج في الإنتاج بتجمع Multiprocessing (على مستوى Workers) مع Async I/O (جوه كل Worker) مع Process Pool (للحسابات الثقيلة) في نظام واحد متكامل.

الفصل العاشر ركّز على استقبال وخدمة الطلبات. الفصل الحادي عشر هياخدنا لجانب مختلف من نفس المشروع: خط الإنتاج بتاع النموذج نفسه — إزاي تستخدم Parallel Programming لتسريع تحميل ومعالجة البيانات، Feature Engineering، Batch Inference، والبحث عن أفضل Hyperparameters، وهنقارن Parallelism على مستوى CPU مقابل GPU في سياق Machine Learning بالتحديد.

الفصل الحادي عشر: البرمجة المتوازية في Machine Learning

الفصل العاشر ركّز على جانب خدمة الطلبات (Serving). الفصل ده هياخدنا لجانب تاني بنفس الأهمية: كل المراحل اللي بتحصل قبل وبعد الـ Serving في مشروع ML حقيقي — تحميل ومعالجة البيانات، هندسة الـ Features، تنفيذ Inference على دفعات كبيرة، والبحث عن أفضل Hyperparameters. هنطبّق كل أدوات المذكرة (Threading, Multiprocessing, Async) على السياقات المحددة دي، وهنقارن أخيرًا بين Parallelism على مستوى CPU وGPU.

Parallel Data Loading

المشكلة التي يحلها

تدريب نموذج Deep Learning بيحتاج تدفق مستمر من دفعات البيانات (Batches) للمعالج الرسومي (GPU). لو تحميل ومعالجة كل دفعة بيحصل بشكل Sequential (تحميل، ثم معالجة، ثم إرسال للـ GPU، وبعدين نبدأ الدفعة اللي بعدها)، الـ GPU هيقعد فاضي (Idle) فترات طويلة وهو مستني الدفعة الجاية تتجهز، وده بيضيع أغلى مورد في عملية التدريب.

> 📌 **ملاحظة مهمة**
>
> الـ GPU من أغلى الموارد في أي مشروع Deep Learning، سواء من ناحية التكلفة المالية أو الوقت. أي وقت GPU بيقضيه فاضي بينتظر بيانات هو هدر مباشر لموارد باهظة الثمن.

الحل: Data Loading متوازي

الحل القياسي هو تحميل ومعالجة الدفعة التالية بالتوازي مع تدريب النموذج على الدفعة الحالية، باستخدام Worker Processes منفصلة (زي DataLoader في PyTorch اللي بيستخدم multiprocessing من تحت السطح). بينما الـ GPU مشغول بمعالجة الدفعة الحالية (عملية CPU/GPU-bound)، عدة Processes بتحمّل وتجهّز الدفعات الجاية (عملية I/O-bound أو CPU-bound خفيفة) في الخلفية.

```
from torch.utils.data import DataLoader

num_workers # بيحدد عدد الـ Processes اللي هتحمّل وتجهّز البيانات بالتوازي
GPU مع تدريب النموذج على الـ #
)train_loader = DataLoader
,dataset
,batch_size=64
,num_workers=4      # مبني على multiprocessing من تحت السطح
,pin_memory=True
(

:for batch in train_loader
workers بينما الـ GPU بيعالج الدفعة دي، الـ # بيجهزوا الدفعة الجاية
train_step(batch)
```

> ✅ **أفضل الممارسات**
>
> قيمة num_workers المثالية مش رقم ثابت — زي ما شرحنا في الفصل التاسع، لازم تُحدَّد بالقياس الفعلي. رقم كبير جدًا ممكن يسبب Overhead في التنسيق والـ Context Switching زيادة عن المطلوب، ورقم صغير جدًا ممكن يخلي الـ GPU ينتظر.

Parallel Feature Engineering و Data Preprocessing

معالجة البيانات الأولية (تنظيف، تطبيع، تحويل) وهندسة الـ Features (استخراج متغيرات جديدة من البيانات الخام) غالبًا عمليات CPU-bound بالكامل، خصوصًا لو مكتوبة بكود Python نقي. زي ما شرحنا في الفصلين الثالث والخامس، الحل الأنسب هنا هو Multiprocessing، مش Threading (اللي هيتقيد بالـ GIL من غير فايدة حقيقية).

```
from concurrent.futures import ProcessPoolExecutor
import pandas as pd

:def process_chunk(chunk)
chunk['feature_1'] = chunk['value'].apply(complex_transform)
chunk['feature_2'] = chunk['category'].map(category_encoding)
return chunk

:'__if __name__ == '__main
df = pd.read_csv('large_dataset.csv')
أجزاء 8 لـ تقسيم البيانات #   chunks = np.array_split(df, 8)
:with ProcessPoolExecutor(max_workers=8) as executor
processed_chunks = list(executor.map(process_chunk, chunks))
result = pd.concat(processed_chunks)
```

> 🏢 **من واقع الشركات**
>
> لو العملية بتعتمد بشكل كبير على مكتبات زي NumPy أو Pandas اللي بتنفذ جزء كبير من عملياتها في كود C خارج الـ GIL (زي ما شرحنا في الفصل التالت)، ممكن تكتشف إن الفايدة من Multiprocessing أقل من المتوقع، لأن جزء من العملية أصلًا شبه متوازي داخليًا. القياس الفعلي (Benchmark) هو الطريقة الوحيدة للتأكد.

Parallel Prediction و Batch Inference

لو محتاج تشغّل Inference على مليون عينة دفعة واحدة (مش طلب واحد في كل مرة زي في خدمة Serving)، أفضل أسلوب بيعتمد على طبيعة النموذج:

السبب	الأسلوب المناسب	نوع الموقف
الـ GPU مصمم للتوازي الداخلي الهائل على مستوى العتاد	دفعات كبيرة (Large Batches) على نفس GPU	نموذج بيشتغل على GPU (Deep Learning)
استغلال عدة Cores فعليًا لتسريع المعالجة الكلية	Multiprocessing على عدة CPU cores	نموذج CPU-bound بسيط (زي Scikit-learn لبعض الخوارزميات)
حسب كون العملية CPU-bound أو I/O-bound	Multiprocessing أو Threading حسب طبيعة المعالجة	Preprocessing قبل الـ Inference
العملية أساسًا I/O-bound (انتظار شبكة)	Async أو Threading	استدعاء عدة نماذج API خارجية للـ Inference

```
from concurrent.futures import ProcessPoolExecutor
```

```
import numpy as np

:def predict_chunk(chunk, model)
return model.predict(chunk)

:'__if __name__ == '__main
data_chunks = np.array_split(large_dataset, 8)
:with ProcessPoolExecutor(max_workers=8) as executor
futures = [executor.submit(predict_chunk, chunk, model) for chunk in
[data_chunks
predictions = [f.result() for f in futures]
final_predictions = np.concatenate(predictions)
```

Distributed Training — المفاهيم الأساسية

لما حجم النموذج أو البيانات يكبر لدرجة إن جهاز واحد (حتى بعدة GPUs) مش كفاية، بندخل في عالم الـ Distributed Training — تدريب النموذج موزّع على عدة أجهزة مع بعض. الفصل ده مش هيغطي التفاصيل التقنية العميقة (ده موضوع كتاب منفصل)، لكن هيشرح المبادئ الأساسية اللي بتبني على كل حاجة اتعلمناها لحد دلوقتي.

Data Parallelism

نسخة من النموذج بالكامل بتتحط على كل جهاز (أو كل GPU)، وكل نسخة بتتدرب على جزء مختلف من البيانات في نفس الوقت. بعد كل خطوة، الأجهزة بتتبادل وتحدّث الـ Gradients مع بعض عشان كل النسخ تفضل متزامنة. المبدأ الأساسي هنا هو نفس فكرة Multiprocessing اللي اتعلمناها (كل وحدة معزولة بذاكرتها الخاصة)، لكن على مستوى أجهزة كاملة بدل Processes على نفس الجهاز.

Model Parallelism

لو النموذج نفسه كبير جدًا لدرجة إنه مش هيدخل في ذاكرة جهاز واحد، بيتم تقسيم النموذج نفسه (طبقات مختلفة) على أجهزة مختلفة، وكل جزء بيعالج البيانات بالتتابع أو بالتوازي حسب البنية. المبدأ هنا أقرب لفكرة Pipeline اللي شرحناها في نمط Producer-Consumer (الفصل التامن)، لكن كل مرحلة هنا موجودة على جهاز مختلف.

متى يُستخدم	الفكرة	النهج
النموذج يدخل في ذاكرة جهاز واحد، والبيانات كبيرة جدًا	نفس النموذج، بيانات مختلفة على كل جهاز	Data Parallelism
النموذج نفسه أكبر من ذاكرة جهاز واحد	نفس البيانات، أجزاء مختلفة من النموذج على كل جهاز	Model Parallelism

من واقع الشركات

الأنظمة الموزّعة للتدريب (زي PyTorch Distributed أو Horovod) بتبني فعليًا فوق نفس مبادئ الـ IPC و Message Passing اللي شرحناها في الفصل التامن، لكن عبر الشبكة بدل داخل جهاز واحد — الفرق في المقياس (Scale)، مش في المبدأ الأساسي.

Parallel Hyperparameter Search

البحث عن أفضل إعدادات نموذج (Hyperparameters) بيحتاج تجربة عدد كبير من التوليفات المختلفة، وكل تجربة (تدريب نموذج بإعدادات معينة) مستقلة تمامًا عن التجارب التانية — سيناريو مثالي جدًا للـ Parallelism، لأن مفيش أي حاجة مشتركة بين التجارب محتاجة تزامن.

```
from concurrent.futures import ProcessPoolExecutor

:def train_with_params(params)
model = build_model(params)
score = train_and_evaluate(model)
return params, score

] = param_combinations
,{lr": 0.01, "depth": 5"}
,{lr": 0.01, "depth": 10"}
,{lr": 0.001, "depth": 5"}
,{lr": 0.001, "depth": 10"}
عشرات أو مئات التوليفات ... #
[

:'__if __name__ == '__main
:with ProcessPoolExecutor(max_workers=4) as executor
results = list(executor.map(train_with_params, param_combinations))
best_params, best_score = max(results, key=lambda r: r[1])
```

⚠️ خطأ شائع

لو كل تجربة تدريب بتاخد وقت طويل جدًا (ساعات)، عدد Processes أكبر من عدد GPUs/Cores المتاحة فعليًا مش هيسرّع حاجة — كل GPU بيقدر يشغّل تجربة واحدة بكفاءة كاملة في كل مرة. المبدأ زي ما شرحنا في الفصل التاسع: العدد الأمثل بيعتمد على الموارد الفعلية المتاحة.

GPU مقابل CPU Parallelism

الفرق الجوهري

الـ CPU مصمم لعدد قليل نسبيًا من الـ Cores (عادة من ٤ لـ ٦٤)، كل واحد قوي جدًا وقادر ينفذ تعليمات معقدة ومتنوعة. الـ GPU على العكس تمامًا: آلاف الـ Cores الأبسط، مصممة أصلًا لتنفيذ نفس العملية البسيطة (زي ضرب أو جمع) على كمية هائلة من البيانات في نفس الوقت بالظبط — وده بالظبط شكل العمليات اللي بتحصل جوه الشبكات العصبية (ضرب مصفوفات ضخمة).

المعيار	CPU Parallelism	GPU Parallelism
عدد وحدات المعالجة	قليل نسبيًا (٤-٦٤ core)	آلاف الـ Cores البسيطة
نوع المهام المناسبة	منطق معقد ومتنوع، مهام عامة	عمليات رياضية متكررة على بيانات ضخمة (مصفوفات)
أفضل استخدام في ML	Preprocessing, Feature Engineering, منطق عام	تدريب وInference للشبكات العصبية
أدوات Python الأساسية	multiprocessing, concurrent.futures	PyTorch/TensorFlow (بيديروا التوازي على الـ GPU داخليًا)

المعيار	CPU Parallelism	GPU Parallelism
مستوى التحكم اليدوي المطلوب	عالٍ (أنت بتدير الـ Workers بنفسك)	منخفض عادة (المكتبة بتدير التوازي تلقائيًا)

⚠ خطأ شائع

غلط شائع: محاولة استخدام multiprocessing يدويًا لتسريع عمليات مصفوفات كبيرة على الـ GPU. مكتبات زي PyTorch وTensorFlow بتدير التوازي على الـ GPU داخليًا بكفاءة عالية جدًا، ومحاولة فرض Multiprocessing من فوقها غالبًا بتضيف تعقيد وOverhead من غير فايدة حقيقية.

كيف تُستخدم هذه المفاهيم داخل الشركات الكبرى

فرق الـ ML في الشركات الكبرى بتصمم خطوط الإنتاج (Pipelines) بتاعتها عشان تفصل بوضوح بين المراحل CPU-bound (تجهيز البيانات، هندسة الـ Features) والمراحل اللي بتستفيد من GPU (التدريب، الـ Inference للنماذج الكبيرة). المرحلتين غالبًا بتشتغلوا بالتوازي مع بعض (Pipeline Parallelism) عشان يقللوا الوقت الكلي، بنفس فلسفة Producer-Consumer اللي شرحناها في الفصل التامن: مرحلة تجهيز البيانات (Producer) بتشتغل بالتوازي مع مرحلة التدريب (Consumer).

أين تقع هذه المفاهيم داخل مشروع Machine Learning حقيقي — ملخص شامل

- تحميل البيانات: Multiprocessing (DataLoader) بالتوازي مع التدريب على GPU.
- معالجة وهندسة Features: Multiprocessing لعمليات CPU-bound ثقيلة.
- التدريب نفسه: Parallelism على مستوى الـ GPU، وأحيانًا Distributed Training على عدة أجهزة.
- البحث عن Hyperparameters: تجارب مستقلة بالتوازي عبر Process Pool.
- Batch Inference: توزيع الدفعات على GPU أو Process Pool حسب طبيعة النموذج.
- خدمة النموذج (Serving): مزيج Async + Process Pool زي ما شرحنا في الفصل العاشر.

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

استخدم DataLoader (أو ما يعادله) بـ num_workers مضبوط بالقياس الفعلي عشان الـ GPU مايفضلش فاضي.

افصل بوضوح بين مراحل CPU-bound (استخدم Multiprocessing) ومراحل GPU-bound (خلي المكتبة تدير التوازي).

استخدم Process Pool للتجارب المستقلة (Hyperparameter Search) طالما الموارد كافية.

قِس دايمًا هل الـ GPU هو عنق الزجاجة فعليًا، أم إن تجهيز البيانات هو اللي بيبطئ العملية الكلية (زي ما شرحنا في الفصل التاسع).

⚠ خطأ شائع

استخدام Threading لمعالجة بيانات CPU-bound بكود Python نقي — الـ GIL هيمنع أي تسريع حقيقي.

محاولة فرض Multiprocessing يدوي فوق عمليات GPU اللي المكتبة أصلًا بتديرها بكفاءة أعلى.

تجاهل الوقت اللي الـ GPU بيقضيه فاضي بسبب بطء تجهيز البيانات، والتركيز فقط على تسريع النموذج نفسه.

تشغيل عدد Processes للـ Hyperparameter Search أكبر بكتير من عدد الموارد الفعلية المتاحة (GPUs/Cores).

مثال عملي صغير

مقارنة بسيطة توضح تأثير num_workers على وقت تحميل ومعالجة دفعات البيانات:

```
import time
from concurrent.futures import ProcessPoolExecutor

:def preprocess_batch(batch_id)
CPU-bound محاكاة معالجة #
x = sum(i**2 for i in range(1_000_000))
return batch_id

:def benchmark_workers(num_workers, num_batches=16)
()start = time.time
:with ProcessPoolExecutor(max_workers=num_workers) as executor
list(executor.map(preprocess_batch, range(num_batches)))
print(f"Workers={num_workers}: {time.time()-start:.2f}s")

:'__if __name__ == '__main
:for w in [1, 2, 4, 8]
benchmark_workers(w)
```

مثال من مشروع Production حقيقي

فريق بيدرّب نموذج Computer Vision كبير على مجموعة بيانات ضخمة. أول نسخة من خط الإنتاج كانت بتحمّل وتعالج الصور بشكل Sequential، والـ GPU كان بيقضي أكتر من ٤٠٪ من وقته فاضي مستني البيانات. بعد تحويل مرحلة التحميل لـ DataLoader بعدد Workers مضبوط بناءً على قياس فعلي (وليس تخمين)، وقت التدريب الكلي قل بشكل كبير جدًا من غير أي تغيير في النموذج نفسه — استغلال أفضل للمورد الأغلى في العملية كلها (الـ GPU).

مشروع صغير لتطبيق ما تم شرحه

📌 **ملاحظة مهمة — مشروع الفصل الحادي عشر**

اكتب دالة بتحاكي 'معالجة دفعة بيانات' (زي عملية حسابية بسيطة)، وقارن وقت معالجة ٢٠ دفعة بشكل Sequential مقابل استخدام ProcessPoolExecutor بعدد Workers مختلف.

اكتب سكريبت بسيط بيحاكي Hyperparameter Search على ٨ توليفات مختلفة (بدالة تدريب وهمية بتاخد وقت عشوائي)، ونفذها بالتوازي باستخدام Process Pool، واطبع أفضل نتيجة.

لو متاح عندك PyTorch أو TensorFlow، جرّب فعليًا تغيير num_workers في DataLoader وقِس تأثيره على وقت تدريب Epoch واحد.

تحديات شائعة

- صعوبة تحديد العدد الأمثل لـ num_workers بدون تجربة فعلية على البيانات والجهاز الحقيقي.

- التفرقة الدقيقة بين الأجزاء اللي المكتبة (PyTorch/TensorFlow) بتديرها تلقائيًا بكفاءة، والأجزاء اللي محتاجة توازي يدوي منك.
- صعوبة قياس مدى استغلال الـ GPU الفعلي (هل هو عنق الزجاجة أم لأ) بدون أدوات مراقبة متخصصة.

ملخص الفصل

- Parallel Data Loading أساسي لمنع الـ GPU من الانتظار فاضي أثناء التدريب.
- معالجة البيانات وهندسة الـ Features غالبًا CPU-bound، والحل الأنسب لها Multiprocessing.
- Batch Inference وHyperparameter Search سيناريوهات مثالية للـ Parallelism لأن الوحدات مستقلة تمامًا عن بعض.
- Data Parallelism وModel Parallelism هما المبدآن الأساسيان للـ Distributed Training، وبيبنوا فوق نفس مبادئ Multiprocessing وMessage Passing لكن عبر الشبكة.
- الـ GPU مصمم لتوازي هائل على عمليات رياضية بسيطة ومتكررة، بينما الـ CPU مناسب لمنطق أعقد وأكتر تنوعًا — والمكتبات المتخصصة (PyTorch/TensorFlow) بتدير توازي الـ GPU تلقائيًا.

لحد دلوقتي غطينا كل تقنيات الـ Concurrency والـ Parallelism وتطبيقاتها في الـ ML. الفصل الثاني عشر هياخدنا لمستوى أعلى: أنماط تصميم جاهزة (Production Patterns) بتُستخدم في بناء أنظمة إنتاج حقيقية موثوقة — Task Queues وRate Limiting، Idempotency، Retry Patterns، Scheduled Jobs، Celery، وDistributed Locks. دي الأنماط اللي بتحوّل الأدوات اللي اتعلمناها لنظام إنتاج جاهز يتحمل الأعطال ويتعافى منها بشكل صحيح.

الفصل الثاني عشر: أنماط الإنتاج — Production Patterns

كل الأدوات اللي اتعلمناها لحد دلوقتي (Threading, Multiprocessing, Async, Queues) هي لبنات بناء. الفصل ده هيوريك إزاي تركّب اللبنات دي في أنماط تصميم جاهزة ومجرّبة، بتُستخدم في كل نظام إنتاج حقيقي محترم: Task Queues موزّعة، جدولة مهام دورية، إعادة المحاولة عند الفشل، ضمان عدم تكرار العملية، تحديد معدل الطلبات، والأقفال الموزّعة.

Task Queue وJob Queue

في الفصل الثامن شرحنا Queue كأداة تواصل داخل عملية واحدة (Threads/Processes على نفس الجهاز). في الإنتاج، محتاجين غالبًا نفس الفكرة لكن موزّعة: مهام بتتحط في طابور، وWorkers (ممكن يكونوا على أجهزة مختلفة تمامًا) بياخدوا منها وينفذوها. الفرق الجوهري: الطابور نفسه لازم يكون نظام مستقل وموثوق (زي Redis أو RabbitMQ)، مش مجرد كائن Queue في الذاكرة، عشان يفضل موجود حتى لو Worker معين اتعطل أو اتعاد تشغيله.

Celery وRedis Queue

Celery هي المكتبة الأشهر في Python لإدارة Task Queues موزّعة في الإنتاج. بتستخدم Broker (زي Redis أو RabbitMQ) لتخزين المهام، وعدد Workers (على نفس الجهاز أو أجهزة مختلفة) بياخدوا المهام وينفذوها بشكل مستقل.

```
from celery import Celery

app = Celery('tasks', broker='redis://localhost:6379/0')

app.task@
:def process_image(image_id)
منفصل Worker في بتتنفذ ،(CPU-bound) ثقيلة معالجة #
"{image_id} الصورة معالجة تمت"return f

:(FastAPI مسار (مثلًا السيرفر كود من #
يرجع فورًا، لا ينتظر النتيجة #   result = process_image.delay(image_id=42)
(Blocking عملية دي في :بحذر) لاحقًا الفعلية النتيجة احتجت لو ()result.get #
```

من واقع الشركات

الفرق بين BackgroundTasks في FastAPI (الفصل العاشر) وCelery: BackgroundTasks بتشتغل جوه نفس الـ Process، مناسبة للمهام الخفيفة والسريعة. Celery بتشتغل في Workers منفصلة تمامًا (وممكن على أجهزة مختلفة)، مناسبة للمهام الثقيلة أو الطويلة أو اللي محتاجة ضمان تنفيذ حتى لو السيرفر الأساسي اتعاد تشغيله.

Scheduled Jobs — المهام المجدولة

بعض المهام لازم تتنفذ بشكل دوري (كل ساعة، كل يوم) بدل ما تتنفذ فور حدوث حدث معين. Celery Beat (جزء من نظام Celery) بيوفر جدولة زمنية لهذا النوع من المهام.

```
from celery.schedules import crontab

} = app.conf.beat_schedule
} :'retrain-model-daily'
,'task': 'tasks.retrain_model'
```

```
كل يوم الساعة 2 صباحًا #    ,schedule': crontab(hour=2, minute=0)'
,{
{
```

Retry Pattern — إعادة المحاولة

في أي نظام موزّع، الفشل المؤقت (شبكة بطيئة، خدمة خارجية مش متاحة لحظيًا) أمر متوقع ومش استثنائي. نمط Retry بيضمن إن المهمة تُعاد محاولتها تلقائيًا عند الفشل، غالبًا مع فترة انتظار متزايدة بين كل محاولة (Exponential Backoff) عشان ميضغطش على النظام المتعطل أكتر.

```
from celery import Celery

app.task(bind=True, max_retries=3, default_retry_delay=5)@
:def call_external_service(self, data)
:try
return external_api_call(data)
:except ConnectionError as exc
raise self.retry(exc=exc, countdown=2 ** self.request.retries)
(Exponential Backoff) 4 الثالثة بعد ،2 الثانية بعد ،1 ثانية بعد المحاولة الأولى #
```

أفضل الممارسات

استخدم دايمًا Exponential Backoff بدل إعادة المحاولة فورًا وبنفس الوتيرة — إعادة المحاولة الفورية المتكررة على خدمة متعطلة أصلًا ممكن تزود الضغط عليها وتأخر تعافيها (ظاهرة تُعرف أحيانًا بـ Retry Storm).

Idempotency — ضمان عدم التكرار

لو مهمة معينة (زي خصم مبلغ من حساب، أو إرسال إشعار) اتنفذت مرتين بسبب Retry أو أي خطأ تقني، النتيجة المفروض تكون نفسها زي لو اتنفذت مرة واحدة بس. العملية اللي بتحقق الخاصية دي بتُسمى Idempotent. تحقيقها عمليًا بيكون غالبًا عن طريق معرّف فريد لكل عملية (Idempotency Key)، والتأكد إن العملية بنفس المعرّف مش بتتنفذ مرتين.

```
:def process_payment(idempotency_key, amount, account_id)
:if payment_already_processed(idempotency_key)
لا تنفذ مرة أخرى #    return get_existing_result(idempotency_key)
result = charge_account(account_id, amount)
save_result(idempotency_key, result)
return result
```

خطأ شائع

أي مهمة بتُعاد محاولتها تلقائيًا (Retry Pattern) لازم تكون Idempotent، وإلا ممكن تنفذ نفس العملية أكتر من مرة بالغلط — الاتنين لازم يتصمموا مع بعض، مش كل واحد لوحده.

Rate Limiting — تحديد معدل الطلبات

لحماية النظام (أو خدمة خارجية) من حمل زائد، Rate Limiting بيحدد أقصى عدد طلبات مسموح بيها في فترة زمنية معينة. المبدأ الأساسي شبيه جدًا بـ Semaphore اللي شرحناه في الفصل الرابع، لكن على مستوى الوقت (Requests per Second) بدل عدد الوحدات المتزامنة.

```
import time

:class RateLimiter
:def __init__(self, max_calls, period)
self.max_calls = max_calls
self.period = period
[] = self.calls

:def allow(self)
()now = time.time
self.calls = [t for t in self.calls if now - t < self.period]
:if len(self.calls) < self.max_calls
self.calls.append(now)
return True
return False

طلب كل دقيقة limiter = RateLimiter(max_calls=100, period=60)   # 100
:()if limiter.allow
()make_request
:else
("تجاوزت الحد المسموح، حاول لاحقًا")print
```

Distributed Locks — الأقفال الموزّعة

الـ Lock اللي شرحناه في الفصل الرابع (threading.Lock) بيشتغل بس جوه نفس الـ Process. لو عندك عدة Workers على أجهزة مختلفة محتاجين يضمنوا إن Worker واحد بس ينفذ عملية معينة في وقت واحد (زي تحديث ملف مشترك، أو تنفيذ مهمة مجدولة مرة واحدة بس مش من كل الأجهزة في نفس الوقت)، محتاج Distributed Lock — قفل مُدار عن طريق نظام مركزي (زي Redis) يقدر كل الأجهزة توصله.

```
import redis

()r = redis.Redis

:def acquire_lock(lock_name, timeout=10)
return r.set(lock_name, "locked", nx=True, ex=timeout)
يعيّن القيمة فقط لو المفتاح مش موجود أصلًا (أي القفل فاضي) :nx=True #
القفل بيتحرر تلقائيًا بعد المدة دي حتى لو حصل عطل :ex=timeout #

:if acquire_lock("daily_report_job")
()generate_daily_report
r.delete("daily_report_job")
:else
("مهمة تانية شغالة بالفعل على جهاز آخر")print
```

✅ أفضل الممارسات

دايمًا حدد مهلة انتهاء (Expiration/TTL) للأقفال الموزّعة. لو الـ Worker الماسك القفل اتعطل فجأة بدون تحرير القفل، بدون مهلة انتهاء، القفل هيفضل ماسك للأبد، وأي عملية تانية محتاجاه هتقف بلا داعٍ (شبيه بمشكلة الـ Deadlock اللي شرحناها في الفصل الرابع، لكن على مستوى النظام الموزّع).

مقارنة الأنماط الأساسية

المشكلة التي يحلها	النمط
تنفيذ مهام ثقيلة أو طويلة خارج السيرفر الأساسي، بشكل موثوق	Task Queue (Celery)
تنفيذ مهام دورية بدون تدخل يدوي	Scheduled Jobs
التعامل مع الفشل المؤقت في الأنظمة الموزّعة بشكل آمن	Retry + Exponential Backoff
ضمان عدم تكرار تنفيذ نفس العملية عند إعادة المحاولة	Idempotency
حماية النظام أو خدمة خارجية من حمل زائد	Rate Limiting
ضمان تنفيذ عملية معينة من جهاز واحد بس في وقت واحد	Distributed Locks

كيف تُستخدم داخل الشركات الكبرى

الأنماط دي مش اختيارية في أنظمة الإنتاج الكبرى — هي المعيار المتوقع. أي خدمة بتتعامل مع مهام خلفية، جدولة، أو تنسيق بين عدة أجهزة، غالبًا بتستخدم مزيج من الأنماط دي مع بعض. مثلًا: Task Queue لتنفيذ المهام، مع Retry وIdempotency لضمان الموثوقية، مع Rate Limiting لحماية الخدمات الخارجية اللي بيتم استدعاؤها.

أين تقع داخل مشروع Machine Learning حقيقي

- إعادة تدريب نموذج بشكل دوري (يوميًا/أسبوعيًا) عن طريق Scheduled Job.
- معالجة صور أو ملفات مرفوعة من المستخدم بشكل غير متزامن عن طريق Task Queue (Celery)، بدل ما المستخدم ينتظر المعالجة كاملة.
- التعامل مع فشل مؤقت في استدعاء API خارجي لجلب Features، باستخدام Retry مع Exponential Backoff.
- ضمان عدم تنفيذ نفس طلب Inference مرتين لو حصل Timeout والمستخدم بعت الطلب تاني، عن طريق Idempotency Key.
- حماية خدمة Model Serving من حمل زائد يفوق قدرتها، عن طريق Rate Limiting.

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

استخدم Task Queue متخصص (Celery أو ما يعادله) للمهام الثقيلة أو الطويلة، مش BackgroundTasks البسيطة.

صمّم أي عملية قابلة لإعادة المحاولة بحيث تكون Idempotent من الأساس.

استخدم Exponential Backoff دايمًا مع Retry، وحدد حد أقصى لعدد المحاولات.

حدد مهلة انتهاء (TTL) واضحة لأي Distributed Lock.

⚠️ خطأ شائع

تصميم Retry بدون التأكد من كون العملية Idempotent — بيؤدي لتنفيذ العملية أكتر من مرة بالخطأ.

إعادة محاولة فورية بلا Backoff على خدمة متعطلة أصلًا — بيزود الضغط عليها ويأخر تعافيها.

Distributed Lock بدون مهلة انتهاء — لو الـ Worker الماسك القفل اتعطل، القفل بيفضل ماسك للأبد.

استخدام BackgroundTasks البسيطة لمهام ثقيلة جدًا بدل Task Queue متخصص، مما يؤثر على أداء السيرفر الأساسي.

مثال عملي صغير

```
import time, random

:()def unreliable_operation
:if random.random() < 0.7
("فشل مؤقت")raise ConnectionError
"نجحت العملية" return

:def retry_with_backoff(func, max_retries=5)
:for attempt in range(max_retries)
:try
()return func
:except ConnectionError
wait = 2 ** attempt
("{wait}s الانتظار، فشلت {attempt+1} محاولة"f)print
time.sleep(wait)
("فشلت كل المحاولات")raise Exception

result = retry_with_backoff(unreliable_operation)
print(result)
```

مثال من مشروع Production حقيقي

خدمة بتستقبل طلبات لمعالجة فيديوهات (عملية ثقيلة جدًا وطويلة). بدل ما تخلي المستخدم ينتظر، الطلب بيتحط في Celery Task Queue، ويرجع للمستخدم فورًا رقم مرجعي (Idempotency Key) يقدر يتابع بيه حالة الطلب. الـ Worker اللي بيعالج الفيديو بيستخدم Retry مع Backoff لو محتاج يستدعي خدمة خارجية (زي تخزين سحابي)، ولو نفس الطلب اتبعت مرتين بالغلط (بسبب مشكلة شبكة عند المستخدم)، الـ Idempotency Key بيضمن إن الفيديو ميتعالجش مرتين.

مشروع صغير لتطبيق ما تم شرحه

📌 **ملاحظة مهمة — مشروع الفصل الثاني عشر**

اكتب دالة retry_with_backoff عامة (زي المثال أعلاه) وطبّقها على عملية وهمية بتفشل بشكل عشوائي.

صمّم دالة process_order بسيطة تستخدم Idempotency Key (Dictionary في الذاكرة كبديل لقاعدة بيانات) لضمان عدم معالجة نفس الطلب مرتين.

لو متاح عندك Redis، جرّب فعليًا تطبّق Distributed Lock بسيط باستخدام SET NX EX، وحاكي جهازين بيحاولوا ياخدوا نفس القفل في نفس الوقت.

تحديات شائعة

- تصميم عمليات Idempotent فعليًا صعب أكتر مما يبدو، خصوصًا مع عمليات معقدة متعددة الخطوات.
- اختيار مدة الـ TTL المناسبة للـ Distributed Locks: قصيرة جدًا قد تنتهي قبل ما العملية تخلص، وطويلة جدًا تأخر التعافي من الأعطال.

- موازنة عدد محاولات Retry مع تجربة المستخدم — كتير جدًا يؤخر معرفة الفشل النهائي، وقليل جدًا يفشل عمليات كانت هتنجح لو انتظرنا شوية.

ملخص الفصل

- Task Queue (زي Celery) توسّع فكرة Queue لتشمل Workers موزّعة على أجهزة مختلفة بشكل موثوق.
- Scheduled Jobs تنظم تنفيذ المهام الدورية بدون تدخل يدوي.
- Retry مع Exponential Backoff أساسي للتعامل مع الفشل المؤقت المتوقع في الأنظمة الموزّعة.
- Idempotency ضرورية لأي عملية قابلة لإعادة المحاولة، لضمان عدم تنفيذها أكتر من مرة بالخطأ.
- Rate Limiting وDistributed Locks أدوات أساسية لحماية النظام وتنسيق العمل بين عدة أجهزة.

لحد دلوقتي بنينا وصممنا أنظمة Concurrent وParallel، لكن السؤال المهم اللي كل مهندس بيواجهه: إيه اللي بيحصل لما حاجة تغلط؟ الفصل الثالث عشر هيغطي Debugging Concurrent Applications: إزاي تكتشف Deadlocks، إزاي تتابع Threads (Debug) وAsync Code، وإزاي تستخدم Logging وTracing وProfiling بشكل صحيح مخصص للأنظمة متعددة الوحدات — مهارة أساسية مختلفة تمامًا عن Debugging الكود التتابعي العادي.

الفصل الثالث عشر: تتبع الأخطاء في التطبيقات المتزامنة

Debugging الكود التتابعي العادي بسيط نسبيًا: خطأ بيحصل، بتشوف الـ Stack Trace، بتلاقي السطر، بتصلحه.
Debugging الكود المتزامن مختلف جذريًا: الأخطاء (زي Race Conditions) ممكن متظهرش في كل تشغيل، الـ Stack Trace ممكن يكون مربك لما فيه عدة Threads أو Coroutines شغالة، والمشكلة ممكن تكون في التوقيت نفسه مش في منطق الكود. الفصل ده هيغطي الأدوات والتقنيات المخصصة لتتبع الأخطاء في الأنظمة متعددة الوحدات.

اكتشاف الـ Deadlocks

زي ما شرحنا في الفصل الرابع، الـ Deadlock بيسبب توقف كامل للبرنامج بدون أي رسالة خطأ واضحة — البرنامج ببساطة بيقف ومايستجبش. أول خطوة في تشخيص Deadlock هي معرفة مين ماسك إيه، ومين مستني إيه.

```
import faulthandler, signal

# تفعيل faulthandler بيسمح لك تطبع حالة كل الـ Threads الحالية
# عن طريق إرسال إشارة معينة للبرنامج وهو واقف (على Unix)
faulthandler.register(signal.SIGUSR1)

# من طرفية تانية وأنت البرنامج واقف (متجمد):
# <kill -SIGUSR1 <PID
# هيطبع الـ Stack Trace الحالي لكل الـ Threads، وهتقدر تشوف
# مين واقف عند أي نقطة (زي lock.acquire()) وليه
```

> ✅ **أفضل الممارسات**
>
> أفضل استراتيجية للتعامل مع Deadlock هي منعه أصلًا (زي ما شرحنا في الفصل الرابع: ترتيب ثابت لأخذ الأقفال) بدل الاعتماد على اكتشافه بعد حدوثه — التشخيص بعد الحدوث ممكن ياخد وقت طويل جدًا في أنظمة معقدة.

تتبع أخطاء Threads

لو Exception حصل جوه Thread فرعي، مش هيوقف البرنامج الرئيسي تلقائيًا زي ما بيحصل في الكود العادي — الخطأ ممكن يمر بصمت لو محدش بيتابع نتيجة الـ Thread ده بشكل صريح.

```
import threading

def risky_task():
    raise ValueError("خطأ حصل جوه Thread")

def thread_with_error_handling(target):
    def wrapper():
        try:
            target()
        except Exception as e:
            print(f"Thread فشل بخطأ: {e}")
            # يمكن هنا تسجيل الخطأ في نظام Logging مركزي
    return threading.Thread(target=wrapper)

t = thread_with_error_handling(risky_task)
t.start()
```

```
()t.join
```

⚠️ خطأ شائع

خطأ شائع جدًا: افتراض إن Exception في Thread فرعي هيوصل تلقائيًا للكود الرئيسي أو هيوقف البرنامج. عمليًا، الخطأ بيتسجل في الـ Thread نفسه بس، ولازم تصميم صريح (زي wrapper بيمسك الأخطاء، أو متابعة عن طريق concurrent.futures اللي بترفع الخطأ عند result()) لضمان عدم ضياعه.

تتبع أخطاء الكود غير المتزامن (Async)

في asyncio، لو Task فشل ومحدش عمل await ليه أو تابعه، الخطأ ممكن يتسجل في الخلفية بس (زي رسالة 'Task exception was never retrieved') من غير ما يوقف البرنامج أو يبان بوضوح، وده بيخلي اكتشافه صعب لو محدش بيراقب الـ Logs بعناية.

```
import asyncio

:()async def failing_task
("async خطأ في مهمة")raise ValueError

:()async def main
task = asyncio.create_task(failing_task())
await asyncio.sleep(1)
await task هنا، الخطأ ممكن يتسجل بصمت لو ما عملناش #
في الخلفية بدون ما يوقف البرنامج #
:try
await task
:except ValueError as e
("{e} :اتلقط الخطأ"f)print

asyncio.run(main())
```

🏢 من واقع الشركات

استخدام asyncio.TaskGroup (اللي شرحناه في الفصل السادس) بيحل المشكلة دي بشكل منهجي: لو أي Task جوه المجموعة فشلت، الباقي بيتلغي تلقائيًا والخطأ بيتم رفعه بوضوح، بدل ما يضيع بصمت.

Logging في الأنظمة المتزامنة

في نظام فيه عدة Threads أو Processes بتشتغل مع بعض، الـ Logging العادي (طباعة رسائل بسيطة) بيبقى غير كافٍ — مستحيل تعرف أي رسالة جاية من أي Thread أو Process. لازم تضيف معرّف واضح لكل وحدة عمل في كل رسالة Log.

```
import logging, threading

)logging.basicConfig
,level=logging.INFO
,'format='%(asctime)s - %(threadName)s - %(message)s
Log تلقائيًا لكل رسالة Thread بيضيف اسم الـ threadName #
```

```
(

:()def worker
("بدأت الشغل")logging.info
معالجة ... #
("خلصت الشغل")logging.info

()threading.Thread(target=worker, name="Worker-1").start
()threading.Thread(target=worker, name="Worker-2").start
```

أفضل الممارسات

في Multiprocessing، لازم كمان تضيف Process ID لكل رسالة Log (زي %d(process) في الـ format)، وغالبًا محتاج نظام Logging مركزي (يستقبل رسائل من كل الـ Processes) بدل كل Process يكتب لملف منفصل، عشان تقدر تتابع تسلسل الأحداث عبر كل الوحدات بترتيب زمني واحد.

Tracing — التتبع الموزّع

في أنظمة أعقد (زي Microservices متعددة، أو نظام فيه عدة مراحل معالجة متوازية)، حتى الـ Logging المحسّن مش كفاية لفهم رحلة طلب واحد عبر كل المراحل. Distributed Tracing بيحل المشكلة دي: كل طلب بياخد معرّف فريد (Trace ID) بيتبعه في كل مرحلة، وبيسمح لك تتابع رحلته الكاملة عبر كل الوحدات المختلفة (حتى لو كانت على أجهزة مختلفة) في مكان واحد.

```
import uuid, logging

:()def handle_request
trace_id = str(uuid.uuid4())
("بدأ استقبال الطلب f"[{trace_id}])logging.info
process_stage_1(trace_id)
process_stage_2(trace_id)
("انتهى الطلب f"[{trace_id}])logging.info
بيتمرر لكل مرحلة، فتقدر تجمع كل الرسائل trace_id نفس #
المرتبطة بنفس الطلب من مصادر مختلفة #
```

Profiling للكود المتزامن

زي ما لمحنا في الفصل التاسع، أدوات Profiling العادية (cProfile) بتحلل تنفيذ Thread واحد بشكل جيد، لكن لتحليل التفاعل بين عدة Threads/Processes (مين شغال إمتى، مين واقف بينتظر إيه)، محتاج أدوات متخصصة أكتر:

الاستخدام	الأداة
رؤية حية لحالة كل الـ Threads في عملية Python شغالة، بدون تعديل الكود	py-spy
تفعيل PYTHONASYNCIODEBUG=1 بيكشف مشاكل شائعة زي Blocking calls جوه Coroutines	asyncio Debug Mode
طباعة Stack Trace لكل الـ Threads الحالية يدويًا وقت الحاجة	threading + traceback
مراقبة شاملة للأداء والأخطاء في بيئة الإنتاج عبر كل الخدمات	أدوات APM (زي Datadog, New Relic)

```
# تفعيل وضع تصحيح الأخطاء المدمج في asyncio
# يكتشف تلقائيًا: Coroutines لم يتم انتظارها، عمليات Blocking بطيئة داخل async، والمزيد
import asyncio
asyncio.run(main(), debug=True)
```

استراتيجيات عامة لتتبع الأخطاء المتزامنة

36. قلّل حجم الكود الحرج (Critical Section) قدر الإمكان — كل ما كان أصغر، كل ما كان أسهل في التحليل والتتبع.
37. أضف Logging مفصّل بمعرّفات واضحة (Thread name, Process ID, Trace ID) من البداية، مش بعد ما تواجه مشكلة.
38. حاول تكرار المشكلة (Reproduce) بشكل مضبوط — زود عدد التكرارات أو الحمل لزيادة احتمال ظهور Race Conditions النادرة.
39. بسّط الكود لأقل سيناريو ممكن يعيد إنتاج المشكلة (Minimal Reproducible Example) قبل محاولة التصحيح.
40. استخدم أدوات متخصصة (py-spy, asyncio debug mode) بدل الاعتماد فقط على print() العادية.

كيف تُستخدم داخل الشركات الكبرى

فرق الهندسة في الشركات الكبرى بتستثمر بشكل كبير في بنية تحتية للمراقبة (Observability): Logging مركزي، Distributed Tracing، وأدوات APM، كجزء أساسي من أي نظام إنتاج، مش كإضافة لاحقة. الهدف إن أي مشكلة في نظام متزامن أو موزّع تُكتشف وتُشخَّص في دقائق، مش ساعات، عن طريق أدوات جاهزة بدل التحقيق اليدوي المعقد.

أين يقع هذا الفصل داخل مشروع Machine Learning حقيقي

- تتبع طلب Inference واحد عبر كل مراحل المعالجة (استقبال، جلب Features، تنفيذ النموذج، إرجاع الرد) باستخدام Trace ID موحد.
- اكتشاف Worker معلّق في خط معالجة بيانات كبير باستخدام py-spy بدون إيقاف النظام بالكامل.
- تفعيل asyncio debug mode أثناء التطوير لاكتشاف أي عملية Blocking مخفية قبل النشر للإنتاج.
- تصميم نظام Logging موحد يجمع رسائل من كل Worker Processes في مكان واحد لتسهيل التحليل.

أفضل الممارسات والأخطاء الشائعة

✅ أفضل الممارسات

أضف معرّفات واضحة (Thread/Process/Trace ID) لكل رسالة Log من بداية المشروع.

فعّل asyncio debug mode أثناء التطوير والاختبار (وليس بالضرورة الإنتاج بسبب تكلفة أداء إضافية).

استخدم concurrent.futures أو TaskGroup بدل إنشاء Threads/Tasks بدون متابعة، لضمان ظهور الأخطاء بوضوح.

استثمر في أدوات Tracing وMonitoring من البداية، مش كرد فعل بعد حدوث مشكلة كبيرة في الإنتاج.

⚠️ خطأ شائع

الاعتماد على print() فقط لتتبع الأخطاء في نظام متعدد الـ Threads — الرسائل بتتداخل ويصعب تتبع مصدرها.

تجاهل رسائل 'Task exception was never retrieved' في asyncio — دي غالبًا مؤشر على خطأ حقيقي بيضيع بصمت.

محاولة تصحيح Race Condition بدون تبسيط الكود لأقل سيناريو ممكن يعيد إنتاجها أولًا.

عدم إضافة مهلة انتهاء (Timeout) عند محاولة تشخيص برنامج متجمد بسبب Deadlock محتمل.

مثال عملي صغير

```
import logging, threading, time

)logging.basicConfig
,level=logging.INFO
'format='%(asctime)s [%(threadName)s] %(message)s
(

:def worker(worker_id, lock, shared_data)
:for i in range(3)
:with lock
("يعدّل البيانات f"Worker {worker_id})logging.info
shared_data.append(worker_id)
time.sleep(0.1)

()lock = threading.Lock
[] = shared_data
] = threads
threading.Thread(target=worker, args=(i, lock, shared_data), name=f"W{i}")
for i in range(3)
[
()for t in threads: t.start
()for t in threads: t.join
print(shared_data)
```

مثال من مشروع Production حقيقي

فريق هندسي بيلاحظ إن خدمة معالجة طلبات بتتوقف أحيانًا بشكل عشوائي وغير متوقع (Hang) تحت حمل عالٍ، بدون رسائل خطأ واضحة. باستخدام py-spy لالتقاط حالة كل الـ Threads وقت التجمّد، الفريق بيكتشف إن Threadين بياخدوا أقفال بترتيب عكسي في مسارين مختلفين من الكود — Deadlock كلاسيكي زي ما شرحنا في الفصل الرابع، لكن كان صعب اكتشافه بدون أداة تتبع مباشرة لحالة الـ Threads وقت حدوث المشكلة فعليًا.

مشروع صغير لتطبيق ما تم شرحه

📌 **ملاحظة مهمة — مشروع الفصل الثالث عشر**

أضف Logging مفصّل (بمعرّف Thread واضح) لمشروع الـ Producer-Consumer اللي بنيته في الفصل الثامن، وتابع تسلسل الأحداث من الـ Logs.

اكتب سكريبت بسيط بيه Deadlock مقصود (زي مثال الفصل الرابع)، وحاول تستخدم faulthandler لالتقاط حالة الـ Threads وقت التجمّد.

فعّل asyncio debug mode على كود async فيه عملية Blocking مخفية (زي time.sleep عادي بدل asyncio.sleep)، وشوف هل الأداة بتكتشف المشكلة.

تحديات شائعة

- صعوبة تكرار (Reproduce) الأخطاء المرتبطة بالتوقيت بشكل موثوق للاختبار.
- كمية ضخمة من رسائل الـ Logs من عدة Workers، مما يصعّب تتبع تسلسل حدث معين بدون أدوات تصفية وتجميع مناسبة.
- التمييز بين مشكلة أداء حقيقية (Bottleneck) ومشكلة توقف كامل (Deadlock) بناءً على الأعراض الأولية فقط.

ملخص الفصل

- الأخطاء في الأنظمة المتزامنة غالبًا مرتبطة بالتوقيت، ومش بتظهر في كل تشغيل، مما يجعلها أصعب في الاكتشاف من أخطاء الكود التتابعي العادي.
- أخطاء Threads وTasks غير المتابَعة ممكن تضيع بصمت — لازم تصميم صريح لمعالجتها.
- Logging مفصّل بمعرّفات واضحة (Thread/Process/Trace ID) أساسي لأي نظام متزامن جاد.
- Distributed Tracing بيسمح بتتبع رحلة طلب واحد عبر عدة مراحل أو أجهزة مختلفة.
- أدوات متخصصة (py-spy, asyncio debug mode) ضرورية لتحليل التفاعل بين عدة وحدات عمل، بخلاف أدوات Profiling التقليدية.

لحد دلوقتي غطينا كل الأدوات والأنماط والتقنيات التفصيلية. الفصل الرابع عشر هيجمع كل حاجة اتعلمناها في مذكرة واحدة، في شكل قواعد ومبادئ عملية مختصرة وواضحة — دليل مرجعي سريع (Best Practices) تقدر ترجع له بسرعة وقت اتخاذ أي قرار Concurrency أو Parallelism، بدل الرجوع لكل الفصول من الأول.

الفصل الرابع عشر: دليل مرجعي شامل — Best Practices

الفصل ده مختلف عن باقي الفصول: مش هيقدّم مفاهيم جديدة، لكنه هيجمع أهم القواعد والمبادئ اللي اتغطت طول المذكرة في مكان واحد مرجعي وسريع، منظم بشكل عملي، عشان ترجع له بسرعة وقت اتخاذ أي قرار Concurrency أو Parallelism في مشروعك، بدل الرجوع لكل فصل على حدة.

خارطة القرار الأساسية: أي أداة أستخدم؟

الفصل المرجعي	الأداة المناسبة	نوع المهمة
الفصل الرابع	Threading	I/O-bound، عدد اتصالات قليل/متوسط
الفصل السادس	Async (asyncio)	I/O-bound، عدد اتصالات ضخم (آلاف)
الفصل الخامس	Multiprocessing	CPU-bound (حسابات، معالجة صور)
الفصل السابع	concurrent.futures	مزيج Threading/Multiprocessing بواجهة موحدة
الفصل الثامن	Queue / Producer-Consumer / Worker Pool	تنسيق وتوزيع مهام بين وحدات عمل
الفصل الثاني عشر	Task Queue (Celery)	مهام ثقيلة أو طويلة في نظام إنتاج موزّع
الفصل العاشر	FastAPI + Async + Process Pool	خدمة API بعدد ضخم من الطلبات المتزامنة
الفصل الحادي عشر	GPU Parallelism (PyTorch/TensorFlow)	تدريب/Inference لنماذج Deep Learning

مبادئ التصميم الأساسية

✅ أفضل الممارسات

حدد نوع المهمة أولًا (CPU-bound أم I/O-bound) قبل اختيار أي أداة — هذا القرار الأول يحدد كل ما بعده.

لا تحسّن أو توازِن قبل القياس الفعلي (Profiling & Benchmarking) — الحدس البشري عن مكان المشكلة غالبًا غلط.

ابدأ بالحل الأبسط (Sequential) وطوّر لـ Concurrency/Parallelism فقط لما تثبت الحاجة الفعلية بالقياس.

قلّل الحالة المشتركة (Shared State) قدر الإمكان — فضّل تمرير البيانات (Message Passing) على مشاركتها المباشرة.

صمّم كل عملية قابلة لإعادة المحاولة لتكون Idempotent من الأساس.

قواعد الأمان في Threading

- استخدم with lock: دايمًا بدل acquire()/release() اليدوية.
- التزم بترتيب ثابت لأخذ الأقفال المتعددة في كل الكود لتجنب Deadlock.
- قلّل حجم الـ Critical Section لأقصى درجة ممكنة.
- استخدم RLock بدل Lock العادي في الدوال المتداخلة التي تأخذ نفس القفل.

- فكّر في هياكل بيانات Thread-safe جاهزة (queue.Queue) قبل بناء Locking يدوي.

قواعد الأمان في Multiprocessing

- استخدم if __name__ == '__main__': دائمًا (ضروري خصوصًا على Windows).
- استخدم Pool أو ProcessPoolExecutor بدل إنشاء Processes منفردة يدويًا.
- قلّل حجم البيانات المنقولة بين Processes — مرر مسارات أو معرّفات بدلًا من بيانات ضخمة كاملة حيث أمكن.
- استخدم Shared Memory (Value/Array) للبيانات البسيطة، وManager للهياكل المعقدة.
- لا تفترض تحسنًا تلقائيًا بزيادة عدد Processes فوق عدد الـ CPU cores الفعلية.

قواعد الأمان في Async

- لا تستدعِ أبدًا مكتبة Synchronous بطيئة داخل async def — استخدم بديلها الـ Async المخصص (aiohttp, asyncpg, redis.asyncio).
- ضع Timeout واضحًا على كل عملية I/O خارجية بدون استثناء.
- استخدم asyncio.TaskGroup أو asyncio.gather لإدارة مجموعات المهام، لا تترك Tasks بلا متابعة.
- استخدم Connection Pools لقواعد البيانات والـ Cache بدلًا من فتح اتصال جديد لكل عملية.

قواعد الأداء العامة

القاعدة العملية	العامل
ابدأ بعدد الـ CPU cores الفعلية، ثم قِس واضبط	عدد Processes للمهام CPU-bound
أكبر من عدد الـ Cores، لكن حدده بالقياس الفعلي لا التخمين	عدد Threads للمهام I/O-bound
قلّله دائمًا؛ تكلفة Serialization حقيقية وقد تلغي فائدة التوازي	حجم البيانات بين Processes
حدد maxsize دائمًا لتفادي استهلاك ذاكرة غير محدود (Backpressure)	حجم Queue في أنظمة الإنتاج
استخدم Exponential Backoff دائمًا، مع حد أقصى لعدد المحاولات	إعادة المحاولة (Retry)

أخطاء شائعة يجب تجنبها دائمًا

> ⚠️ **خطأ شائع**
>
> استخدام Threading على أمل تسريع كود Python نقي حسابي ثقيل (الـ GIL يمنع ذلك — استخدم Multiprocessing).
>
> استخدام Multiprocessing لمهام I/O-bound بسيطة (تكلفة الإنشاء أكبر من الفائدة — استخدم Threading أو Async).
>
> نسيان تحرير Lock بعد استثناء (استخدم with دائمًا).
>
> أخذ أكثر من Lock بترتيب مختلف في أماكن مختلفة من الكود (سبب شائع جدًا للـ Deadlock).
>
> نسيان استدعاء result() على Future، مما يخفي أخطاء حقيقية حصلت داخل المهمة.

تصميم Retry بدون التأكد من كون العملية Idempotent.

Queue بلا maxsize في نظام إنتاج حقيقي، مما قد يؤدي لاستهلاك ذاكرة غير محدود.

Distributed Lock بدون مهلة انتهاء (TTL)، مما قد يعلّق النظام بالكامل عند تعطل الـ Worker الماسك للقفل.

قائمة تحقق قبل نشر أي نظام Concurrent/Parallel للإنتاج

41. هل حددت نوع كل مهمة بوضوح (CPU-bound أم I/O-bound) واخترت الأداة المناسبة؟
42. هل قِست الأداء فعليًا (Profiling/Benchmarking) بدل الاعتماد على التخمين؟
43. هل كل الأقفال (Locks) في الكود بترتيب ثابت لا يتغير بين الدوال المختلفة؟
44. هل كل Future/Task في الكود يتم استدعاء نتيجته أو متابعته بشكل صريح؟
45. هل كل عملية I/O خارجية لها Timeout واضح؟
46. هل كل عملية قابلة لإعادة المحاولة مصممة لتكون Idempotent؟
47. هل لديك Logging مفصّل بمعرّفات واضحة (Thread/Process/Trace ID) لتسهيل التتبع لاحقًا؟
48. هل حددت maxsize لأي Queue تستخدمه في الإنتاج؟
49. هل عدد الـ Workers (Threads/Processes) مبني على قياس فعلي، لا رقم عام مأخوذ من مقال؟

> 🏢 **من واقع الشركات**
>
> قائمة التحقق دي مش شرط تنطبق كل بنودها على كل مشروع صغير، لكنها نقطة انطلاق ممتازة لمراجعة أي نظام قبل نشره في بيئة إنتاج حقيقية تخدم مستخدمين فعليين.

اعتبارات قابلية التوسع (Scalability Considerations)

- Threading يتوسع رأسيًا بشكل محدود (عشرات الوحدات)، ومحصور داخل حدود الجهاز الواحد لأن الـ Threads تشارك ذاكرة نفس الـ Process.
- Multiprocessing يتوسع بعدد الـ CPU cores المتاحة على الجهاز، وهو الأساس الذي تُبنى فوقه أنظمة Distributed Computing عند الحاجة لتوسع أكبر من جهاز واحد.
- Async يوفر أفضل توسع رأسي لعدد الاتصالات I/O-bound المتزامنة على نفس الجهاز، لكنه يحتاج Multiprocessing (عدة Workers) لاستغلال أكثر من Core.
- Task Queues الموزّعة (Celery) وWorker Pools تتوسع أفقيًا بسهولة: زيادة الحمل تعني ببساطة زيادة عدد الـ Workers، وأحيانًا عبر عدة أجهزة.
- قلّل الحالة المشتركة (Shared State) واعتمد على Message Passing كل ما أمكن — الأنظمة الأقل اعتمادًا على Locking اليدوي المعقد تتوسع وتُصان بسهولة أكبر مع نمو الفريق وحجم الكود.

خلاصة الرحلة

من الفصل الأول لحد هنا، بنينا فهمًا متدرجًا: من المفاهيم الأساسية (Concurrency وParallelism)، لآليات نظام التشغيل (Process وThread وContext Switching)، لقيود اللغة نفسها (GIL)، لأدوات عملية (Threading, Multiprocessing, Async, concurrent.futures)، لأنماط تصميم أنظمة كاملة (Communication,

Production Patterns)، لمنهجية القياس والتصحيح. الهدف من كل ده مش حفظ الأدوات، لكن بناء القدرة على السؤال الصحيح في أي موقف: إيه طبيعة المهمة دي؟ وبناءً على الإجابة، القرار بيبقى واضح.

الفصل الخامس عشر والأخير هيجمع كل حاجة اتعلمناها في ٥ مشاريع Production متكاملة من الألف للياء: خط معالجة صور عالي الأداء، نظام معالجة بيانات متوازٍ، خدمة Batch Prediction موزّعة، واجهة برمجية لمعالجة مهام في الخلفية، وخدمة Inference لحظية — كل مشروع بيطبّق مجموعة من الأدوات والأنماط اللي اتعلمناها معًا في سياق واحد متكامل وواقعي.

الفصل الخامس عشر: مشاريع Production متكاملة

وصلنا للفصل الأخير. هنا مش هنتعلم مفاهيم جديدة، هنطبّق كل حاجة اتعلمناها في ٥ مشاريع متكاملة، كل واحد بيمثل سيناريو حقيقي شائع جدًا في عمل ML Engineer داخل شركة تقنية. كل مشروع بيدمج عدة أدوات من المذكرة مع بعض، وبيوضح إزاي القرارات النظرية بتتحول لكود إنتاج فعلي.

المشروع الأول: خط معالجة صور عالي الأداء

Pipeline بيستقبل صور خام، يعالجها (تغيير حجم، تطبيع، استخراج Features)، ويخزن النتائج — بيطبّق Multiprocessing للمعالجة CPU-bound، وMonitoring لقياس الأداء.

البنية والأدوات المستخدمة

- Multiprocessing (ProcessPoolExecutor) لتوزيع معالجة الصور على كل الـ CPU cores المتاحة (الفصل الخامس والسابع).
- Work Queue لتوزيع الصور على الـ Workers بشكل متوازن (الفصل الثامن).
- Profiling لتحديد المرحلة الأبطأ في المعالجة (الفصل التاسع).
- Docker لتغليف الخدمة وضمان تناسق البيئة بين التطوير والإنتاج.

```
from concurrent.futures import ProcessPoolExecutor
from PIL import Image
import time, logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')

:def process_image(path)
()start = time.time
img = Image.open(path).convert('RGB').resize((224, 224))
(Features استخراج ،(تطبيع إضافية معالجة ... #
duration = time.time() - start
return path, duration

:def run_pipeline(image_paths, max_workers=None)
[] = results
:with ProcessPoolExecutor(max_workers=max_workers) as executor
:for path, duration in executor.map(process_image, image_paths)
("{duration:.3f}s في {path} عولجت"f)logging.info
results.append((path, duration))
return results

:'__if __name__ == '__main
run_pipeline(image_paths=["img1.jpg", "img2.jpg"], max_workers=8)
```

> 🏢 **من واقع الشركات**
>
> تذكير من الفصل التاسع: عدد الـ Workers الأمثل هنا لازم يتحدد بقياس فعلي على حجم الصور وموارد السيرفر الحقيقية، مش بافتراض إن أكبر عدد ممكن هو الأفضل.

المشروع الثاني: نظام معالجة بياناتٍ متوازٍ

Pipeline لتحويل ملف بيانات ضخم (CSV) وتجهيزه كـ Features جاهزة للتدريب — بيطبّق Multiprocessing لتقسيم البيانات، وMessage Passing بين مراحل المعالجة.

```
from concurrent.futures import ProcessPoolExecutor
import pandas as pd, numpy as np

:def transform_chunk(chunk)
()chunk = chunk.dropna
/ chunk['normalized'] = (chunk['value'] - chunk['value'].mean())
()chunk['value'].std
return chunk

:def process_large_csv(filepath, num_workers=4)
df = pd.read_csv(filepath)
chunks = np.array_split(df, num_workers)
:with ProcessPoolExecutor(max_workers=num_workers) as executor
processed = list(executor.map(transform_chunk, chunks))
return pd.concat(processed, ignore_index=True)

:'__if __name__ == '__main
result = process_large_csv("large_dataset.csv", num_workers=4)
result.to_parquet("processed_features.parquet")
```

المشروع الثالث: خدمة Batch Prediction موزّعة

خدمة بتستقبل طلب لتنفيذ Inference على ملف كبير من العينات (بدل طلب واحد)، بتستخدم Celery لتنفيذ المهمة في الخلفية وTask Queue للتعامل مع الحمل الثقيل.

```
from celery import Celery
import numpy as np

app = Celery('batch_prediction', broker='redis://localhost:6379/0')

app.task(bind=True, max_retries=3)@
:def run_batch_prediction(self, file_path, job_id)
:try
data = load_data(file_path)
تقسيم البيانات ومعالجتها بالتوازي داخل نفس المهمة #
chunks = np.array_split(data, 4)
:with ProcessPoolExecutor(max_workers=4) as executor
predictions = list(executor.map(model.predict, chunks))
save_results(job_id, np.concatenate(predictions))
return {"status": "completed", "job_id": job_id}
:except Exception as exc
raise self.retry(exc=exc, countdown=2 ** self.request.retries)

:FastAPI من واجهة #
job = run_batch_prediction.delay(file_path="data.csv", job_id=str(uuid.uuid4())) #
يرجع job_id فورًا، والمستخدم يتابع الحالة لاحقًا #
```

من واقع الشركات

هذا المشروع يجمع Multiprocessing (داخل كل مهمة) مع Task Queue موزّع (Celery) على مستوى النظام ككل — نموذج مصغّر لهندسة متعددة المستويات (الفصلين الخامس والثاني عشر).

المشروع الرابع: واجهة برمجية لمعالجة مهام في الخلفية

API بتستقبل طلبات رفع ملفات (زي مستندات لتحليلها)، بترجع رد فوري، وبتعالج الملف في الخلفية باستخدام BackgroundTasks للمهام الخفيفة وCelery للمهام الثقيلة، مع Rate Limiting لحماية الخدمة.

```
from fastapi import FastAPI, UploadFile, BackgroundTasks, HTTPException
import time

()app = FastAPI
[] = request_log

:def rate_limit_check(max_per_minute=60)
()now = time.time
request_log[:] = [t for t in request_log if now - t < 60]
:if len(request_log) >= max_per_minute
("تجاوزت الحد المسموح من الطلبات"=status_code=429, detail)raise HTTPException
request_log.append(now)

:def log_upload(filename)
("{filename} :تم تسجيل رفع الملف"f)print

app.post("/upload")@
:async def upload_document(file: UploadFile, background_tasks: BackgroundTasks)
()rate_limit_check
()contents = await file.read
ثقيلة job_id = process_document.delay(contents, file.filename)   # Celery task
مهمة خفيفة فورية #        background_tasks.add_task(log_upload, file.filename)
return {"job_id": str(job_id), "status": "processing"}
```

المشروع الخامس: خدمة Inference لحظية

خدمة FastAPI كاملة لخدمة نموذج ML في الوقت الفعلي، بتجمع كل الأدوات: Async I/O لاستقبال الطلبات، Process Pool للـ Inference الثقيل، Redis للـ Caching، وMonitoring للأداء — الهندسة الأشمل في المذكرة كلها.

```
from fastapi import FastAPI
from concurrent.futures import ProcessPoolExecutor
import redis.asyncio as redis
import asyncio, json, hashlib, time

()app = FastAPI
process_pool = ProcessPoolExecutor(max_workers=4)
redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

:def run_inference(features)
منفصل Process Pool ثقيلة، بتشتغل في CPU-bound عملية #
return model.predict(features)
```

```
app.post("/predict")@
:async def predict(features: list[float])
()start = time.time
()cache_key = hashlib.sha256(json.dumps(features).encode()).hexdigest

cached = await redis_client.get(cache_key)
:if cached
return {"prediction": json.loads(cached), "cached": True}

()loop = asyncio.get_event_loop
result = await loop.run_in_executor(process_pool, run_inference, features)

await redis_client.set(cache_key, json.dumps(result), ex=3600)

duration = time.time() - start
("{duration:.3f}s استغرق Inference طلب"f)logging.info
return {"prediction": result, "cached": False}

app.get("/health")@
:()async def health
return {"status": "ok"}
```

النشر (Deployment)

```
مبسّط Dockerfile #
FROM python:3.12-slim
WORKDIR /app
. COPY requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
. . COPY
\ ,"CMD ["gunicorn", "main:app", "--workers", "4
\ ,"worker-class", "uvicorn.workers.UvicornWorker--"
["bind", "0.0.0.0:8000--"
```

Monitoring

في بيئة إنتاج حقيقية، الخدمة دي غالبًا هتتراقب عن طريق: مسار /health بيتفحص دوريًا من نظام الـ Orchestration (زي Kubernetes)؛ مقاييس أداء (Metrics) زي زمن الاستجابة ومعدل الطلبات؛ وLogging مركزي بمعرّفات واضحة (زي ما شرحنا في الفصل الثالث عشر) لتسهيل تتبع أي مشكلة.

خلاصة المشاريع الخمسة

الفصل المرجعي الأهم	الأداة الأساسية	المشروع
الفصل الخامس والثامن	Multiprocessing + Work Queue	خط معالجة صور
الفصل الخامس والحادي عشر	Multiprocessing + Pandas/NumPy	معالجة بيانات متوازية
الفصل الثاني عشر	Celery + Multiprocessing + Retry	Batch Prediction موزّعة
الفصل العاشر والثاني عشر	BackgroundTasks + Celery + Rate Limiting	API معالجة خلفية

الفصل المرجعي الأهم	الأداة الأساسية	المشروع
الفصل السادس والعاشر	FastAPI + Async + Process Pool + Redis	Inference لحظية

🏢 من واقع الشركات

لاحظ إن كل مشروع من الخمسة مش بيستخدم أداة واحدة بمعزل، لكن مزيج مدروس من عدة أدوات — وده جوهر الرسالة الأساسية للمذكرة كلها: القرار الصح مش 'إيه أفضل أداة Concurrency؟'، لكن 'إيه مزيج الأدوات المناسب لكل جزء من نظامي، بناءً على طبيعة كل جزء؟'

تحديات شائعة عبر المشاريع الخمسة

- تحديد التوازن الصحيح بين البساطة والتعقيد — مش كل مشروع محتاج كل أداة في المذكرة؛ استخدم أبسط حل يحقق المتطلبات فعليًا.
- قياس الأداء الفعلي لكل مشروع على بيئته الحقيقية بدل تعميم أرقام من مشروع لآخر.
- التعامل الصحيح مع الأعطال الجزئية (Partial Failures) في الأنظمة الموزّعة — جزء من النظام بيفشل بينما الباقي يستمر في العمل.

ملخص الفصل والمذكرة كاملة

- المشاريع الخمسة دي أمثلة واقعية لكيفية دمج Threading, Multiprocessing, Async, Task Queues, وCaching في أنظمة إنتاج حقيقية.
- كل قرار تصميم في المشاريع دي مبني على مبدأ واحد أساسي اتكرر طول المذكرة: حدد طبيعة المهمة (CPU-bound أم I/O-bound) أولًا، ثم اختر الأداة، ثم قِس الأداء فعليًا.
- الأنظمة الحقيقية نادرًا ما تعتمد على أداة واحدة — الهندسة الناضجة تجمع عدة أدوات معًا، كل واحدة في الطبقة المناسبة لها.
- الاستثمار في Monitoring وLogging وDebugging من بداية أي مشروع يوفر وقتًا هائلاً لاحقًا عند حدوث مشاكل حتمية في الإنتاج.

بهذا نكون قد أكملنا رحلة كاملة: من المفاهيم الأساسية للـ Concurrency والـ Parallelism، مرورًا بآليات نظام التشغيل، قيود اللغة، الأدوات العملية، أنماط التصميم، منهجية القياس والتصحيح، وأخيرًا مشاريع إنتاج متكاملة. الهدف النهائي لم يكن حفظ الأدوات، بل بناء القدرة على اتخاذ القرار الصحيح في أي موقف جديد تواجهه كـ Machine Learning Engineer داخل بيئة إنتاج حقيقية.